Introduction to Reinforcement Learning[1]

Zixian Yang (zixian@umich.edu) and Lei Ying (leiying@umich.edu)

The University of Michigan, Ann Arbor

[1]This draft is not well polished, and has many typos and missing references. Any comments/suggestions/corrections for improving the book will be greatly appreciated.

Chapter 1

Dynamic Programming and Bellman Equations

Dynamic programming (DP) is the mathematical foundation of reinforcement learning. In this chapter, we will introduce deterministic dynamic programming, stochastic dynamic programming, and the Bellman equation.

1.1 Introduction to Dynamic Programming

Dynamic programming is a method to solve sequential decision problems. Deterministic dynamic programming is for problems where the state transitions are deterministic. We first use a simple example to explain the key concepts and the reason it is computationally more efficient than native exclusive search.

1.1.1 Example: The Most Rewarding Path Problem

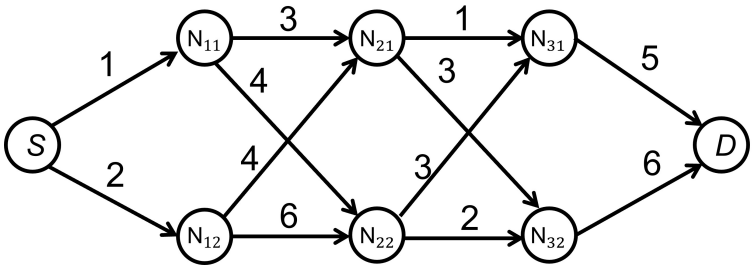


Figure 1.1: The most rewarding path problem.

We consider the most rewarding path problem as shown in Figure 1.1, which includes a source node S and a destination node D. The objective is to find a path from S to D such that the cumulative reward is maximized among all possible paths from S to D.

Exclusive Search

A naive solution to this problem is exclusive search, which enumerates all the possible paths from S to D, calculates the cumulative reward for each path and selects the one with the largest value. For example, the cumulative reward of path $\{S \to N_{11} \to N_{22} \to N_{31} \to D\}$ is $1+4+3+5=13$. The computational complexity of this approach depends on the number of possible paths from S to D, which, in this example, is $2 \times 2 \times 2 \times 1 = 8$. Therefore, we need to calculate the cumulative rewards of all 8 paths to find out the maximum reward and each requires three additions. If we increase the path lengths so that it takes T steps from the source to the destination and each node, except the destination, connects to two next-hop nodes, then the computational complexity becomes $T2^T$, where 2^T is the number of possible paths and T is the number of additions for calculating the total reward of a path.

Dynamic Programming (DP)

Instead of going from the source to the destination, i.e., forward search, let us look at another method that starts from the destination and then goes back to the source, i.e., a backward search algorithm. In a typical dynamic programming problem, the problem is divided into subproblems at multiple stages as shown in Figure 1.2

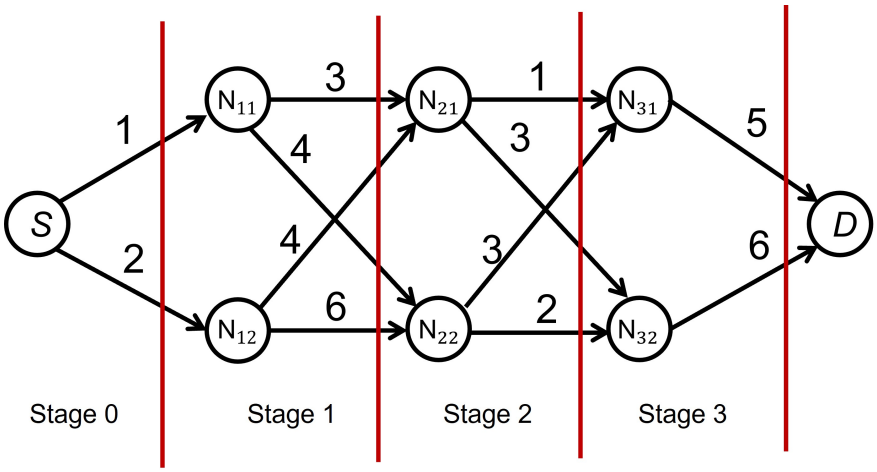


Figure 1.2: Stage 3

Stage 3: The first step of the backward search or dynamic programming is to calculate the maximum reward from nodes $N_{3,1}$ and $N_{3,2}$ to destination D. Since there is only one feasible path from node $N_{3,1}$ ($N_{3,2}$) to destination D, the calculation is straightforward. Let V_{ij} denote the maximum cumulative reward from node $N_{i,j}$ to destination D. We can immediately obtain

$$V_{31} = 5 \quad \text{and} \quad V_{32} = 6.$$

Stage 2: We then move one-step backward to stage 2 and consider the most rewarding paths from nodes N_{21} and N_{22}. From node N_{21}, we have two choices for the next hop: node N_{31} or node N_{32} :

- the cumulative reward of going through node N_{31} is $1+5=6$ and

- the cumulative reward of going through node N_{32} is $3 + 6 = 9$.

Therefore, we can conclude that the most rewarding path starting from node N_{21} has a total reward of 9. Note that for each node, it only needs two additions and one comparison. Similarly, we can obtain that the maximum cumulative reward starting from node N_{22} is 8 as shown in Figure 1.3.

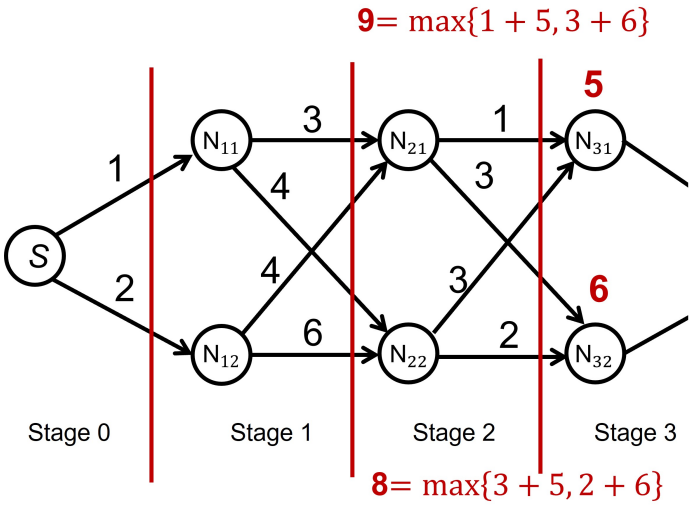


Figure 1.3: Stage 2

Stage 1: We can then move further back to stage 1. Consider the most rewarding path from node N_{11}. A key observation we have is that to calculate the cumulative reward of the most rewarding path from node N_{11} to D, it is sufficient to know the total rewards of the most rewarding paths from node N_{21} and N_{22} to destination D, respectively. This is because node N_{11} has only two possible next hops, node N_{21} or node N_{22}. Selecting node N_{21} as the next hop, we first collect a reward of 3 over edge (N_{11}, N_{21}) and a reward of 9 following the most rewarding path from node N_{21} to destination D. So the total reward is $3 + 9 = 12$ by choosing node N_{21} as the next hop. If we choose to go to node N_{22} from node N_{11}, then the total reward would be $4 + 8 = 12$, where 4 is reward over edge (N_{11}, N_{22}) and 8 is the maximum reward from N_{22} to destination D. We observe that to calculate the maximum reward from node N_{11} to destination D, it is sufficient to know the maximum rewards from node N_{21} and N_{22}. This reduces not only the information we need at stage 1 but also the amount of calculation required. Similarly, it is easy to see that the most rewarding path from node N_{21} has a total reward of

$$\max\{4 + 9, 6 + 8\} = 14.$$

The calculation at each node only requires two additions and one comparison, exactly the same as in stage 2.

Stage 0: We finally reach stage 0 as shown in Figure 1.5, the source node. The agent compares the options of going to node N_{11} or node N_{12} for the maximum rewarding path and can conclude the maximum reward is

$$\max\{1 + 12, 2 + 14\} = 16.$$

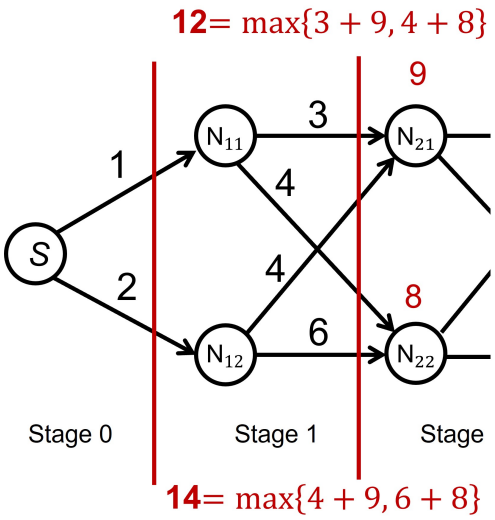


Figure 1.4: Stage 1

In the backward search, the calculation involves 5 comparisons and 10 additions, so 15 elementary operations instead of more than 24 in the exclusive search.

The Computational Complexity of Exclusive Search versus Dynamic Programming

Consider a problem with T stages. In general, the computational complexity of the exclusive search (forward search) is exponential in T while the computational complexity of dynamic programming is linear in T.

1.1.2 Principle of Optimality and the Bellman Equation

Principle of optimality and the Bellman equation are at the center of DP and RL [5]. The concepts were introduced by Richard E. Bellman in 1950s. We will first introduce both concepts using the previous example. We will then introduce the general formulation in the next section.

The principle of optimality says a subsolution of an optimal solution is the optimal solution for the corresponding subproblem. The backward search uses this principle of optimality. For example, When calculating the most rewarding path from source S, the backward search utilizes the fact that the most rewarding path from node N_{11} has reward 12 and the most rewarding path from node N_{12} has reward 14. The most reward path from node S has to include either the most rewarding path from node N_{11} to the destination or the one from node N_{12} to the destination, which leads to the following equation:

The cumulative reward of the most rewarding path from S to $D = \max\{1 + 12, 2 + 14\} = 16$.

In this example, letting $V_k(j)$ denote the maximum cumulative reward from node j at stage k to destination D (or called the value of state j at stage k), the principle of optimality can be expressed as the following mathematical equation, called the Bellman equation:

$$V_k(j) = \max_{i \in \mathcal{N}_k(j)} \left(r_k(j,i) + V_{k+1}(i) \right),$$

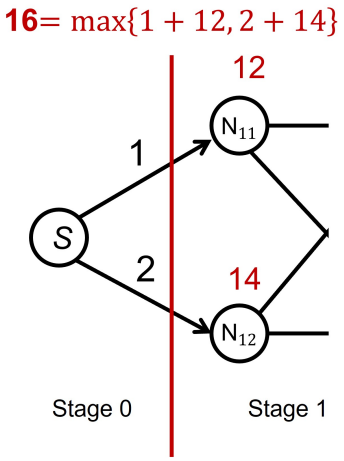


Figure 1.5: Stage 0

where $\mathcal{N}_k(j)$ is the set of node j's neighbors at stage $k+1$, $r_k(j,i)$ is the reward of edge (j,i). The backward calculation is a repeatedly use of the Bellman equation, starting from the final stage of the problem.

We next provide a simple proof of the Bellman equation by considering the most rewarding path problem.

"$\geq$:" We note two facts: (1) the most rewarding path from node j has to go through some node in $\mathcal{N}_k(j)$ and (2) a path, which goes from node j to node i and then follows the most rewarding path starting from node i, is a feasible path from node j to destination D. Therefore, we have

$$V_k(j) \geq r_k(j,i) + V_{k+1}(i) \quad \forall\ i \in \mathcal{N}_k(j),$$

from which we conclude that

$$V_k(j) \geq \max_{i \in \mathcal{N}_k(j)} r_k(j,i) + V_{k+1}(i).$$

"$\leq$:" Now suppose that the most rewarding path starting from node j goes through node i^*. Let $\tilde{V}_{k+1}(i^*)$ denote the cumulative reward from i^* to D by following this most rewarding path. Then by definition, we immediately obtain that

$$V_k(j) = r_k(j,i^*) + \tilde{V}_{k+1}(i^*).$$

According to the definition of $V_{k+1}(i^*)$, we have $V_{k+1}(i^*) \geq \tilde{V}_{k+1}(i^*)$ and

$$\begin{aligned} V_k(j) =& r_k(j,i^*) + \tilde{V}_{k+1}(i^*) \\ \leq& r_k(j,i^*) + V_{k+1}(i^*) \\ \leq& \max_{i \in \mathcal{N}_k(j)} \left(r_k(j,i) + V_{k+1}(i) \right). \end{aligned}$$

1.2 Finite-Horizon Deterministic DP

In this section, we introduce the general form of a finite-horizon deterministic DP. Consider the following system equation:

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, 1, \ldots, T-1,$$

where $x_k \in \mathcal{X}_k$ is the system state at stage k, $\mathcal{X}_k$ is the set of possible states at stage k, $u_k \in \mathcal{U}_k(x_k)$ is action at stage k, and $\mathcal{U}_k(x_k)$ is the set of possible actions in state x_k at stage k. Note that in many RL applications, different stages represent different time instances and x_k and u_k are the state and action at time k, respectively.

For given initial state x_0, the total reward over action sequence $(u_0, u_1, \ldots, u_{N-1})$ is defined to be

$$V(x_0; u_0, \ldots, u_{N-1}) = r_N(x_N) + \sum_{k=0}^{N-1} r_k(x_k, u_k).$$

The maximum cumulative reward (also called the value of state x_0) is defined to be

$$V_0(x_0) = \max_{u_0, \ldots, u_{N-1}} V(x_0; u_0, \ldots, u_{N-1}).$$

We can similarly define the value of state x_k as

$$V_k(x_k) = \max_{u_k, \ldots, u_{N-1}} \left(r_N(x_N) + \sum_{l=k}^{N-1} r_l(x_l, u_l) \right).$$

To solve the optimal total reward or value function $V_k(x)$, we can use backward computation based on the Bellman equation. The first step is trivial:

$$V_N(x_N) = r_N(x_N).$$

For example, in the most-rewarding path problem, we have $r_N(x_N) = 0$.

Next, according to the principle of optimality, we have

$$V_{N-1}(x_{N-1}) = \max_{u_{N-1}} \left(r_{N-1}(x_{N-1}, u_{N-1}) + V_N(f_{N-1}(x_{N-1}, u_{N-1})) \right).$$

Note that $V_N(\cdot)$ is known when we calculate $V_{N-1}(\cdot)$ and the reward function $r_{N-1}(\cdot, \cdot)$ and the system equation $f_{N-1}(\cdot, \cdot)$ are given to us.[1] Therefore, we can easily calculate $V_{N-1}(\cdot)$. In general, we can use the following Bellman equation to recursively solve $V_k(\cdot)$

$$V_k(x_k) = \max_{u_k} \left(r_k(x_k, u_k) + V_{k+1}(f_k(x_k, u_k)) \right).$$

After obtaining $V_k(\cdot)$, we can further find the optimal action at stage k, denoted by u_k^*, by solving

$$u_k^* \in \arg \max_{u_k \in \mathcal{U}_k(x_k)} \left(r_k(x_k, u_k) + V_{k+1}(f_k(x_k, u_k)) \right).$$

This procedure is often called a backward-forward method, where the backward search gives us the values and the forward search gives us the optimal actions. The procedure is summarized below.

[1] We will deal with unknown reward functions and system equations later.

Deterministic Dynamic Programming

- Initialization: Set $V_N(x_N) = r_N(x_N)$ for all possible $x_N \in \mathcal{X}_N$.
- Backward Search: For $k = 1, \cdots, N$, calculate

$$V_{N-k}(x_{N-k}) = \max_{u_{N-k} \in \mathcal{U}_{N-k}(x_{N-k})} (r_{N-k}(x_{N-k}, u_{N-k}) + V_{N-k+1}(f_{N-k}(x_{N-k}, u_{N-k})))$$

  for all $x_{N-k} \in \mathcal{X}_{N-k}$.
- Forward Search: For $k = 0, 1, \cdots, N$, calculate

$$u_k^* \in \arg \max_{u_k \in \mathcal{U}_k(x_k)} (r_k(x_k, u_k) + V_{k+1}(f_k(x_k, u_k)))$$

  for all $x_k \in \mathcal{X}_k$.
- Output the optimal actions $\{u_k^*\}_{k=0,\cdots,N}$.

1.3 Examples and Codes for Deterministic DP

In this section, we will present a few examples that can be solved using deterministic DP. The key to solving these problems is to formulate the problems as deterministic DP by properly defining the reward, stage, action space, state space and system equation. After the problem is correctly formulated, solving it becomes a routine practice of the DP algorithm.

We provide a pseudo code for the implementation of the backward search for deterministic DP, which is shown in Algorithm 1.

Algorithm 1 Backward Search for Deterministic DP

Input: number of stages N; number of states S; reward function $r(k, x, u)$ and transition function $f(k, x, u)$ for stage k, state x, and action u; the value function for the last stage $v_N(x)$ for state x; a function $U(k, x)$ that returns a list of possible actions for state x at stage k.

```
V = np.zeros((N + 1, S))        # an 2D array
for x in range(S):
  V[N][x] = v_N(x)        # V[N][x] is set to a known value for all x = 0, ..., S − 1
for k in range(N − 1, −1, −1):        # from N − 1 to 0
  for x in range(S):
    action_values = []
    for u in U(k, x):
      action_values.append(r(k, x, u) + V[k + 1][f(k, x, u)])
    V[k][x] = max(action_values)

return V        # V is the value function.
```

Rod cutting problem

Given a rod of length 4 and prices of rod pieces as shown in Table 1.1, we want to find the most profitable way to cut the rod.

Length	1	2	3	4
Price	1	4	6	7

Table 1.1: The prices of different rod pieces

To formulate the problem as a deterministic DP, we define

- Stage k : the $k+1$th cut.
- State x_k : the remaining length after the kth cut.
- Action u_k : the length of the rod piece cut at stage k.
- Reward $r_k(x_k, u_k)$: the price of the kth rod piece.
- State transition: $x_{k+1} = x_k - u_k$ for $x_k \geq u_k$.

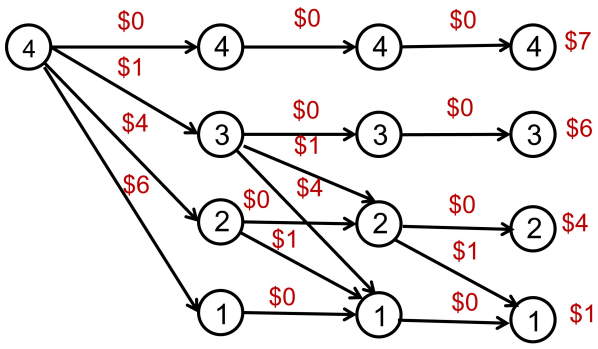


Figure 1.6: A pictorial description of the DP formulation of the rod cutting problem

The pictorial description of the problem can be found in Figure 1.6. Following the backward search, we can calculate the value function as shown in Table 1.2. The maximum value is obtained by following the path $4 \to 2 \to 2 \to 2$, which is to cut the rod into two pieces with length 2 each.

stage	0	1	2	3
State 4	\$8	\$7	\$7	\$7
State 3	-	\$6	\$6	\$6
State 2	-	\$4	\$4	\$4
State 1	-	\$1	\$1	\$1

Table 1.2: The value function $V_k(x)$ of the rod cutting problem

The reader may have noticed that the representation in 1.6 is overly complicated because we enforce that each path has a length of three to make the problem match the definition of finite-horizon deterministic DP. We can in fact simplify the representation slightly by enforcing that the agent has to cut the rod unless the remaining length is zero. We can then obtain an alternative representation as shown in Figure 1.7. The value function is summarized in Table 1.3. The path to obtain \$8 is $4 \to 2 \to 0$. Again, the optimal solution is to cut the rod into two pieces of length 2.

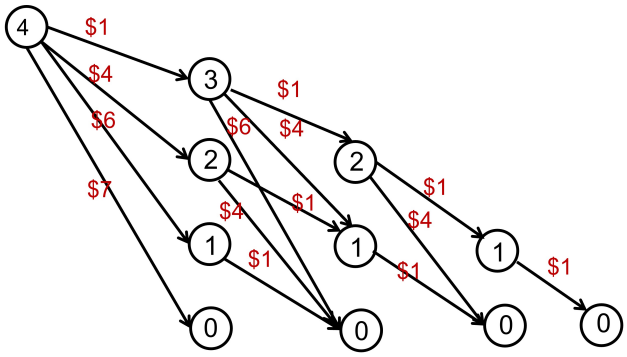


Figure 1.7: A pictorial description of an alternative DP formulation of the rod cutting problem

stage	0	1	2	3	4
State 4	\$8	-	-	-	-
State 3	-	\$6	-	-	-
State 2	-	\$4	\$4	-	-
State 1	-	\$1	\$1	\$1	-
State 0	-	\$0	\$0	\$0	\$0

Table 1.3: The value function $V_k(x)$ of the rod cutting problem

The Exclusive Search versus Dynamic Programming

A key reason that DP reduces the computational complexity compared with exclusive search is that it repeatedly uses the solutions to subproblems instead of "resolving" these problems. Comparing with the exclusive search which considers all possibilities as shown in Figure 1.8. We can see that DP merges the repeated states on each stage (e.g. states 0 and 1 at stage 2). While the reduction on this example is not much, the reduction becomes very significant as the number of stages increases.

0-1 knapsack problem

The 0-1 knapsack problem [13] can be described as follows. Given n items, each with a weight $w_k > 0$ and a value v_k, $k = 0, 1, ..., n-1$. Given a knapsack that has a limited weight capacity $W > 0$, determine which items to include so that the total weight is less than or equal to the given limit W and the total value is as large as possible. Assume that all weights $w_k, k = 0, 1, ..., n-1$ and W are integers. Note that $v_k \in \mathbb{R}, k = 0, 1, ..., n-1$ are not necessarily integers.

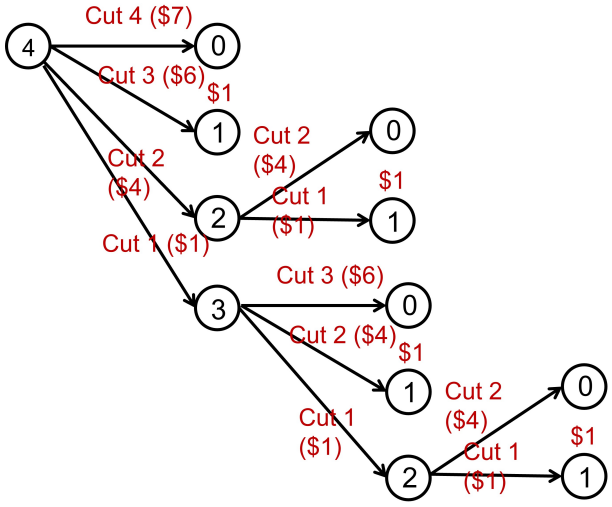


Figure 1.8: A pictorial description of exclusive search of the rod cutting problem

We can formulate the 0-1 knapsack problem into a finite-horizon deterministic DP problem as follows.

- Stage k: the time when we make decision on item k.

  We determine whether to include each item one by one. Let stage k be the time when we decide whether to include item k or not, $k = 0, 1, ..., n-1$. Let stage n be the final stage after all decisions.

- State x_k: the remaining weight capacity of the knapsack at stage k.

  By assumption, x_k is an integer between 0 and W.

- Action u_k:

  - $u_k = 1$: to include item k in the knapsack;
  - $u_k = 0$: not to include item k in the knapsack.

  Note that if $w_k > x_k$, i.e., the weight of item k is larger than the remaining weight capacity of the knapsack, then $u_k = 0$.

- Reward $r_k(x_k, u_k)$:

  $r_k(x_k, u_k) = v_k$ if $u_k = 1$ and $w_k \leq x_k$; otherwise, $r_k(x_k, u_k) = 0$.

- State transition function $f_k(x_k, u_k)$:

$$x_{k+1} = f_k(x_k, u_k) = \begin{cases} x_k - w_k, & u_k = 1, w_k \leq x_k \\ x_k, & u_k = 0. \end{cases}$$

The objective of this problem is to maximize the total reward $\sum_{k=0}^{n-1} r_k(x_k, u_k)$, i.e., the total value in the knapsack, by finding the optimal sequence of actions $u_0, \ldots . u_{n-1}$. The value function, $V_k(x_k)$, for state x_k at stage k, is defined to be

$$V_k(x_k) = \max_{u_k, \ldots u_{n-1}} \sum_{j=k}^{n-1} r_j(x_j, u_j).$$

The optimal value function $V_k(x_k)$ can be interpreted as the largest total value we can obtain given remaining weight capacity x_k and items $k, k+1, ..., n-1$. Note that $V_n(s_n) = 0$ by definition. Next, we can write down the Bellman equation for this problem. Let

$$\mathcal{U}_k(x_k) = \begin{cases} \{1, 0\} & x_k \geq w_k \\ \{0\} & x_k < w_k. \end{cases}$$

For $k = 0, 1, ..., n-1$, we have

$$\begin{aligned} V_k(x_k) &= \max_{u_k \in \mathcal{U}_k(x_k)} \{r_k(x_k, u_k) + V_{k+1}(x_{k+1})\} \\ &= \begin{cases} \max\{v_k + V_{k+1}(x_k - w_k), V_{k+1}(x_k)\} & x_k \geq w_k \\ V_{k+1}(x_k) & x_k < w_k. \end{cases} \end{aligned} \tag{1.1}$$

Then we can recursively solve $V_k(\cdot)$ by using the Bellman equation (1.1). The implementation code can be found in our GitHub repository [1] in the folder "knapsack-problem".

Shortest path problem

We are given an $m \times n$ grid world with nonpositive rewards inside each square. Consider an agent walking from the source to the destination, where the source is always located at the lower-left corner (i.e. square $(m-1, 0)$, and the destination is always located at the upper-right corner (i.e. square $(0, n-1)$). When the agent reaches a square, the agent receives the nonpositive reward (or penalty) inside the square. The objective of this problem is to find a path from the source to the destination such that the total reward is maximized. This can also be interpreted as finding the shortest path if we view the absolute value of the reward as the number of steps needed to cross the square. For example, consider the grid world in Fig. 1.9, where the red squares are the source and the destination.

This problem can be formulated as a finite-horizon deterministic DP problem. First, we notice that the number of squares on the path that maximizes the total reward should not exceed mn, including both the source and destination. This is because any path with a loop is suboptimal and the reward can be increased by removing the loop. Hence, it suffices to consider a finite horizon problem with fixed horizon $mn-1$ and we allow the agent to stay in the current grid for each step. We define the DP problem as follows:

- Stage k: the time when the agent makes the $(k+1)^{\text{th}}$ decision, $k = 0, 1, \ldots, mn-2$. Stage $mn-1$ is the stage after all $mn-1$ decisions are made.
- State x_k: the square where the agent is located at stage k.

  For example, $x_k = (2, 0)$ means that the agent is located in the square of row 2, column 0. The initial state is $x_0 = (m-1, 0)$, which is the starting point of the agent.

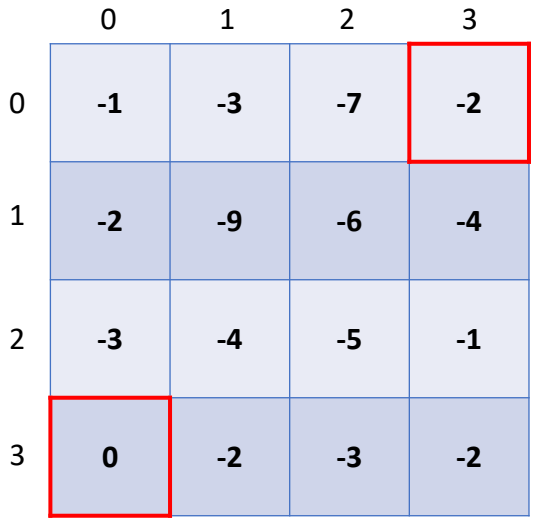


Figure 1.9: Shortest path problem

- Action u_k:

  $u_k = 0$: stay, $u_k = 1$: up, $u_k = 2$: right, $u_k = 3$: down, and $u_k = 4$: left.

  Note that if the agent is at the border, then they cannot move in some of the directions. For example, if the agent is in square $(3, 3)$ in Fig. 1.9, they cannot move right or down.

- State transition function: Under a feasible action u_k

$$x_{k+1} = f_k(x_k, u_k) = \begin{cases} (x_{k,i}, x_{k,j}), & u_k = 0 \\ (x_{k,i} - 1, x_{k,j}), & u_k = 1 \\ (x_{k,i}, x_{k,j} + 1), & u_k = 2 \\ (x_{k,i} + 1, x_{k,j}), & u_k = 3 \\ (x_{k,i}, x_{k,j} - 1), & u_k = 4, \end{cases}$$

  where $(x_{k,i}, x_{k,j})$ is the coordinate of the state x_k. For example, if $x_k = (3, 0)$, $u_k = 1$, then $x_{k+1} = f_k(x_k, u_k) = (2, 0)$.

- Reward $r_k(x_k, u_k)$: the reward obtained by taking action u_k in state x_k at stage k.

  For $k = 0, \cdots, mn - 2$: If $u_k = 0$, then $r_k(x_k, u_k) = 0$. Otherwise, $r_k(x_k, u_k)$ is the reward associated with square x_{k+1}. In other words, the agent receives the reward of the square the agent moves into and receives no reward for staying in the current square. Consider the example in Fig. 1.9. If $x_k = (3, 0)$ and $u_k = 1$, $r_k((3, 0), 0) = -3$.

  For $k = mn - 1$, $r_k(x_k, u_k) = 0$ if $x_{mn-1} = (0, n - 1)$, i.e., the destination is reached and $r_k(x_k, u_k) = -\infty$ otherwise.

The objective of the prolem is to maximize the total reward $\sum_{k=0}^{mn-2} r_k(x_k, u_k)$. Then we can define the value function in this problem. Let $V_k(x_k)$ denote the optimal value function for state

x_k at stage k, defined by

$$V_k(x_k) = \max_{u_k,...u_{mn-2}} \sum_{j=k}^{mn-1} r_j(x_j, u_j),$$

where by definition,

$$V_{mn-1}(x_{mn-1}) = \begin{cases} 0, & x_{mn-1} = (0, n-1) \\ -\infty, & \text{else,} \end{cases}.$$

Next, we can write down the Bellman equation for this problem. Let $\mathcal{U}_k(x_k)$ be the set of possible actions in state x_k. Then for $k = 0, 1, ..., mn-2$, we have

$$V_k(x_k) = \max_{u_k \in \mathcal{U}_k(x_k)} \{r_k(x_k, u_k) + V_{k+1}(x_{k+1})\}. \tag{1.2}$$

Then we can recursively solve $V_k(\cdot)$ using the Bellman equation (1.2). After obtaining the value function $V_k(\cdot)$, we can find the sequence of actions that maximize the total reward by calculating forward with $V_k(\cdot)$. Finally, we can drop the "stay" action and obtain the shortest path by recording the state that the agent visited. For example, in Fig. 1.9, the shortest path from [3,0] to [0,3] is given by the following list:

```
[[3,0],[3,1],[3,2],[3,3],[2,3],[1,3],[0,3]].
```

The implementation code can be found in our GitHub repository [1] in the folder "shortest-path-problem".

1.4 Finite-Horizon Stochastic DP

In the previous section, we considered deterministic DP where the state transitions are deterministic. In RL, it is very common that the environment is stochastic so that the state transitions are also stochastic. To introduce finite-horizon stochastic DP, we consider a variation of the most rewarding path problem.

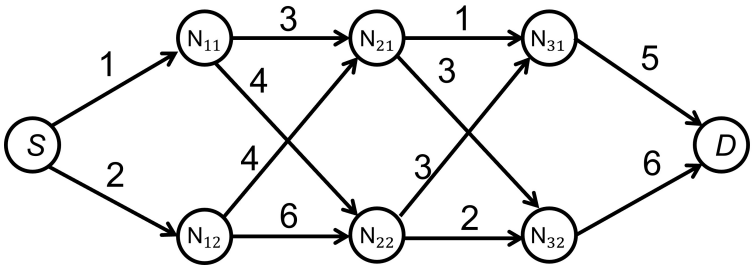


Figure 1.10: The most rewarding path problem.

Recall the most rewarding path problem (Figure 1.10). The objective is to find a path from S to D such that the cumulative reward is maximized. We now assume the environment has disturbance

so that the result of an action is stochastic. In particular, we assume the agent has two actions to choose from {up, down}. The result of an action is probabilistic as follows:

$$u_k = \text{up} \quad \to \begin{cases} \Pr(x_{k+1} = N_{k+1,1}) = 0.6 \\ \Pr(x_{k+1} = N_{k+1,2}) = 0.4 \end{cases} \quad \text{and } u_k = \text{down} \quad \to \begin{cases} \Pr(x_{k+1} = N_{k+1,1}) = 0.4 \\ \Pr(x_{k+1} = N_{k+1,2}) = 0.6 \end{cases}.$$

In other words, an action is correctly executed with probability 0.6.

We note that the solution to a deterministic DP is a fixed sequence of actions $(u_0^*, u_1^*, \ldots, u_{N-1}^*)$. For example, the solution to the deterministic version of the problem is

$$(\text{down}, \text{down}, \text{down}, \text{down}).$$

When implementing this solution, we know exactly the location of the agent and the reward the agent collects.

In a stochastic DP, since the agent's location is stochastic, instead of a fixed sequence of actions, the agent needs to pick the best action depending on the state the agent is in, i.e. finding state-dependent actions (also called a policy):

$$\mu_k(x_k), \ \forall x_k,$$

where μ_k is a function that maps a state x_k in the state space at stage k, $\mathcal{X}_k$ to an action in the action space for state x_k at stage k, $\mathcal{U}_k(x_k)$.

In a stochastic DP, the next state x_{k+1} given current state x_k and action u_k is a random variable following distribution $p(x_{k+1}|x_k, u_k)$ (also called transition kernel). For given initial state x_0 and policy $(\mu_0, \mu_1, \ldots, \mu_{N-1})$, the *expected* total reward (also called value) of the policy is defined to be

$$V(x_0; \mu_0, \ldots, \mu_{N-1}) = \mathbb{E}\left[r_N(x_N) + \sum_{k=0}^{N-1} r_k(x_k, \mu_k(x_k)) \right].$$

The value of the optimal policy for initial state x_0 is defined to be

$$V_0(x_0) = \max_{\mu_0, \ldots, \mu_{N-1}} V(x_0; \mu_0, \ldots, \mu_{N-1}).$$

We often use π to denote a policy, so the problem above can be written as

$$V_0(x_0) = \max_{\pi} V(x_0; \pi).$$

Similarly, we define the value of state x_k at stage k to be

$$V_k(x_k) = \max_{\mu_k, \ldots, \mu_{N-1}} \mathbb{E}\left[r_N(x_N) + \sum_{l=k}^{N-1} r_l(x_l, \mu_l(x_l)) \right].$$

We can similarly obtain the Bellman equation for stochastic DP

$$\begin{aligned} V_k(x_k) &= \max_{u_k} \mathbb{E}\left[r_k(x_k, u_k) + V_{k+1}(x_{k+1}) \right] \\ &= \max_{u_k} \sum_r r \Pr(r_k = r | x_k, u_k) + \sum_x V_{k+1}(x) \Pr(x_{k+1} = x | x_k, u_k). \end{aligned}$$

We will next use the most rewarding path example (the stochastic version) to illustrate the use of the Bellman equation for solving finite-horizon stochastic DP. In the discussion below, we will call

$$\sum_r r \Pr(r_k = r|x_k, u_k) + \sum_x V_{k+1}(x) \Pr(x_{k+1} = x|x_k, u_k)$$

as the action value function with action u_k (it is also called the Q-function in Q-learning, to be introduced in a later chapter). This is the reward agent receives from stage k to stage N by taking action u_k and then following the optimal policy after that.

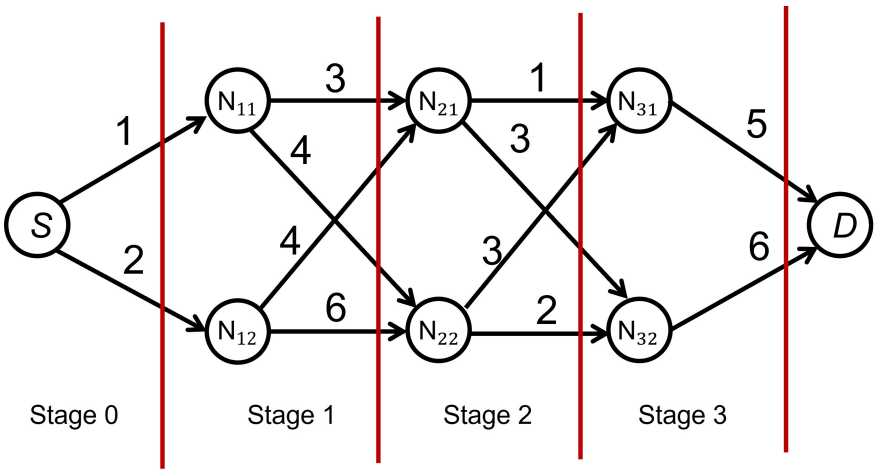


Figure 1.11: The most rewarding path problem

Stage 3: A node at stage 3 has only one possible path to reach destination D. Therefore, there is no randomness in the result of an action. We have

$$V_3(N_{31}) = 5 \text{ and } V_3(N_{32}) = 6.$$

Stage 2: We first study the action agent should take at node N_{21} by considering the following two cases:

- Action-value-function with action Up is $0.6 \times (1 + 5) + 0.4 \times (3 + 6) = 3.6 + 3.6 = 7.2$.
- Action-value-function with action Down is $0.4 \times (1 + 5) + 0.6 \times (3 + 6) = 7.8$.

Therefore, action down results in a higher expected reward. We have

$$V_2(N_{21}) = 7.8 \quad \text{and} \quad \mu_2^*(N_{21}) = \textbf{down}.$$

Similarly, we can find out the optimal action to be taken at node N_{22} by considering the following two cases;

- Action-value-function with action Up is $0.6 \times (3 + 5) + 0.4 \times (2 + 6) = 8$.
- Action-value-function with Down is $0.4 \times 8 + 0.6 \times 8 = 8$.

Since the expected rewards are the same for both actions, so both are optimal and

$$V_2(N_{22}) = 8.$$

Stage 1: At stage 1, we can repeat the same calculations. At node N_{11}, we have

- Action-value-function with action Up is $0.6 \times (3 + 7.8) + 0.4 \times (4 + 8) = 11.28$.
- Action-value-function with action Down is $0.4 \times (3 + 7.8) + 0.6 \times (4 + 8) = 11.52$.

Therefore, we have

$$V_1(N_{11}) = 11.52 \quad \text{and} \quad \mu_1^*(N_{11}) = \textbf{down}.$$

At node N_{12}, we have

- Action-value-function (Up): $0.6 \times (4 + 7.8) + 0.4 \times (6 + 8) = 12.68$.
- Action-value-function (Down): $0.4 \times (4 + 7.8) + 0.6 \times (6 + 8) = 13.12$.

So

$$V_1(N_{12}) = 13.12 \quad \text{and} \quad \mu_1^*(N_{12}) = \textbf{down}.$$

Stage 0: At stage 0, we can now find out the maximum expecte reward by comparing

- Action-value-function with action Up: $0.6 \times (1 + 11.52) + 0.4 \times (2 + 13.12) = 13.56$
- Action-value-function with action Down: $0.4 \times (1 + 11.52) + 0.6 \times (2 + 13.12) = 14.08$

We can conclude

$$V(S) = 14.08 \quad \text{and} \quad \mu_0^*(S) = \textbf{down}.$$

The value functions and the state-dependent optimal actions are shown in Figure 1.12.

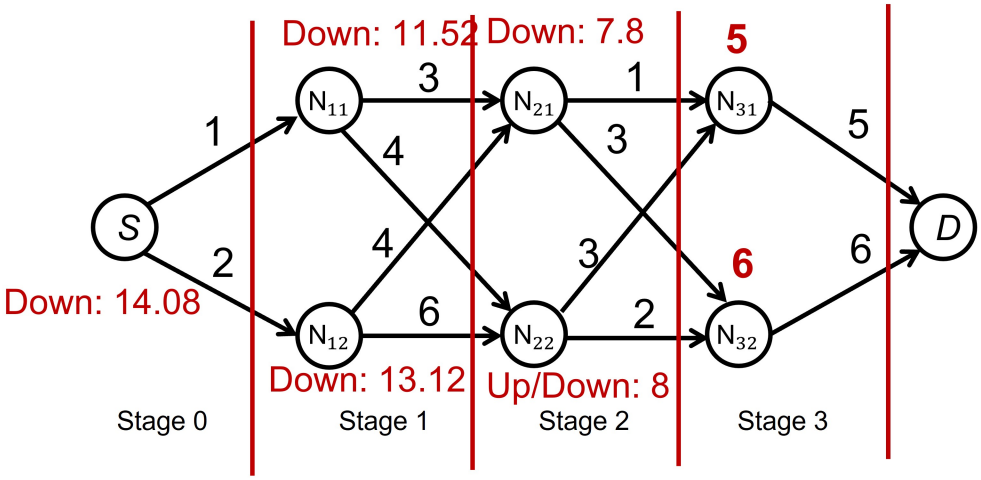


Figure 1.12: The most rewarding path problem

1.5 Examples and Codes for Stochastic DP

In this section, we will present a few examples and codes for stochastic DP. First, we provide a pseudo code for the implementation of the backward search for stochastic DP, which is shown in Algorithm 2.

Algorithm 2 Backward Search for Stochastic DP

Input: number of stages N; number of states S; a function that outputs the probability of reward $Pr_r(k, r, x, u)$ given stage k, reward r, state x, and action u; a function that outputs the transition probability $Pr_t(k, x, u, x_next)$ given stage k, current state x, current action u, and next state x_next; the value function for the last stage $v_N(x)$ for state x; a function $U(k, x)$ that returns a list of possible actions for state x at stage k; a list R of possible rewards.

V = np.zeros($(N + 1, S)$) # an 2D array
for x in range(S):
  $V[N][x] = v_N(x)$ # $V[N][x]$ is set to a known value for all $x = 0, \ldots, S - 1$
for k in range($N - 1, -1, -1$): # from $N - 1$ to 0
  for x in range(S):
    action_values = []
    for u in $U(k, x)$:
      mean_reward = sum($[r * Pr_r(k, r, x, u)$ for r in $R]$)
      future_value = sum($[V[k + 1][x_next] * Pr_t(k, x, u, x_next)$ for x_next in range(S)])
      action_values.append(mean_reward + future_value)
    $V[k][x]$ = max(action_values)

return V # V is the value function.

A betting game

You have \$2 and have to play a game 3 times. For each game, the chance of winning is 0.4 and the money you bet is doubled, and chance of losing is 0.6 and you lose the money you bet. You can only bet an integer dollar amount, including betting \$0. The goal is to find a policy that maximizes your chance of ending up with \$4 or more.

We can formulate the problem above as a stochastic DP:

- Stage k: the $k + 1$th game (for $k = 0, 1, 2, 3$)
- State x_k : the money available at the beginning of game $k + 1$ or after game k
- Action u_k : the amount of money the agent bets in game $k + 1$. Again, u_k has to be a nonnegative integer, including zero.
- State transition probabilities: given any $u_k \leq x_k$,

$$\Pr(x_{k+1} = x_k + u_k | x_k, u_k) = 0.4 \quad \text{and} \quad \Pr(x_{k+1} = x_k - u_k | x_k, u_k) = 0.6$$

- Reward at stage k: when $k=3$, $r_3(x_3)=\begin{cases}1, & x_3\geq 4\\ 0, & \text{otherwise}\end{cases}$
  when $k=0,1,2$, $r_k(x_k)=0$.

According to the definition of the reward, it is easy to see that $\mathbb{E}[r_3(x_3)]=\Pr(x_3\geq 4)$ and $\mathbb{E}[r_k(x_k)]=0$ for $k\leq 2$. Therefore, maximizing the probability of ending up with \$4 or more is equivalent to

$$\max_{\mu_k}\mathbb{E}\left[\sum_{k=0}^{3}r_k(x_k)\middle| x_0=2\right].$$

For $k<3$, the Bellman equation of this problem is

$$V_k(x_k)=\max_{u\in\{0,1,\cdots,x_k\}}\left(V_{k+1}(x_k+u)\times 0.4+V_{k+1}(x_k-u)\times 0.6\right).$$

We next apply the Bellman equation to solve this problem.

Stage 3: Stage 3 is the end of the three games, so

$$V_3(x_3)=\begin{cases}1, & x_3\geq 4\\ 0, & x_3=0,1,2,3\end{cases}.$$

Stage 2: Note that $u_k^*=0$ when $x_k\geq 4$ because the agent does not need to bet anymore to get \$4 or more and $V_2(x_2)=1$ for any $x_2\geq 4$. Therefore, we only need to consider $x_2\in\{1,2,3\}$.

- If the agent has \$1 at the beginning of the third game, then it is impossible for the agent to get \$4 or more, so
$$V_2(x_2=1)=0.$$

- If the agent has \$2 at the beginning of the third game, then the agent has a chance to have \$4 or more by betting the \$2, and has no chance to have \$4 if betting anything less than \$2. Therefore, we have
$$V_2(x_2=2)=\max\{0,0,0.4V_2(4)+0.6V_2(0)\}=0.4,$$
where the first two 0's correspond to betting \$0 and betting \$1.

- If the agent has \$3 at the beginning of the third game, then the agent has a chance to have \$4 or more by betting at least \$1. Therefore, we obtain
$$V_2(x_2=3)=\max\{0,0.4V_2(4)+0.6V_2(2),\ 0.4V_2(5)+0.6V_2(1),\ 0.4V_2(6)+0.6V_2(0)\}=0.4.$$

Stage 1: Similar to the analysis for stage 2, following the Bellman equation, we obtain

- $V_1(x_1=0)=0$
- $V_1(x_1=1)=0.16$ and $u_1^*=1$.
- $V_1(x_1=2)=0.4$ and $u_1^*\in\{0,2\}$.
- $V_1(x_1=3)=0.64$ and $u_1^*=1$.

- $V_1(x_1 \geq 4) = 1$.

Stage 0: Finally at stage 0, we have \$2 dollars. Following the Bellman equation and considering three possible actions $\{0, 1, 2\}$, we have

- $V_0(x_0 = 2) = 0.4$ with $u_0^* \in \{0, 2\}$.

This concludes the backward pass for computing the value function. Based on the value function, the optimal policy is to bet \$2 once in one of the three games.

Secretary problem

The secretary problem [4] can be described as follows. Imagine an administrator who wants to hire the best secretary out of n available candidates. They are interviewed in succession in random order, that is, for each interview the administrator picks a candidate uniformly at random from the remaining candidates. After interviewing the kth candidate, the administrator knows the ranking of the kth candidate among the candidates interviewed so far, but is unaware of the quality of unseen candidates. The administrator needs to determine whether to hire or reject the kth candidate immediately after the interview. Note that the administrator cannot go back to hire candidate $1, \ldots, k-1$ after interviewing the kth candidate. At most one candidate can be hired. The goal is to maximize the probability of hiring the best candidate.

We can formulate the secretary problem into a finite-horizon stochastic DP problem as follows.

- Stage k: the time when the administrator makes decision after interviewing the kth candidate. $k \in \{1, ..., n\}$.
- State x_k: the ranking of the kth candidate among the candidates interviewed so far. $x_k \in \{1, ..., k\}$. Define a terminal state g such that if the administrator has already hired one candidate in one of the previous interviews, then $x_k = g, x_{k+1} = g, \ldots, x_n = g$. That is, once the administrator decides to hire a candidate, the process will go to the terminal state and remain in the terminal state.
- Action u_k:

  If $x_k \neq g$, then

  - $u_k = 1$ means hiring the kth candidate;
  - $u_k = 0$ means rejecting the kth candidate.

  If $x_k = g$, then the only available action is $u_k = 0$, meaning rejecting the kth candidate, because $x_k = g$ means that the administrator has already hired one candidate previously.
- Reward $r_k(x_k, u_k)$:

  $r_k(x_k, u_k) = 1$ if $x_k \neq g$, $u_k = 1$ and the kth candidate is the best candidate among all the n candidates.

  Otherwise, $r_k(x_k, u_k) = 0$.

  Note that if $x_k \neq 1$, $r_k(x_k, 1) = 0$ because $x_k \neq 1$ means that the kth candidate is not the best among the k candidates that have been interviewed and of course not the best among

all candidates. If $x_k = 1$, we can calculate the probability that the kth candidate is the best among all candidates as follows:

$$\Pr\left(\text{the } k^{\text{th}} \text{ candidate is the best among all candidates} | x_k = 1\right) = \frac{\binom{n-1}{k-1}(k-1)!(n-k)!}{\binom{n}{k}(k-1)!(n-k)!} = \frac{k}{n}.$$

Therefore, $r_k(1,1) = 1$ with probability k/n.

- Transition probabilities:

  For any $m \in \{1, \ldots, k+1\}$ and $j \in \{1, \ldots, k\}$, we have

  $$\Pr(x_{k+1} = m | x_k = j, u_k = 0) = \frac{\binom{n}{k+1}(k-1)!(n-k-1)!}{\binom{n}{k+1}\binom{k+1}{k}(k-1)!(n-k-1)!} = \frac{1}{k+1},$$

  which means that x_{k+1} will be uniformly distributed in $\{1, \ldots, k+1\}$ if $x_k \neq g$ and $u_k = 0$. And for any $j \in \{1, \ldots, k\}$,

  $$\begin{aligned} \Pr(x_{k+1} = g | x_k = j, u_k = 0) &= 0, \\ \Pr(x_{k+1} = g | x_k = j, u_k = 1) &= 1, \\ \Pr(x_{k+1} = g | x_k = g, u_k = 0) &= 1. \end{aligned}$$

- Goal: maximize the probability of hiring the best candidate. Note that this goal can be written as

  $$\max_{\mu_1, \ldots, \mu_n} \mathbb{E}\left[\sum_{k=1}^{n} r_k(x_k, \mu_k(x_k))\right].$$

Let $V_k(x_k)$ denote the optimal value function for state x_k at stage k, defined by

$$V_k(x_k) = \max_{\mu_k, \ldots, \mu_n} \mathbb{E}\left[\sum_{j=k}^{n} r_j(x_j, \mu_j(x_j))\right].$$

The optimal value function $V_k(x_k)$ can be interpreted as the probability that the best candidate will be hired under the optimal policy given x_k at stage k. For example, let $n = 10$, $k = 5$, $x_5 = 1$. In this scenario, there are 10 candidates total. After interviewing the 5th candidate, we know that this candidate ranks first among the 5 candidates that have been interviewed. Then $V_5(1)$ is the probability that we can successfully hire the best candidate if we follow the optimal policies at and after stage 5.

Based on the above definition and analysis, the Bellman equation for $k = 1, ..., n-1$ can be written as:

$$\begin{aligned} V_k(1) &= \max\left\{\mathbb{E}[r_k(1,1)], \mathbb{E}[V_{k+1}(x_{k+1})]\right\} = \max\left\{\frac{k}{n}, \frac{1}{k+1}\sum_{x=1}^{k+1} V_{k+1}(x)\right\} \\ V_k(x_k) &= \max\left\{0, \mathbb{E}[V_{k+1}(x_{k+1})]\right\} = \frac{1}{k+1}\sum_{x=1}^{k+1} V_{k+1}(x), x_k = 2, ..., k. \end{aligned} \tag{1.3}$$

For $V_k(1)$, the first term inside the maximum function, $\mathbb{E}[r_k(1,1)]$, is the expected reward given $u_k = 1$, i.e., hiring the kth candidate. The total future reward is 0 since the process terminates. The second term $\mathbb{E}[V_{k+1}(x_{k+1})]$ is the expected value given $u_k = 0$, i.e., rejecting the kth candidate, since the instantaneous reward is 0. For $V_k(x_k)$ where $x_k = 2, ..., k$, the expected value for hiring the kth candidate is 0 since the ranking of the kth candidate among all the n candidates must be larger than or equal to 2. Similar to the case for $V_k(1)$, the expected value of rejecting the kth candidate is $\mathbb{E}[V_{k+1}(x_{k+1})]$.

We start by calculating the optimal value function at the last stage n, i.e., the values of $V_n(x_n)$ for all $x_n \in \{1, ..., n\}$. $V_n(x_n) = 1$ if $x_n = 1$ and $V_n(x_n) = 0$ otherwise because at stage n we have interviewed all of the n candidates and $x_n = 1$ means that the last candidate is the best among n candidates. Then we can calculate $V_k(x_k)$ for $k = n-1, \ldots, 1$ using backward search with the Bellman equation (1.3). After obtaining the value function, we can determine the optimal policy by a forward pass. The implementation code can be found in our GitHub repository [1] in the folder "secretary-problem".

Actually, we can solve the optimal value function analytically and obtain closed-form formulas for the value function. Then using the Bellman equation (1.3), we can obtain the optimal policy, that is, at stage k, hire the candidate if

- $k \geq k^*$, where $k^* \approx \frac{n}{e}$ for large n [4], and
- they are the best candidate so far ($x_k = 1$).

In other words, we reject first $k^* - 1$ candidates and hire the best candidate so far after that.

Exercising a stock option [26]

You have a call option, which gives you the option to buy a share at the"striking price" p (a positive integer) before the option expires. You have up to N days to exercise it. If you exercise the option on day k when the stock price is x_k, you can immediately make a profit of $x_k - p$. If you do not exercise it on the last day, the option expires and has no value. Suppose the price of a share of the stock is an integer and obeys the equation $x_{k+1} = x_k + w_k$, where $\{w_k\}$ are i.i.d. random variables uniformly distributed in $\{-1, 0, 1\}$. Given an initial stock price x_0, the aim is to exercise the option optimally to maximize the expected profit.

This problem can be formulated into a finite-horizon stochastic DP problem as follows:

- Stage k: Day k, $k \in \{0, \ldots, N-1\}$; stage N is the time after the last day.
- State x_k: the price of the stock at Day k. Define a terminal state g such that $x_k = g$ means that the option has already been exercised.
- Action: u_k: If $x_k \neq g$, then
  - $u_k = 1$: exercise the stock option;
  - $u_k = 0$: do not exercise.

  If $x_k = g$, then the only available action is $u_k = 0$ because the stock option has already been exercised.

- Reward: $r(x_k, u_k)$:

$$r(x_k, u_k) = \begin{cases} x_k - p, & \text{if } x_k \neq g \text{ and } u_k = 1; \\ 0, & \text{otherwise.} \end{cases}$$

- Transition: For any $x \neq g$,

$$\begin{aligned} &\Pr(x_{k+1} = x - 1 | x_k = x, u_k = 0) \\ =&\Pr(x_{k+1} = x | x_k = x, u_k = 0) \\ =&\Pr(x_{k+1} = x + 1 | x_k = x, u_k = 0) = \frac{1}{3}, \end{aligned}$$

  and

$$\begin{aligned} \Pr(x_{k+1} = g | x_k = g, u_k = 0) = 1, \\ \Pr(x_{k+1} = g | x_k = x, u_k = 1) = 1. \end{aligned}$$

- Goal: given x_0, maximize the expected total reward:

$$\max_{\mu_0, \ldots, \mu_{N-1}} \mathbb{E}\left[\sum_{k=0}^{N-1} r_k(x_k, \mu_k(x_k)) | x_0\right].$$

Let $V_k(x_k)$ denote the optimal value function for state x_k at stage k, defined by

$$V_k(x_k) = \max_{\mu_k, \ldots, \mu_{N-1}} \mathbb{E}\left[\sum_{l=k}^{N-1} r_l(x_l, \mu_l(x_l))\right].$$

$V_k(x_k)$ is the optimal expected profit given the stock price x_k and that there are still $N - k$ days (Day k,..., Day $N - 1$) to go. Based on the formulation, the Bellman equation can be written as

$$\begin{aligned} V_k(x_k) &= \max\{x_k - p, \mathbb{E}[V_{k+1}(x_{k+1})]\} \\ &= \max\left\{x_k - p, \frac{1}{3}V_{k+1}(x_k) + \frac{1}{3}V_{k+1}(x_k + 1) + \frac{1}{3}V_{k+1}(x_k - 1)\right\}, \end{aligned}$$

for any $k = 0, \ldots, N - 1$. We know $V_N(x) = 0$ for any value of x since the option expires. Also notice that given x_0, the values of $x_1, \ldots, x_N$ are from $x_0 - N$ to $x_0 + N$. Using backward search, we can calculate the value function $V_k(x_k)$. The optimal policy can be obtained by a forward pass. The implementation code can be found in our GitHub repository [1] in the folder "exercising-a-stock-option".

1.6 Infinite-Horizon Discounted DP

In this section, we introduce another DP formulation — infinite-horizon discounted DP, which probably is most popular formulation for RL applications. In an infinite-horizon, discounted DP, the goal of the agent is to find a policy $\{\mu_k^*\}_{k=0,\cdots,\infty}$ such that

$$\{\mu_k^*\}_{k=0,\cdots,\infty} \in \arg\max_{\{\mu_k\}_{k=0,\cdots,\infty}} \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, \mu_k(x_k))\right],$$

where $\alpha \in (0,1)$ is a discount factor that is introduced so that $\mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, \mu_k(x_k))\right]$ is finite and the problem is well defined.

For infinite-horizon problems, for the sake of tractability, we often consider time-homogeneous stochastic systems (or environments) defined below:

- Transition kernel $\Pr(x_k = j | x_{k-1} = i, u_{k-1} = a) = \Pr(x' = j | x = i, u = a)$ does not depend on k
- Reward function $r_k(x_k = x, u_k = u) = r(x, u)$ does not depend on k

For a time-homogeneous system, given discount factor $0 < \alpha < 1$ and initial state x_0, an infinite-horizon discounted DP is to find a time-homogeneous policy μ such that

$$V(x_0) := \max_{\mu} \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, \mu(x_k))\right].$$

Note that the optimization problem above only considers time-homogeneous. It is a well known result that for time-homogeneous systems (under some mild technical assumptions), one of the optimal policies is time-homogeneous (see Section 6.2 [16]) (call stationary policies). we therefore can restrict the policy space to stationary policies.

While initially we may think an infinite-horizon problem is much harder than a finite-horizon problem because the horizon of the problem becomes unbounded so the backward computation does not have a starting point, the problem actually becomes simpler when it is time-homogeneous. Recall the Bellman equation for finite-horizon DP under policy μ :

$$V_k(x_k) = \max_{\mu_k} \mathbb{E}\left[r(x_k, \mu_k(x_k)) + \alpha V_{k+1}(x_{k+1})\right].$$

Since it is a infinite horizon problem, given a time homogeneous policy, we have

$$\begin{aligned} V_{\mu,k}(x) =& \mathbb{E}\left[r(x_k, \mu_k(x_k)) + \alpha V_{k+1}(x_{k+1}) | x_k = x\right] \\ V_{\mu,k+1}(x) =& \mathbb{E}\left[r(x_{k+1}, \mu(x_{k+1})) + \alpha V_{k+2}(x_{k+2}) | x_{k+1} = x\right]. \end{aligned}$$

It is not difficult to see that

$$V_{\mu,k}(x) = V_{\mu,k+1}(x)$$

since they start from the same state, have the same policy, and have the same remaining horizon which is infinity. Or if we define $k' = k + 1$, then the two equations are identical.

Therefore, for a time-homogeneous stochastic system, the value function is also time-homogeneous and the Bellman equation becomes

$$V(x) = \max_{\mu} \mathbb{E}\left[r(x, \mu(x)) + \alpha V(x')\right].$$

In other words, the same value function appears at both sides of the equation. We can then use recursive methods to solve the Bellman equation to obtain the value functions. We will leave the algorithms that can be used to solve the equation in later chapters.

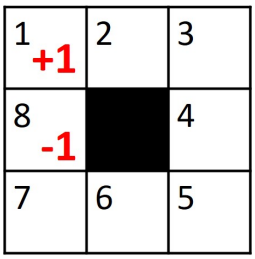


Figure 1.13: A simple grid world.

An example—a simple grid world

We provide an example of the infinite-horizon discounted DP problem. As shown in Fig. 1.13, there is a 3×3 grid world, where the middle black grid is inaccessible. For each time slot, the agent has two possible actions, clockwise (c) and counter-clockwise (cc), and the agent can move only one step at a time. Therefore, the agent will move to one of the neighboring grids for each time slot. However, the environment is stochastic in the sense that the agent follows the action with probability 0.8 and moves to the opposite direction with probability 0.2. For example, suppose the agent is currently at grid 3. If the action is c, then the agent will move to grid 4 with probability 0.8 and will move to grid 2 with probability 0.2. There are two states with nonzero rewards, $+1$ in state 1, and -1 in state 8. That is, $r(1, *) = 1$ and $r(8, *) = -1$, where $*$ is any action. Otherwise, the reward is 0. We want to find a policy that maximizes the discounted total rewards with the discount factor 0.9.

Chapter 2

Markov Decision Processes (MDPs) and Model-Based Algorithms

Theoretically, many reinforcement learning problems can be formulated as Markov Decision Processes (MDPs), for which many fundamental results have been established when the model is known, where the model includes the transition kernel of the MDP, the reward function, etc. A unique aspect of RL is that it has to deal with unknown MDPs. Nevertheless, dynamic programming and Markov decision processes are the mathematical foundations of RL and have been used to guide the design of RL algorithms. For example, most RL algorithms use the Bellman loss as the loss function, which comes from the fact that the optimal solution to an MDP problem is a solution to the corresponding Bellman equation. Furthermore, most existing algorithms are extensions of iterative algorithms for solving the Bellman equation for a known MDP such as the value iteration algorithm or the policy iteration algorithm, which will be introduced in this chapter.

This chapter provides a brief introduction of basic concepts of Markov chains and MDPs, which the readers will encounter when reading research papers on RL. We will only present some key concepts and results that are necessary for understanding the materials to be covered in this book. Comprehensive studies of Markov chains and MDPs can be found in [16].

2.1 Markov Chains

In this book, we only consider discrete-time Markov chains. A discrete-time Markov chain is a special class of discrete-time random processes. Consider a random process $\{x_k\} \in \mathcal{X}$, k is the time index, and $\mathcal{S}$ is the state space. We assume $\mathcal{X}$ is a finite set for simplicity. We will introduce RL algorithms for infinite and even uncountable state and action spaces, but our theoretical analysis will limit to countable state and action spaces for simplicity.

The random process $\{x_k\}$ is called a Markov chain if the following property holds:

$$\Pr(x_k = j | x_{k-1} = i, x_{k-2} = i_{k-2}, \ldots, x_0 = i_0) = \Pr(x_k = j | x_{k-1} = i) \quad \forall k, i, j, i_{k-2}, \ldots, i_0.$$

This property is called the *Markov property* that states that conditioned on the most recent history x_{k-1}, the distribution of the state at time k does not depend on older history. We further restrict ourselves to *time-homogeneous* Markov chains such that

$$\Pr(x_k = j | x_{k-1} = i)$$

does not depend on the time k. For time-homogeneous Markov chains, we define a matrix $\mathbf{P}$ such that

$$P_{ij} = \Pr(X_k = j | X_{k-1} = i),$$

called the *transition probability matrix*.

We further define an $|\mathcal{X}|$ dimensional vector p_k which is the distribution of the Markov chain at time k such that

$$p_k(i) = \Pr(X_k = i).$$

Then the evolution and distribution of the Markov chain are uniquely determined by the transition probability matrix and the initial distribution p_0 :

$$p_k = \left(\mathbf{P}^\top\right)^k p_0,$$

where $\mathbf{P}^\top$ is the transpose of $\mathbf{P}$. This can be easily verified because $p_{k+1} = \mathbf{P}^\top p_k$ according to the definitions of p_k and $\mathbf{P}$.

Given a time-homogeneous, discrete-time Markov chain, we further assume the Markov chain has a unique stationary distribution π such that $\pi = \mathbf{P}^\top \pi$ and $\lim_{k\to\infty} p_k = \pi$ for all p_0.

2.2 Example

In this section, we present an application of Markov chains.

PageRank

PageRank is the search algorithm behind the Google search engine []. PageRank ranks the importance of webpages based on the hyperlinks that connect the webpages. The algorithm does not look at the contents of the webpages and ranks their importance only based on the topology of the graph. Yet, the algorithm has been remarkably successful and is a key component of the Google's search algorithm. The way the PageRank algorithm can be understood from different perspectives. We will use the Markov chain perspective to explain the algorithm:

PageRank

Imagine an agent who is randomly surfing the webpages. At any time k, the agent is on webpage i. At time $k+1$ the agent picks a hyperlink uniformly at random and goes to the webpage by clicking the hyperlink. Let $p_k(i)$ be the probability that the agent is at web i at time k. The rank of web i is $p_\infty(i)$.

Let x_k be the webpage the agent is looking at at time k, then $\{x_k\}$ is a Markov chain because the next webpage x_{k+1} only depends on the current webpage x_k. Assuming the Markov chain has a unique stationary distribution π, the rank provided by the PageRank algorithm is π. NOte that the stationary distribution represents the fraction of time the agent spends on browsing webpage i.

Consider a simple example as shown in Figure 2.1. This problem can be formulated as a Markov chain as shown in Figure 2.2. The Markov chain is a finite-state, irreducible, and aperiodic Markov

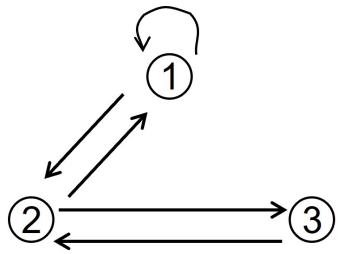


Figure 2.1: A PageRank Example

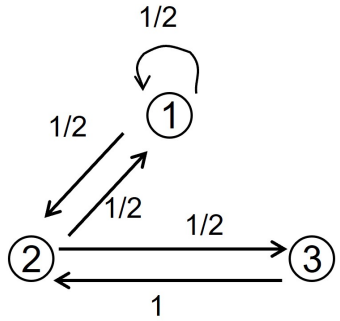


Figure 2.2: The Markov chain representation

chain with transition probability matrix $\begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \end{pmatrix}$. We assume the initial distribution of the Markov chain $p_0 = \begin{pmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{pmatrix}$. In other words, the agent uniformly and randomly picks a webpage from all webpages. The Markov chain has a unique stationary distribution of $\pi = \begin{pmatrix} \frac{2}{5} \\ \frac{2}{5} \\ \frac{1}{5} \end{pmatrix}$, which means that webpage 1 and webpage 2 are more important than webpage 3.

2.3 Markov Decision Processes

A controlled Markov chain is a Markov chain in which state-transition probabilities can be controlled, depending on the action that is taken. In particular, under action u, transition probability from i to j is $P_{ij}(u)$, which is a function of action u. Recall that in stochastic DP, we introduce policy, which is a function that maps states to actions. For a controlled MDP, when the policy is given, the transition probabilities are fixed so it becomes an uncontrolled Markov chain in the previous section.

We consider the infinite-horizon discounted DP under the Markov chain model, and study the optimal policy with MDPs. Recall the infinite-horizon, discounted problem whose goal is to

maximize the following discounted total reward:

$$\lim_{T\to\infty} \mathbb{E}\left[\sum_{k=0}^{T} \alpha^k r(x_k, u_k)\right], \quad 0 < \alpha < 1.$$

For solving an MDP, at time k, we assume we have access to $\{x_0, \ldots, x_k\}$, $\{u_0, \ldots, u_{k-1}\}$ and $\{r(x_0, u_0), \ldots, r(x_{k-1}, u_{k-1})\}$, i.e. the past history of states, actions and rewards, and the current state. Define μ_k, a function (policy) which decides the action to be taken at time k.

$$u_k = \mu_k(x_0, \ldots, x_k, u_0, \ldots, u_{k-1}, r_0, \ldots, r_{k-1})$$

a function of past history and current state. We note that μ_k can be a random function (random policy).

For discounted MDP, it is sufficient to consider policies depending only on the current state x_k, i.e.

$$u_k = \mu_k(x_k),$$

called Markov policies. It can be proved that while an optimal policy may not be a Markov policy, one of the Markov policies is an optimal policy. Therefore, for the purpose of finding *one* optimal policy, it is sufficient to focus on Markov policies, which significantly reduces the size of the policy space. For time-homogeneous Markov chains, we can further reduce the size of the policy space to the set of *stationary* policies. If a policy does not explicitly depend on k, i.e. $u_k = \mu(x_k)$, then it is called a stationary policy. Detailed discussions and proofs can be bound in [16]. In this book, we assume one of the optimal policies is stationary for the MDPs we consider and focus on stationary policies.

2.4 Value Iteration for MDPs

We next introduce model-based solutions for solving MDPs, including value iteration and policy iteration. First let us look at value iteration. Consider an infinite-horizon, discounted MDP problem. Recall the Bellman equation

$$V(x) = \max_{\mu} \mathbb{E}\left[r(x, \mu(x)) + \alpha V(x')\right],$$

where μ is a policy, $V(\cdot)$ is the value function, $r(x, \mu(a))$ is the reward received when taking action $\mu(x)$ in state x, $0 < \alpha < 1$ is the discount factor, and x' is the next state. We assume that both the state space $\mathcal{X}$ and action space $\mathcal{U}$ are finite. Furthermore, assume that we know the model, namely, the transition probability $P_{x,x'}(u)$ and the mean reward $\bar{r}(x, u) = \mathbb{E}[r(x, u)]$ for any $x, x' \in \mathcal{X}$ and any $u \in \mathcal{U}$.

The following algorithm, called value iteration, solves the value function.

Value Iteration

Starting from $(V_0(x))_{x\in\mathcal{X}}$, iteratively compute

$$V_{k+1}(x) \leftarrow \max_{u\in\mathcal{U}}\left(\bar{r}(x,u) + \alpha\sum_{x'} P_{x,x'}(u)V_k(x')\right) \text{ for all } x\in\mathcal{X},. \tag{2.1}$$

We can show that $V_k(x) \to V(x)$ for all $x \in \mathcal{X}$ as k goes to infinity. In practice, we will stop when $\max_{x\in|calX}|V_k(x)-V_{k-1}(x)| < \epsilon$ for some pre-selected $\epsilon > 0$ that controls the precision. After finding V, we can find the optimal policy by solving

$$\arg\max_u \left(\bar{r}(x,u) + \alpha\sum_{x'} P_{x,x'}(a)V(x')\right).$$

Note that we abuse notation here to use $V_k(\cdot)$ to denote the value function calculated in the kth iteration of the algorithm, not the value function for the kth stage in a finite-horizon DP problem. Since a infinite-horizon, discounted problem does not have "stages", we hope this abuse of notation does not cause any confusion.

2.4.1 Examples and Codes

A simple grid world

Recall the simple grid world introduced in the previous chapter. As shown in Fig. 2.3, there is a 3×3 grid world, where the middle black grid is inaccessible. For each time slot, the agent has two possible actions, clockwise (c) and counter-clockwise (cc), and the agent can move only one step at a time. Therefore, the agent will move to one of the neighboring grids for each time slot. However, the environment is stochastic in the sense that the agent follows the action with probability 0.8 and moves to the opposite direction with probability 0.2. For example, suppose the agent is currently at grid 3. If the action is c, then the agent will move to grid 4 with probability 0.8 and will move to grid 2 with probability 0.2. There are two states with nonzero rewards, $+1$ in state 1, and -1 in state 8. That is, $r(1,*) = 1$ and $r(8,*) = -1$, where $*$ is any action. Otherwise, the reward is 0. We want to find a policy that maximizes the discounted total rewards with the discount factor 0.9.

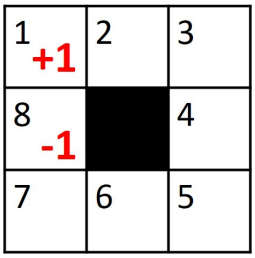


Figure 2.3: A simple grid world.

Let us use value iteration to solve this problem. Initialize $V_0(x) = 0$ for all $x = 1,\cdots,8$.

According to Equation. (2.1), we can obtain

$$\begin{aligned} V_1(1) &= \max_{a\in\{c,cc\}} \mathbb{E}\left[r(1,a) + 0.9V_0(x')\right] \\ &= \max_{a\in\{c,cc\}} \mathbb{E}\left[1 + 0.9V_0(x')\right] \\ &= 1. \end{aligned}$$

Similarly,

$$\begin{aligned} V_1(2) &= \max_{a\in\{c,cc\}} \mathbb{E}\left[r(2,a) + 0.9V_0(x')\right] \\ &= \max_{a\in\{c,cc\}} \mathbb{E}\left[0.9V_0(x')\right] \\ &= \max\{0,0\} = 0, \end{aligned}$$

where the second line follows from $r(2,a) = 0$ for any action a, and the third line is due to the fact that the elements of V_0 are all zero. Repeating the calculation for $x = 3, \cdots, 8$, we obtain

$$V_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ -1 \end{pmatrix}.$$

Next, we calculate V_2 using V_1 and have

$$\begin{aligned} V_2(1) &= \max_{a\in\{c,cc\}} \mathbb{E}\left[r(1,a) + 0.9V_1(x')\right] \\ &= \max_{a\in\{c,cc\}} \mathbb{E}\left[1 + 0.9V_1(x')\right] \\ &= \max\left\{1 + 0.9 \times [0.8 \times V_1(2) + 0.2 \times V_1(8)], 1 + 0.9 \times [0.8 \times V_1(8) + 0.2 \times V_1(2)]\right\} \\ &= \max\left\{1 + 0.9 \times (0.8 \times 0 + 0.2 \times (-1)), 1 + 0.9 \times (0.8 \times (-1) + 0.2 \times 0)\right\} \\ &= 0.82, \end{aligned}$$

where in the third line, $0.8 \times V_1(2) + 0.2 \times V_1(8)$ is the expected value of the next state given that the agent selects action c in state 1, and $0.8 \times V_1(8) + 0.2 \times V_1(2)$ is the expected value of the next state given that the agent selects action cc in state 1. Similarly, we have We have

$$\begin{aligned} V_2(2) &= \max_{a\in\{c,cc\}} \mathbb{E}\left[r(2,a) + 0.9V_1(x')\right] \\ &= \max_{a\in\{c,cc\}} \mathbb{E}\left[0.9V_1(x')\right] \\ &= \max\left\{0.9 \times (0.8 \times V_1(3) + 0.2 \times V_1(1)), 0.9 \times (0.8 \times V_1(1) + 0.2 \times V_1(3))\right\} \\ &= \max\left\{0.9 \times (0.8 \times 0 + 0.2 \times 1), 0.9 \times (0.8 \times 1 + 0.2 \times 0)\right\} \\ &= 0.72. \end{aligned}$$

Similarly, we can calculate the remaining values and obtain

$$V_2 = \begin{pmatrix} 0.82 \\ 0.72 \\ 0 \\ 0 \\ 0 \\ 0 \\ -0.18 \\ -0.28 \end{pmatrix}.$$

Continuing the calculation for many iterations, eventually, we have

$$V = \begin{pmatrix} 3.36 \\ 2.86 \\ 2.43 \\ 2.07 \\ 1.77 \\ 1.54 \\ 1.49 \\ 1.69 \end{pmatrix}.$$

Then we can find the optimal policy μ^* using V as follows:

$$\begin{aligned} \mu^*(x) &\in \arg\max_{a \in \{c, cc\}} \mathbb{E}\left[r(x,a) + 0.9V(x')\right] \\ &= \arg\max_{a \in \{c, cc\}} \mathbb{E}\left[0.9V(x')\right]. \end{aligned}$$

where the last equality holds because the reward $r(x, a)$ only depends on x and does not depend on a so $r(x, c) = r(x, cc)$ for all x.

Letting x_c and x_{cc} denote the clockwise and counter-clockwise neighbor of x, respectively, we further have

$$\mu^*(x) \in \arg\max_{c, cc} \left\{0.9\left(0.8V(x_c) + 0.2V(x_{cc})\right), 0.9\left(0.8V(x_{cc}) + 0.2V(x_c)\right)\right\}.$$

By removing the discount factor and subtracting $0.2V(x_c) + 0.2V(x_{cc})$, we reduce the last line to

$$\mu^*(x) \in \arg\max_{c, cc} \left\{V(x_c), V(x_{cc})\right\},$$

which suggests that the optimal policy is to move to the neighbor whose value is larger. Therefore, according to the V we obtained, the optimal policy is

$$\mu^* = \begin{pmatrix} c \\ cc \\ cc \\ cc \\ cc \\ cc \\ c \\ c \end{pmatrix}.$$

Web page recommendation problem

A simplified version of the web page recommendation problem [22] can be described as follows. A user enters our website and begins requesting web pages. Considering the web page this user has visited, the system has to predict in what other page the user is probably interested and recommend it to the user. Suppose there are W web pages in total, denoted by page 0, page 1, ..., page $W-1$. At the time when we need to make a recommendation, we have the information of the last web page that the user has visited, denoted by web page x, where $x \in \{0, 1, \ldots, W-1\}$. We recommend one page out of the W candidate web pages to the user, denoted by web page a, where $a \in \{0, 1, \ldots, W-1\}$. Then the user may choose to visit the recommended web page a with probability $p(x, a)$ or choose to visit other pages uniformly at random, i.e., visit web page $w \in \{0, \ldots, a-1, a+1, \ldots, W-1\}$ with probability $\frac{1-p(x,a)}{W-1}$. A recommendation is considered successful and a reward 1 is obtained if the user chooses to visit the recommended page a. Otherwise, the reward is 0. Let x' denote the web page that the user chooses to visit. After the user finishes the visit of web page x', the process goes to the next step and we need to make a recommendation again. An example of the process is shown in Figure 2.4.

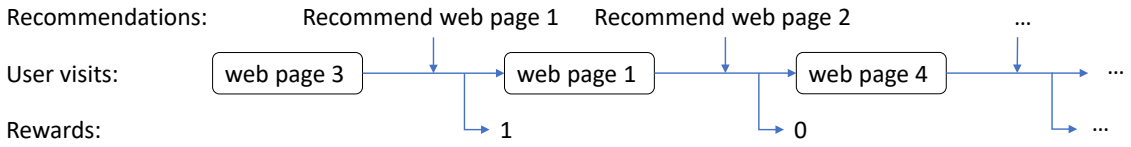


Figure 2.4: An example of the web page recommendation.

Assume that the choice of the user's next visit x' depends only on the last visit x and the recommendation a. Assume that the first web page that the user visits follows the uniform distribution. Let α denote the discount factor for the rewards. The goal of the system is to maximize the discounted total rewards.

This problem can be formulated into an infinite-horizon discounted DP problem as follows.

- State x: the last web page that the user has visited, $x \in \{0, 1, \ldots, W-1\}$.
- Action a: the recommended web page, $a \in \{0, 1, \ldots, W-1\}$.
- Transition: Let x_k and a_k denote the state and the action at step k, respectively.

$$\Pr(x_{k+1} = x' | x_k = x, u_k = a) = \begin{cases} p(x, a) & \text{if } x' = a \\ \frac{1-p(x,a)}{W-1} & \text{if } x' \neq a, \end{cases}$$

  where $x, a, x' \in \{0, 1, \ldots, W-1\}$.

- Reward: $r(x, a)$:

$$r(x, a) = \begin{cases} 1 & \text{with probability } p(x, a) \\ 0 & \text{with probability } 1 - p(x, a), \end{cases}$$

- Goal: maximize the expected discounted total reward:

$$\mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, a_k)\right].$$

Let $V^*(x)$ denote the optimal value function for state x, defined by

$$V^*(x) = \max_{\mu} \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, \mu(x_k)) \,|x_0 = x\right],$$

where the "max" is taken over any stationary policy μ. Then the Bellman equation can be written as

$$\begin{aligned} V^*(x) &= \max_{\mu} \mathbb{E}\left[r(x, \mu(x)) + \alpha V^*(x')\right] \\ &= \max_{a} \left(\mathbb{E}[r(x,a)] + \alpha \sum_{x'} \Pr(x_{k+1} = x' | x_k = x, a_k = a) V^*(x')\right) \\ &= \max_{a} \left(p(x,a) + \alpha p(x,a) V^*(a) + \alpha \sum_{x' \neq a} \frac{1 - p(x,a)}{W - 1} V^*(x')\right). \end{aligned}$$

If we are given W and $p(x, a)$ for all x, a, the problem can be solved using value iteration. The implementation code can be found in our GitHub repository [1] in the folder "web-page-recommendation-model-based".

2.5 Policy Iteration

In the previous section, we introduced value iteration algorithm, which is an iterative method to compute the value function. Value iteration is a very efficient algorithm. However, you may notice that the initial value V_0 can be arbitrarily chosen, so is not associated with any policy or has any physical meaning. In fact, this is true for V_k for any k, except the final value function, which is the value function associated with the optimal policy.

In this section, we introduce another model-based algorithm for solving the Bellman equation, called policy iteration. The policy iteration takes a different approach where we have a policy at each iteration and improve upon it to obtain a better policy. This is the key difference between value iteration and policy iteration. As the name suggests, under policy iteration, we will iteratively improve the policy until we find the optimal policy.

Policy Iteration

- Choose an initial policy μ_0, set $k = 0$.
- Let V_{μ_k} denote the value function of the policy μ_k. Compute V_{μ_k} by solving the Bellman

equation:

$$V_{\mu_k}(x) = \mathbb{E}[r(x, \mu_k(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_k(x)) V_{\mu_k}(x') \quad \text{for all } x.$$

Note that this is a set of linear equations.

- Based on V_{μ_k}, compute a new policy μ_{k+1} as

$$\mu_{k+1}(x) \in \arg\max_a \left(\mathbb{E}[r(x, a)] + \alpha \sum_{x'} P_{x,x'}(a) V_{\mu_k}(x') \right) \quad \text{for all } x.$$

- Based on μ_{k+1}, compute $V_{\mu_{k+1}}$ like Step 2. If $V_{\mu_{k+1}} \neq V_{\mu_k}$, let $k \leftarrow k+1$, go back to Step 3 and repeat.

Under the policy iteration algorithm, we have $V_{\mu_{k+1}}(x) \geq V_{\mu_k}(x)$ for all states x, i.e. the value function improves at each step before the algorithm terminates. Therefore, the policy strictly improves until we find the optimal policy. The proof of convergence will be provide in a later chapter.

2.5.1 Examples and Codes

The grid world example

We will use policy iteration algorithm to solve the grid world problem in Section 2.4.1. First, let us

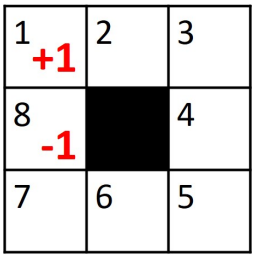


Figure 2.5: A simple grid world.

choose an initial policy μ_0, where $\mu_0(x) = c$ for all x, i.e. the initial policy is to move clockwise in any state. Then we can compute V_{μ_0} according to Step 2 of the algorithm, i.e., solving the following system of linear equations:

$$V_{\mu_0}(x) = \mathbb{E}[r(x, c)] + 0.9 \times (0.8 V_{\mu_0}(x_c) + 0.2 V_{\mu_0}(x_{cc})) \quad \text{for all } x.$$

After solving this set of equations, we obtain

$$V_{\mu_0} = \begin{pmatrix} 1.04 \\ 0.13 \\ -0.08 \\ -0.14 \\ -0.18 \\ -0.21 \\ -0.25 \\ -0.30 \end{pmatrix}.$$

Then according to Step 3 of the algorithm, we can compute a new policy μ_1 as

$$\mu_1(x) \leftarrow \arg\max_{a \in \{c, cc\}} \{\mathbb{E}[r(x,c)] + 0.9\left(0.8V_{\mu_0}(x_c) + 0.2V_{\mu_0}(x_{cc})\right), \\ \mathbb{E}[r(x,cc)] + 0.9\left(0.8V_{\mu_0}(x_{cc}) + 0.2V_{\mu_0}(x_c)\right)\} \quad \text{for all } x,$$

where the first term inside the arg max corresponds to the value of $a = c$ and the second term corresponds to the value of $a = cc$. In this way, we obtain an improved policy μ_1:

$$\mu_1 = \begin{pmatrix} c \\ cc \\ cc \\ cc \\ cc \\ cc \\ cc \\ c \end{pmatrix}$$

Recall that the optimal policy is

$$\mu^* = \begin{pmatrix} c \\ cc \\ cc \\ cc \\ cc \\ cc \\ c \\ c \end{pmatrix}.$$

We can see that the policy improves and μ_1 differs from μ^* only in state 7.

Web page recommendation problem

Recall the web page recommendation problem in Section 2.4.1. Based on the infinite-horizon discounted DP formulation, if we are given the number of web pages W and the probabilities of accepting recommendations $p(x, a)$ for all x, a, we can also use policy iteration to solve the problem. The implementation code can be found in our GitHub repository [1] in the folder "web-page-recommendation-model-based".

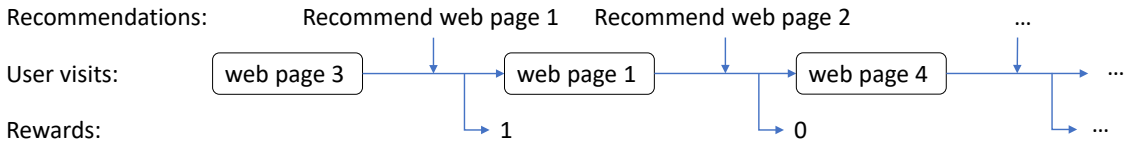


Figure 2.6: An example of the web page recommendation.

2.6 Challenges of Applying Model-based Algorithms

We introduced two model-based algorithms, value iteration and the policy iteration for solving the Bellman equation. Both are very efficiently algorithms. However, there are several fundamental challenges of applying these model-based algorithms in practice.

- First, these algorithms are model-based algorithms, assuming that we know the mean reward $\bar{r}(x, a)$ for any given state x and action a and also the transition kernel $P_{x,x'}(a)$ given any action a. However, in practice, knowing this information is actually difficult, especially if we are facing a complex environment. Learning is needed to address this issue.

- The second challenge is the scalability issue. Value iteration and policy iteration algorithms maintain the value function for each state and assume the number of states is finite. They are called tabular methods. Therefore, they only work for finite state space and finite action space. In practice, we often deal with problems with a very large state space, for example, Go game and many other gaming scenarios. Also, we often encounter continuous state and action space, for example, many applications in robotics. Functional approximation is needed to address this issue.

In the following chapters, we will introduce RL algorithms that address the issues mentioned above.

Chapter 3

Q-Learning and SARSA: Mode-Free RL Algorithms

At the end of the previous chapter, we discussed the limitations of value iteration and policy iteration algorithms. Recall that one of the limitation is that these algorithms need the knowledge of the model, i.e, transition kernel and the mean reward. To resolve this issue, we will introduce a popular reinforcement learning algorithm called Q-learning in this chapter. The algorithm was developed by Watkins in 1989 [25].

3.1 Q-learning

To introduce the Q-learning algorithm, we first define Q-function, which is also called action value function. The Q-function given state x and action u is defined by

$$Q(x,u) = \mathbb{E}\left[r(x,u) + \alpha V(x)\right] = \bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) V(x')$$

The Q-function $Q(i,u)$ is the discounted cumulative reward the agent receives by taking action u in state x and then following the optimal policy starting from the next state x'. In other words,

$$Q(x,u) = \bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) \mathbb{E}\left[\sum_{k=1}^{\infty} \alpha^{k-1} r(x_k, \mu^*(x_k)) \middle| x_1 = x'\right],$$

where μ^* is the optimal policy. From the definition of Q-function, we can see that the only difference between the Q-function $Q(x,u)$ and the value function $V(x)$ is that for the Q-function we assume that the first action is u, which is not necessarily the optimal action, but after the first action u, we always follow the optimal policy. Therefore, Q-function depends not only on the current state but also on the action at the first step. That is why it is also called the action value function.

From its definition, we have

$$Q(x,u) = \bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) V(x') \tag{3.1}$$

$$V(x) = \max_u Q(x,u). \tag{3.2}$$

What is the advantage of learning the Q-function instead of value function? Suppose in some way we were able to learn the Q-function accurately. Then, given that we are in state x, we do not need to know any model information to decide what the optimal action is. We can simply compare $Q(x,u)$ across all possible actions u and find the action which gives the largest Q-value $Q(x,u)$ as suggested in Equation (3.2) because the optimal action is the action which gives us the largest value. That is to say, the optimal policy can be expressed as

$$\mu^*(x) \in \arg\max_u Q(x,u).$$

So learning the Q-function immediately gives us the optimal policy. In contrast, if we only know the value function $V(x)$, we still need the model , $\bar{r}(x,u)$ and $P_{x,x'}(u)$, to decide the optimal action by calculating

$$\arg\max_u \left(\bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) V(x') \right).$$

Q-learning is a learning algorithm to learn the Q-function. Letting $\tilde{Q}(x)$ denote the estimate of $Q(x)$, Q-learning updates $\tilde{Q}$ as follows.

Q-Learning

Given an experience (current state $x_k = x$, current action $u_k = u$, received reward $r(x_k, u_k) = r$, and next state $x_{k+1} = x'$), Q-learning updates $\tilde{Q}(x,u)$ as follows:

$$\tilde{Q}(x,u) \longleftarrow (1-\beta_k)\tilde{Q}(x,u) + \beta_k (r + \alpha \max_v \tilde{Q}(x',v)).$$

The experience $(x_k, u_k, r(x_k,u_k), x_{k+1}) = (x,u,r,x')$ is from the data samples we collect, which either we collect beforehand or online. An experience consists of four pieces of information, current state x_k, current action u_k, the reward received under the current state and action pair $r(x_k,u_k)$, and the next state x_{k+1}.

In the right hand side of the update formula, the first term, $\tilde{Q}(x,u)$, is the current estimate of the Q-value, and the second term, $r + \alpha \max_v \tilde{Q}(x',v)$, can be thought as a new estimate of the Q-value, which comes from the following Bellman equation:

$$Q(x,u) = \bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) V(x') = \bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) \max_v Q(x',v).$$

That is, in the update formula, we use

$$r + \alpha \max_v \tilde{Q}(x',v)$$

as an estimate of

$$\bar{r}(x,u) + \alpha \sum_{x'} P_{x,x'}(u) \max_v Q(x',v)$$

since $P_{x,x'}(u)$ is unknown.

Basically, Q-learning updates the Q-estimate from a linear combination of the old estimate and the new estimate. The weight, $\beta_k \in (0,1)$, is called the learning rate, which determines how

aggressively the new estimate is used in the algorithm. By combining the β_k terms, the Q-learning update formula can also be written as

$$\begin{aligned}\tilde{Q}(x,u) \longleftarrow & (1-\beta_k)\tilde{Q}(x,u) + \beta_k(r + \alpha \max_v \tilde{Q}(x',v)) \\ = & \tilde{Q}(x,u) + \beta_k \left(r + \alpha \max_v \tilde{Q}(x',v) - \tilde{Q}(x,u)\right).\end{aligned}$$

The term inside the parenthesis in the second line, $r + \alpha \max_v \tilde{Q}(x',v) - \tilde{Q}(x,u)$, is called temporal difference (TD). Temporal difference is the difference between the new estimate and the old estimate. If the Q-value estimate is accurate, we expect the temporal difference to be small. Another way of interpreting Q-learning is that it increases the estimate of the Q-value if the new estimate is larger than the old estimate, and reduces the estimate otherwise.

Finally, we would like to remark that given $(x_k, u_k, r(x_k,u_k), x_{k+1}) = (x,u,r,x')$, Q-learning only updates the Q-value of one state-action pair (x,u) instead of all $|\mathcal{X}| \times |\mathcal{U}|$ Q-values.

3.2 Examples and Codes

The grid world example

Recall the grid world example in Section 2.4.1. We will use this grid world example to illustrate

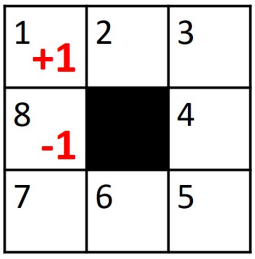


Figure 3.1: A simple grid world.

the Q-learning algorithm. Assume that current estimates of the Q values are as shown in Table 3.1. Given an experience $(x_k = 5, u_k = c, r_k = 0, x_{k+1} = 6)$, Q-learning updates $\tilde{Q}(5,c)$ as follows:

Table 3.1: The current estimates of the Q values in the grid world example.

(1,c)	(2,c)	(3,c)	(4,c)	(5,c)	(6,c)	(7,c)	(8, c)
1	0.4	0.38	0.14	0.2	-0.8	-0.9	-1.2
(1,cc)	(2,cc)	(3,cc)	(4,cc)	(5,cc)	(6,cc)	(7,cc)	(8,cc)
0.9	0.85	0.78	0.34	0.3	-0.5	-0.34	-1

$$\begin{aligned}\tilde{Q}(5,c) \longleftarrow & \tilde{Q}(5,c) + \beta_k \left(r_k + \alpha \max_v \tilde{Q}(x_{k+1},v) - \tilde{Q}(5,c)\right) \\ = & \tilde{Q}(5,c) + \beta_k \left(0 + \alpha \max\{\tilde{Q}(6,c), \tilde{Q}(6,cc)\} - \tilde{Q}(5,c)\right) \\ = & 0.2 + \beta_k \left(0 + \alpha \max\{-0.8, -0.5\} - 0.2\right).\end{aligned}$$

Web page recommendation problem

Recall the web page recommendation problem in Section 2.4.1. Suppose we know the number of web pages W. Suppose we do not know the probabilities of accepting recommendations $p(x, a)$ but have access to the user's visits, the recommendations and the rewards as we recommend web pages to the user. Then we can use Q-learning to solve the problem. The implementation code can be found in our GitHub repository [1] in the folder "web-page-recommendation-model-free".

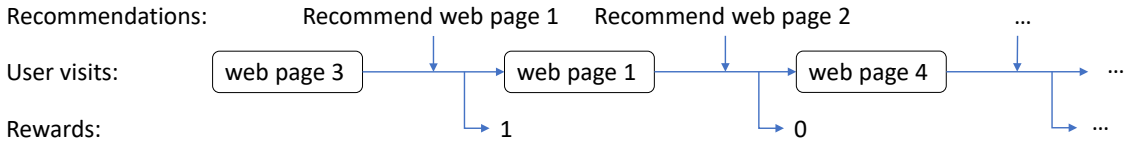


Figure 3.2: An example of the web page recommendation.

3.3 Off-Policy and On-Policy RL Algorithms

You may already notice in the introduction of the Q-learning algorithm, we assume we have access to some experiences (current state, current action, received reward, and next state), but we have not talked about how these experiences are generated and collected, in particular, under which policy these experiences were generated. Actually it is not critical for Q-learning. To generate experiences for training Q-learning, we can use any policy under which every state-action pair will be visited for a sufficient number of times. Since Q-learning learns the optimal policy from experiences generated from some other policies, the policy to be learned is different from the policy that generated the experiences (samples). Such an RL algorithm is called *off-policy*. In an off-policy RL algorithm, the policy to be learned is called the target policy and the policy that generates the samples is called the behavior policy. In Q-learning, the target policy is the optimal policy.

When the target policy and the behavior policy are the same, the RL algorithm is called *on-policy*. We will introduce an on-policy version of Q-learning, called SARSA.

3.4 SARSA

In this section, we will introduce a variation of the Q-learning algorithm, state-action-reward-state-action (SARSA) algorithm, which was proposed by Rummery and Niranjan in 1994 [17]. Different from Q-learning, SARSA is an on-policy RL algorithm.

Let $\tilde{Q}$ be the estimate of the Q value. Choose $\{\epsilon_k\}$ such that $\epsilon_k \to 0$ as $k \to \infty$. The SARSA algorithm is as follows.

SARSA

- At step k, with probability $1 - \epsilon_k$, choose action u_k such that

$$u_k \in \arg\max_v \tilde{Q}(x_k, v)$$

and with probability ϵ_k, choose an action u_k uniformly at random.

- Apply action u_k and receive next state x_{k+1} and the reward $r(x_k, u_k)$.
- With experience $(x_{k-1}, u_{k-1}, r(x_{k-1}, u_{k-1}), x_k, u_k)$, update Q such that

$$\begin{aligned}\tilde{Q}(x_{k-1}, u_{k-1}) \longleftarrow & (1-\beta_k)\tilde{Q}(x_{k-1}, u_{k-1}) + \beta_k(r(x_{k-1}, u_{k-1}) + \alpha\tilde{Q}(x_k, u_k)) \\ & = \tilde{Q}(x_{k-1}, u_{k-1}) + \beta_k\left(r(x_{k-1}, u_{k-1}) + \alpha\tilde{Q}(x_k, u_k) - \tilde{Q}(x_{k-1}, u_{k-1})\right)\end{aligned}$$

- Go to the next step $k+1$.

You may notice that SARSA looks very similar to Q-learning. However, they are different because SARSA is an on-policy algorithm while Q-learning is an off-policy algorithm. In SARSA, the experience $(x_{k-1}, u_{k-1}, r(x_{k-1}, u_{k-1}), x_k, u_k)$ is generated by following a policy based on the current Q-value estimates, which chooses a greedy action $u_k \in \arg\max_v \tilde{Q}(x_k, v)$ with probability $1-\epsilon_k$ and a random action with probability ϵ_k. This is called an ϵ-greedy policy. In the update formula of $\tilde{Q}$, we can see that the new estimate, $r(x_{k-1}, u_{k-1}) + \alpha\tilde{Q}(x_k, u_k)$, is an estimate of the Q-value following the same policy (the ϵ-greedy policy) because this u_k was also used to generate the samples (no max operator in the update formula). Therefore, the policy for which the agent is learning the Q-values is also the ϵ-greedy policy, which means that SARSA is an on-policy algorithm. We also note that at iteration k, SARSA updates the Q-value for state-action pair (x_{k-1}, u_{k-1}) instead of (x_k, u_k).

3.5 Examples and Codes

The grid world example

Recall the grid world example in Section 2.4.1. We will also use this grid world example to illustrate

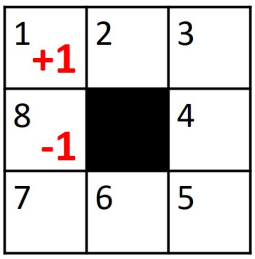


Figure 3.3: A simple grid world.

the SARSA algorithm. Assume that at time k, the estimate of the Q-value is shown in Table 3.2. Assume that we are given an experience, $x_{k-1} = 4$, $u_{k-1} = cc$, $r_{k-1} = 0$, and $x_k = 5$. Then the SARSA algorithm chooses action

$$u_k = \arg\max_{c,cc}\{\tilde{Q}(5, c) = 0.2, \tilde{Q}(5, cc) = 0.3\} = cc.$$

Table 3.2: The current estimate of the Q values in the grid world example.

(1,c)	(2,c)	(3,c)	(4,c)	(5,c)	(6,c)	(7,c)	(8, c)
1	0.4	0.38	0.14	0.2	-0.8	-0.9	-1.2
(1,cc)	(2,cc)	(3,cc)	(4,cc)	(5,cc)	(6,cc)	(7,cc)	(8,cc)
0.9	0.85	0.78	0.34	0.3	-0.5	-0.34	-1

with probability $1-\epsilon_k$ and chooses an action uniformly at random with probability ϵ_k. Suppose it turns out that the algorithm chose the action $u_k = cc$. Then the algorithm updates the estimate of $\tilde{Q}(4, cc)$ as follows:

$$\begin{aligned}\tilde{Q}(4,cc) \longleftarrow & \tilde{Q}(4,cc) + \beta_k \left(r_{k-1} + \alpha \tilde{Q}(x_k, u_k) - \tilde{Q}(4,cc)\right) \\ &= \tilde{Q}(4,cc) + \beta_k \left(0 + \alpha \tilde{Q}(5,cc) - \tilde{Q}(4,cc)\right) \\ &= 0.34 + \beta_k \left(0 + \alpha \times 0.3 - 0.34\right).\end{aligned}$$

Web page recommendation problem

Recall the web page recommendation problem in Section 2.4.1. Consider the same setting as that in Section 3.2, where we do not know the model but have access to the data. Then we can also use SARSA to solve the problem. The implementation code can be found in our GitHub repository [1] in the folder "web-page-recommendation-model-free".

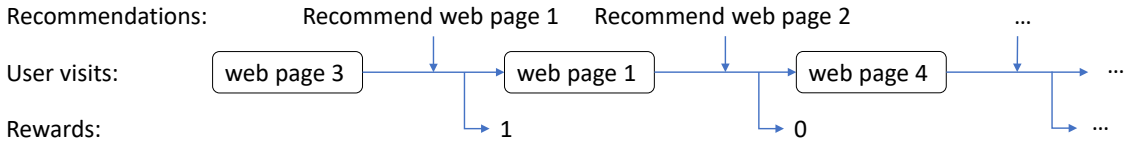


Figure 3.4: An example of the web page recommendation.

On-Policy versus Off-Policy

Target policy: The policy to be learned
Behavior policy: The policy used to generate samples

- Q-learning: target policy - optimal policy
  behavior policy - any policy under which each state-action pair is visited infinitely often
- SARSA: target policy - ϵ-greedy
  behavior policy - ϵ-greedy

Chapter 4

Q-Learning with Function Approximation: Linear Function Approximation and Deep Q-Learning

So far, we focused on tabular RL algorithms for problems with finite state and action spaces. While model-free algorithms such as Q-learning and SARSA, resolve the difficulty of learning the model. Tabular approaches face the problem of scalability and the curse-of-dimensionality. For example, a simplified, 9×9 Go game has 10^{35} states, which makes it computationally impossible to use a tabular method to solve the problem.

Despite this difficulty, Alpha-Go and Alpha-Zero have defeated the best Go players in the world. The key to their successes is to use function approximation, in particular, using deep neural networks to approximate value functions and parameterize policies. To understand function approximations, we start with linear function approximation in this chapter and then introduce nonlinear function approximation.

4.1 Linear Function Approximation

Linear function approximation [6] has been successfully used in RL algorithms that play games like Tetris. Under linear function approximation, we assume that the value function or the Q function can be approximated using a linear function. Considering linear function approximation for Q function, we assume

$$Q(x, u) \approx \sum_{m=1}^{M} w_m \phi_m(x, u) = w^\top \phi(x, u),$$

where

- $\phi(x, u)$: M-dimensional feature vector for state-action pair (x, u), which is assumed to be known, and
- w: M-dimensional weight vector, which is common for all (state, action) pairs.

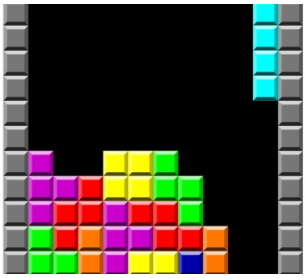


Figure 4.1: Tetris Game

For the Tetris game as shown in Figure 4.1[1], the features may include the height of the highest column, differences in height between neighboring columns, holes, and wells.

With linear function approximation, learning $Q(x,u)$ ($|\mathcal{X}| \times |\mathcal{U}|$) pairs) is equivalent to learning the weight vector w because the Q-values for all state and action pairs are determined when the weight vector w is given. There are two fundamental advantages of using this linear function approximation: computation and generalization.

- If $M << |\mathcal{X}| \times |\mathcal{U}|$, the computation complexity reduces significantly because the algorithm only needs to learn M values instead of $|\mathcal{X}| \times |\mathcal{U}|$ values.
- Learning w also allows us to generalize when facing a new state-action pair. Under tabular algorithms, the Q-value of (x,u) is only updated when given a sample whose current state and action pair is (x,u). Therefore, a tabular algorithm will not be able to learn the Q-values of unseen state and action pairs. However, under linear function approximation, even for an unseen state and action pair (x,u), we can estimate its Q-value with $w^\top \phi(x,u)$. So Q-learning with linear function approximation has the generalization power that the tabular Q-learning does not have.

We next look at the implementation of Q-learning with linear function approximation. With linear function approximations, instead of maintaining a Q table, the agent maintains the weight vector w, the weight vector, based on the data samples we collect. We first recall the tabular Q-learning updates Q-values based on the temporal-difference:

$$\delta_k = r(x_k,u_k) + \alpha \max_v \tilde{Q}(x_{k+1},v) - \tilde{Q}(x_k,u_k).$$

We next explain the tabular Q-learning algorithm as a gradient descent algorithm to minimize the temporal difference δ_k^2. To see this, consider the following optimization problem, where we minimize the temporal difference by optimizing the Q-value of current state action pair $\tilde{Q}(x_k,u_k)$:

$$\min_Q \frac{1}{2}\delta_k^2 = \min_Q \frac{1}{2}\left(r(x_k,u_k) + \alpha \max_v \tilde{Q}(x_{k+1},v) - Q\right)^2.$$

A gradient descent algorithm to minimize a function $f(x)$ is to move x along the opposite direction of the gradient $\triangledown f(x)$, which is the direction the value of $f(x)$ decreases. The gradient descent algorithm therefore is

$$x \leftarrow x - \beta \triangledown f(x),$$

[1]Picture is from `https://en.wikipedia.org/wiki/Tetris`

where β is the step size. Applying the idea to minimize the temporal difference error, we obtain

$$\begin{aligned}\tilde{Q}(x_k,u_k) \leftarrow & \tilde{Q}(x_k,u_k) - \beta_k \frac{\partial \frac{1}{2}\delta_k^2}{\partial \tilde{Q}(x_k,u_k)} \\ = & \tilde{Q}(x_k,u_k) + \beta_k \left(r(x_k,u_k) + \alpha \max_v \tilde{Q}(x_{k+1},v) - \tilde{Q}(x_k,u_k) \right).\end{aligned}$$

We now apply the same idea to derive the Q-learning algorithm with linear function approximation. Note that with linear function approximation, the temporal difference is

$$\delta_k(w) = r(x_k,u_k) + \alpha \max_v w^\top \phi(x_{k+1},v) - w^\top \phi(x_k,u_k).$$

Following the same idea, we update the weight vector w using the gradient descent algorithm to minimize an objective function defined by the temporal difference $\frac{1}{2}\delta_k^2(w)$. Note that this function is not easily differentiable because $\alpha \max_v w^\top \phi(x_{k+1},v)$ has a max operator so it is not differentiable at some point. Therefore, we will treat δ_k differently and view only the third term as a function of w. Let w_k denote the weight vector at iteration k. We consider the following optimization problem for updating w :

$$\min_w \frac{1}{2} \left(r(x_k,u_k) + \alpha \max_v w_k^\top \phi(x_{k+1},v) - w^\top \phi(x_k,u_k) \right)^2,$$

where we replace the optimization variable w in the second term with w_k and do not view it as an optimization variable anymore. Therefore, the gradient descent algorithm becomes

$$w \leftarrow w + \beta_k \left(r(x_k,u_k) + \alpha \max_v w_k^\top \phi(x_{k+1},v) - w^\top \phi(x_k,u_k) \right) \phi(x_k,u_k).$$

Note that w_k has the same value with w before updating. Therefore, the algorithm can be written as

$$w \leftarrow w + \beta_k \delta_k(w) \phi(x_k,u_k).$$

Q-Learning with Linear Function Approximation

Given an experience (current state $x_k = x$, current action $u_k = u$, received reward $r(x_k,u_k) = r$, and next state $x_{k+1} = x'$), Q-learning with linear function approximation updates w as follows:

$$w \longleftarrow w + \beta_k \delta(w) \phi(x,u),$$

where

$$\delta(w) = r(x,u) + \alpha \max_v w^\top \phi(x',v) - w^\top \phi(x,u).$$

4.2 Examples and Codes

Backgammon game

Backgammon game is a classic two-player game in which players take turns and move their checkers (Backgammon playing pieces) according to the roll of the dice. The starting board is shown in Figure

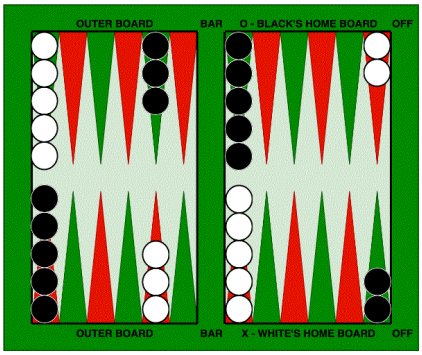


Figure 4.2: Backgammon Game

4.2[2]. At each turn, the player first rolls two dice. Then the player moves their checkers around the board towards their home board according to the two dice rolled. Once all of the player's checkers arrive at their home board, they can start to remove (called "bear off") their checkers from the board. The first player to bear off all their checkers wins the game. The game can be formulated into an infinite-horizon discounted MDP as follows.

- State: the board configuration and the roll of the dice at our turn.
- Action: the move of the checkers at our turn.
- Transition:

  The current state is the board configuration and the roll of the dice. The current action is moving our checkers. The next state is defined as the board configuration of our next turn and the roll of the dice of our next turn after the opponent plays their move.
- Reward: +1 if we win, and 0 if the opponent wins. The reward is zero for all the intermediate steps.
- Objective: maximize the expected total discounted reward.

We can use Q-learning with linear function approximation to train an agent to play this game. The features may include the number of checkers in each column, the number of bar checkers, the number of off checkers, pip count, blots, blocked points, and prime strength[3]. The implementation code can be found in our GitHub repository [1] in the folder "backgammon-game".

4.3 Q-Learning with Nonlinear Function Approximation

The limitation of linear function approximation is that we have to select the features, oftentimes, hand-crafted features. In this section, we introduce nonlinear function approximation so we can use deep neural networks for approximating Q-functions, which leads to the Deep Q Network (DQN) where the Q-network learns the features automatically.

[2]Picture is from `https://github.com/dellalibera/gym-backgammon`

[3]These concepts can be found in `https://bkgm.com`

Consider nonlinear function approximation parameterized by w, i.e. $\tilde{Q}(x,u;w)$ is an estimate of $Q(x,u)$, where parameters w define the function and (x,u) is the input to the function.

Given an experience $(x_k,u_k,r(x_k,u_k),x_{k+1})=(x,u,r,x')$, the temporal difference is

$$\delta_k(w)=r+\alpha\max_v\tilde{Q}(x',v;w)-\tilde{Q}(x,u;w).$$

The (improper) gradient of $\frac{1}{2}\delta_k^2(w)$, where we again view $\max_v\tilde{Q}(x',v;w)$ as a constant instead of a function of w to avoid differentiating a max function, is

$$\tilde{\nabla}\frac{1}{2}\delta_k^2(w)=-\left(r+\alpha\max_v\tilde{Q}(x',v;w)-\tilde{Q}(x,u,w)\right)\nabla\tilde{Q}(x,u;w).$$

The gradient descent algorithm is

$$\begin{aligned}w\longleftarrow&w+\beta_k\left(r+\alpha\max_v\tilde{Q}(x',v;w)-\tilde{Q}(x,u,w)\right)\nabla\tilde{Q}(x,u;w)\\=&w+\beta_k\delta(w)\nabla\tilde{Q}(x,u;w).\end{aligned}$$

Comparing it with Q-learning with linear function approximation, the only difference is that the agent needs to calculate $\nabla\tilde{Q}(x,u;w)$. Note that the reward $r(x_k,u_k)$ is from the experience and $\max_v\tilde{Q}(x',v;w)$ and $\tilde{Q}(x,u;w)$ can be easily computed from the approximator with weight w.

4.4 Deep Q-Learning

To update the weight vector using the gradient descent algorithm, the key is to figuring out how to compute

$$\nabla_w\tilde{Q}(x,u;w)?$$

With the recent success of deep RL, where neural networks are used for function approximation, we will focus on answering this question by considering neural network approximations, where $\tilde{Q}(x,u;w)$ is a neural network and w are the weights of the neural net.

The answer to the question is to connect it to the training a neural network using the gradient descent algorithm. By defining a correct loss function, the gradient can be easily calculated using existing tools such as PyTorch or TensorFlow.

4.4.1 A Supervised Learning Perspective

When training of a neural network for a supervised learning problem, the gradient descent algorithm is to minimize the loss function $\mathcal{L}(w)$ with respect to weights w with dataset $\{(x_k,y_k)\}_k$, where y_k is the label of data sample x_k. Consider the following square loss function

$$\mathcal{L}(w)=\frac{1}{n}\sum_{k=1}^{n}(\hat{y}(x_k;w)-y_k)^2,$$

where $\hat{y}(x_k;w)$ is the estimate of label y_k with weight w. The gradient descent algorithm, which can be implemented using a few lines of standard commands, is to update w as follows:

$$w\leftarrow w-\nabla\mathcal{L}(w).$$

Now to train a Q-network like a supervised learning problem, we can do so by defining the input data, the label and the loss function as follows:

- input data sample is (x_k, u_k).
- the label of input (x_k, u_k) is its Q-value estimated from the next state, which is $y_k = r_k + \alpha \max_v \tilde{Q}(x_{k+1}, v; w)$, and
- the estimator is the output of the Q-network $\hat{y}(x_k, u_k; w) = \tilde{Q}(x_k, u_k; w)$.

Since we view the label of a data sample as the ground truth and should not depend on the estimator, y_k is treated as a constant independent of parameters w. In PyTorch, we will .detach() so the label y_k becomes a number instead of function of w.

While tabular Q-learning is guaranteed to converge, there is currently no theoretical guarantees on optimality and convergence of deep Q-learning algorithms. In fact, it has been observed that the training of deep Q-learning algorithms can be quite unstable if the vanilla version is implemented. We next introduce a few ideas that make deep Q-learning more stable in practice. Specifically, we will introduce experience replay, target network, and how to implement deep Q-learning algorithms using PyTorch.

4.4.2 Replay Buffer and Experience Replay

Suppose we have a supervised learning problem

$$\frac{1}{2} \min_w \frac{1}{N} \sum_{k=1}^{N} \Big(y_k - \tilde{Q}(x_k, u_k; w) \Big)^2,$$

where $y_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v; w)$ is the label of sample (x_k, u_k). Recall that here we ignore the fact that y_k is a function of w like in linear function approximation. Then the gradient descent algorithm updates w as follows when given data sample $((x_k, u_k), y_k)$:

$$w \leftarrow w + \beta_k (y_k - \tilde{Q}(x_k, u_k; w)) \nabla_w \tilde{Q}(x_k, u_k; w).$$

A critical assumption in supervised learning is that the samples are *independently and identically distributed (i.i.d.)*. However, for RL applications, data samples

$$(y_1, (x_1, u_1)) \to (y_2, (x_2, u_2)) \to (y_3, (x_3, u_3)) \to \cdots$$

are highly correlated because x_{i+1} is a result of x_i and u_i.

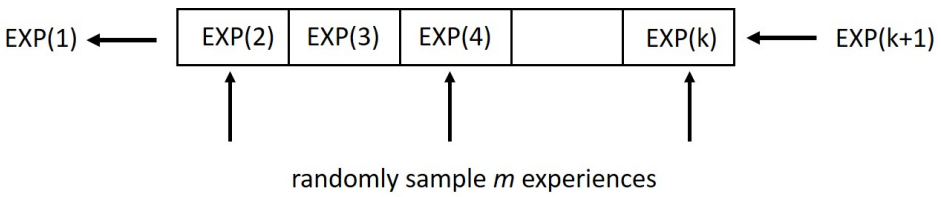


Figure 4.3: Replay buffer and experience replay.

In order to "decorrelate" the samples, in practice, replay buffer and experience replay are used as shown in Figure 4.3. The idea of experience replay is to maintain a replay buffer, as shown in

Fig. 4.3. Here, we call a data sample $(x_k, u_k, x_{k+1}, r(x_k, u_k))$ an experience. In Fig. 4.3, "EXP(k)" means the k-th experience we collected from the interaction with the environment. Each time when we get a new experience, we will store it into the replay buffer. If the buffer is full, then the oldest experience will be discarded. Each time before we train the neural network, we will randomly sample m experiences (called a mini-batch) from the buffer and then train the network using mini-batch SGD, shown as follows:

$$w \leftarrow w + \frac{\beta}{m} \sum_{b=1}^{m} (y_{k_b} - \tilde{Q}(x_{k_b}, u_{k_b}; w)) \nabla_w \tilde{Q}(x_{k_b}, u_{k_b}; w),$$

where b is the index of one of the m samples in the mini-batch, and k_b is the time index of the bth experience in the mini-batch, in other words, the bth experience is $(x_{k_b}, u_{k_b}, r(x_{k_b}, u_{k_b}), x_{k_b+1})$. By maintaining a large replay buffer, the samples become less correlated with experience replay.

4.4.3 Target Network

From the supervised learning perspective, the target value $y_k = r(x_k, u_k) + \alpha \max_v \tilde{Q}(x_{k+1}, v; w)$ should be independent of parameter w, which is not the case in RL. The idea of target network is used to weaken this dependency.

In addition to the Q-network, We use another neural network called target Q-network Q_{target} in the training. Let w and w_{target} be the weights of the original Q-network (also called online Q-network) and the target Q-network, respectively. We copy w to w_{target} once every n iterations and use Q_{target} to calculate y_k. In this way, the target value y_k becomes less dependent on current parameters w.

4.4.4 Deep Q-Learning (DQN) in PyTorch

Fig. 4.4 shows two different architectures of deep Q-networks. For finite action space, we usually use the architecture on the right, where the input is state x and the output is the Q-values associated with the state x, i.e. a $|\mathcal{U}|$ dimensional vector. This architecture couples the estimation of Q-values $(Q(x, u))$ of different actions for the same state x. The input to the last layer can be interpreted as features of the input state x and the weights of the last layer are related to different actions u. Therefore, the features are the same for different actions, but the weights are different. Note that we will use the left architecture in Fig. 4.4 when the action space becomes too large or becomes continuous because the output dimension becomes too large to maintain.

DQN in PyTorch

The following is one iteration of the training:

1. Sample a mini-batch $\{(x_{k_b}, u_{k_b}, r_{k_b}, x_{k_b+1})\}_{b=1,\cdots,m}$ from the replay buffer.
   `random.sample(replay_buffer, batch_size)`
2. Generate target-values $\{y_{k_b}\}_{b=1,\cdots,m}$ using the target network.
   `next_state_values = target_net(next_states).max(1)[0]`
   `target_values= rewards + alpha * next_state_values.detach()`

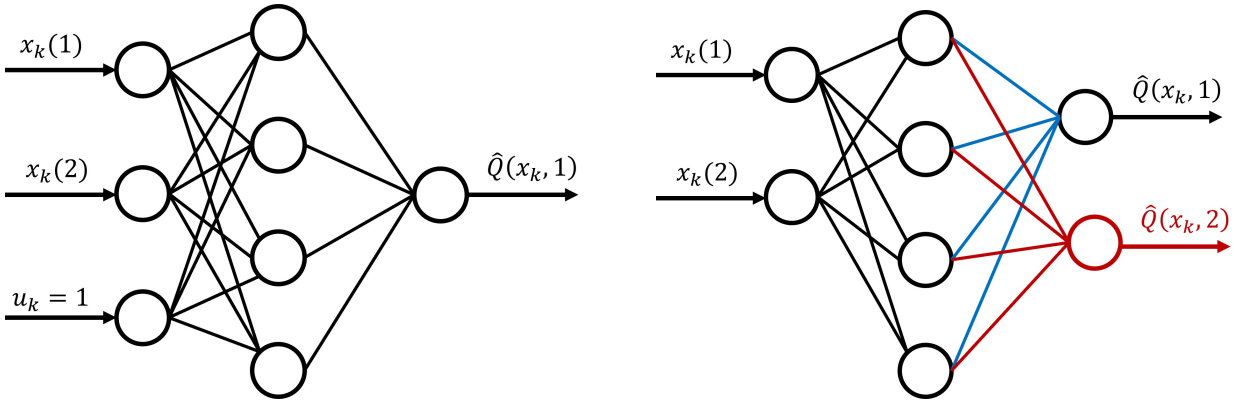


Figure 4.4: Architectures of deep Q-networks

3. Calculate the current Q value $\tilde{Q}(x_{k_b}, u_{k_b}; w)$ using the online network.
   current_Q_values = online_net(current_states).gather(1, current_actions)
4. Define a loss function $\mathcal{L}(w) = \frac{1}{m}\sum_{b=1}^{m}\left(\tilde{Q}(x_{k_b}, u_{k_b}; w) - y_{k_b}\right)^2$.
   loss = torch.nn.MSELoss()(current_Q_values, target_values)
5. Update $w \leftarrow w - \beta\nabla_w\mathcal{L}(w)$.
   optimizer.zero_grad() loss.backward() optimizer.step()

The target network will be updated once every n iterations.
target_net.load_state_dict(online_net.state_dict())

Next, we will show how to train the DQN [15] using PyTorch. The last step uses the automatic differentiation mechanism in PyTorch, which uses the back propagation algorithm to calculate the gradients of the loss function with respect to the weights in the network.

Chapter 5

Variations of DQN

It has been observed in practice that Q-learning with function approximation often overestimates the Q-values [23]. This overestimation issue because of the following fundamental inequality: Given N independent random variables $\{Y_i\}_{i=1,\cdots,N}$,

$$\mathbb{E}\left[\max_i Y_i\right] \geq \max_i \mathbb{E}\left[Y_i\right].$$

This inequality is fairly easy to verify. For example, assuming Y_i are i.i.d. Bernoulli random variables with mean 0.5, we have

$$\max_i \mathbb{E}[Y_i] = \max_i 0.5 = 0.5$$

but

$$\lim_{N\to\infty} \mathbb{E}\left[\max_{i=1,\cdots,N} Y_i\right] = 1$$

because

$$\Pr\left(\max_{i=1,\cdots,N} Y_i = 1\right) = 1 - \Pr\left(Y_i = 0 \ \forall i\right) = 1 - 0.5^N.$$

Assume the following function approximation of Q-values

$$\tilde{Q}(x,u) = Q(x,u) + Y(x,u),$$

where $Y(x,u) = Q(x,u) - \tilde{Q}(x,u)$ is the estimation error (or noise). Assume $Y(x,u)$ is a random variable and $\mathbb{E}[Y(i,u)] = 0$ (zero mean). For simplicity, assume the rewards are deterministic.

Given experience $(x, u, r(x,u), x')$, Q-learning updates $\tilde{Q}$ as follows:

$$\tilde{Q}_{\text{new}}(x,u) \leftarrow \tilde{Q}(x,u) + \beta\left(r(i,u) + \alpha \max_v \tilde{Q}(x',v) - \tilde{Q}(x,u)\right).$$

If the correct Q-values are known, then we have

$$Q_{\text{new}}(x,u) \leftarrow Q(x,u) + \beta\left(r(i,u) + \alpha \max_v Q(x',v) - Q(x,u)\right).$$

We next analyze $Z(x,u) = \tilde{Q}_{\text{new}}(x,u) - Q_{\text{new}}(x,u)$ the estimation error due to noise in function approximation:

$$\begin{aligned} Z(x,u) &= \tilde{Q}(x,u) - Q(x,u) + \beta\left(r(x,u) + \alpha \max_v \tilde{Q}(x',v) - \left(r(x,u) + \alpha \max_v Q(x',v)\right)\right) \\ &= \tilde{Q}(x,u) - Q(x,u) + \beta\alpha\left(\max_v \tilde{Q}(x',v) - \max_v Q(x',v)\right) \\ &= Y(x,u) - \beta\alpha\left(\max_v Q(x',v) - \max_v \left(Q(x',v) + Y(x',v)\right)\right). \end{aligned}$$

We next use an example to show that $\mathbb{E}[Z(x,u)] > 0$ even when $\mathbb{E}[Y(x,u)] = 0$ for all x and u.

Consider a special case where

$$Q(x,u) = Q(x,v) \quad \forall u \neq v,$$

i.e. Q-values are the same for all actions in the same state x. Assume $Y(x,u)$ takes value from $\{-\epsilon, \epsilon\}$ equally likely and are independent across the actions. Then we have

$$\begin{aligned} \mathbb{E}[Z(x,u)] &= \beta\alpha\mathbb{E}\left[-Q(x',u) + Q(x',u) + \max_v Y(x',v)\right] \\ &= \beta\alpha\mathbb{E}\left[\max_v Y(x',v)\right]. \end{aligned}$$

Let $n = |\mathcal{U}|$ be the number of actions. We have

$$\begin{aligned} \mathbb{E}[Z(x,u)] =& \beta\alpha\mathbb{E}\left[\max_v Y(x',v)\right] \\ =& \beta\alpha\left(\epsilon \Pr(\max_v Y(x',v) = \epsilon) - \epsilon \Pr(\max_v Y(x',v) = -\epsilon)\right) \\ =& \beta\alpha\epsilon\left(1 - 2\Pr\left(Y(x',v) = -\epsilon \ \forall v\right)\right) \\ =& \beta\alpha\epsilon\left(1 - 0.5^{n-1}\right) \\ >& 0. \end{aligned}$$

This example shows that even when $\tilde{Q}(x,u)$ is an unbiased estimate of Q since $Y(x,u)$ is zero mean, the new Q-values estimated based on $\tilde{Q}$ using Q-learning overestimates Q.

5.1 Double Q-Learning and Double DQN

In order to overcome the overestimation issue, double Q-learning algorithm [8] was proposed. To understand double Q-learning, recall that the overestimation is due to the following inequality: Given N independent random variables $\{Y_i\}_{i=1,\cdots,N}$,

$$\mathbb{E}\left[\max_i Y_i\right] \geq \max_i \mathbb{E}\left[Y_i\right].$$

Consider N random variables $(Y_i)_{i=1,\cdots,N}$. For each random variable Y_i, we obtain a set of i.i.d. samples, denoted by $\mathcal{S}_i$. We want to estimate the maximum $\max_i \mathbb{E}[X_i]$. The single estimator

$$\max_i \frac{1}{|\mathcal{S}_i|} \sum_{s \in \mathcal{S}_i} y_{i,s}$$

overestimates the maximum because

$$\mathbb{E}\left[\max_i \frac{1}{|\mathcal{S}_i|}\sum_{s\in\mathcal{S}_i} y_{i,s}\right] \geq \max_i \frac{1}{|\mathcal{S}_i|}\mathbb{E}\left[\sum_{s\in\mathcal{S}_i} y_{i,s}\right] = \max_i \mathbb{E}[Y_i].$$

We now consider a double estimator.

Double Estimator

1. Split each $\mathcal{S}_i$ into $\mathcal{S}_i^{(1)}$ and $\mathcal{S}_i^{(2)}$ such that $\mathcal{S}_i^{(1)} \cup \mathcal{S}_i^{(2)} = \mathcal{S}_i$ and $\mathcal{S}_i^{(1)} \cap \mathcal{S}_i^{(2)} = \varnothing$.
2. Define
$$\mu_i^{(1)} = \frac{1}{\left|\mathcal{S}_i^{(1)}\right|}\sum_{s\in S_i^{(1)}} y_{i,s}, \qquad \mu_i^{(2)} = \frac{1}{\left|\mathcal{S}_i^{(2)}\right|}\sum_{s\in S_i^{(2)}} y_{i,s}.$$
3. Pick i^* such that $\mu_{i^*}^{(1)} = \max_i \mu_i^{(1)}$ and approximate $\max_i \mathbb{E}[Y_i]$ with $\mu_{i^*}^{(2)}$.

The key idea of a double estimator is that it separates finding maximizer frm evaluating the maximizer. The estimator selects the maximizer from the $\mathcal{S}_i^{(1)}$ and then evaluates the mean of the maximizer using $\mathcal{S}_i^{(2)}$. Interestingly, the double estimator is an underestimator.

In the double estimator, if i^* is the maximizer, then

$$\mathbb{E}\left[\mu_{i^*}^{(2)}\right] = \mathbb{E}[Y_{i^*}] = \max_i \mathbb{E}[Y_i].$$

If not, $\mathbb{E}[\mu_{i^*}^{(2)}] < \max_i \mathbb{E}[X_i]$. Therefore,

$$\mathbb{E}\left[\mu_{i^*}^{(2)}\right] \leq \max_i \mathbb{E}[X_i].$$

Double Q-learning is based on double estimator, where we maintain two Q-function estimators (such as neural networks). Denote the parameters of these two estimators by w and w'.

Double Q-Learning

Given experience $(x_k, u_k, r(x_k, u_k), x_{k+1}) = (x, u, r, x')$, the algorithm randomly picks one estimator to update as follows.

- If Estimator 1 is chosen, then
$$w \leftarrow w + \beta_k \delta \nabla_w Q(x, u; w) \quad \text{with}$$
$$\delta = r + \alpha Q\left(x', \arg\max_v Q(x', v; w); w'\right) - Q(x, u; w).$$
- If Estimator 2 is chosen, then
$$w' \leftarrow w' + \beta \delta'_k \nabla_{w'} Q(x, u; w') \quad \text{with}$$
$$\delta' = r + \alpha Q\left(x', \arg\max_v Q(x', v; w'); w\right) - Q(x, u; w').$$

Double DQN algorithm [24] combines double Q-learning and DQN. In DQN, the target network is used to select the action u and evaluate the target value. In other words,

$$y_k = r(x_k, u_k) + \alpha \tilde{Q}(x_{k+1}, \arg\max_v Q(x_{k+1}, v; w_{\text{target}}); w_{\text{target}}).$$

Since selecting the optimal action and evaluating the Q-function together lead to an overestimation, double DQN algorithm separates action selection and evaluation. The update of the target network remains the same. For the target value y_k, the action is selected by the online network and the value is evaluated by the target network, i.e.,

$$\begin{aligned} u_{k+1} &= \arg\max_v \tilde{Q}(x_{k+1}, v; w_{\text{online}}) \\ y_k &= r(x_k, u_k) + \alpha \tilde{Q}(x_{k+1}, u_{k+1}; w_{\text{target}}), \end{aligned}$$

where w_{online} and w_{target} are the weights of the online network and the target network, respectively.

Double DQN in PyTorch

The following is one iteration of the training:

1. Sample a mini-batch $\{(x_{k_b}, u_{k_b}, r_{k_b}, x_{k_b+1})\}_{b=1,\cdots,m}$ from the replay buffer.
   random.sample(replay_buffer, batch_size)
2. Select the best next action using the online network.
   best_next_actions = torch.argmax(online_net(next_states), dim=1, keepdim=True)
3. Generate target-values $\{y_{k_b}\}_{b=1,\cdots,m}$ using the target network.
   next_state_values = target_net(next_states).gather(1, best_next_actions)
   target_values= rewards + alpha * next_state_values.detach()
4. Calculate the current Q value $\tilde{Q}(x_{k_b}, u_{k_b}; w_{\text{online}})$ using the online network.
   current_Q_values = online_net(current_states).gather(1, current_actions)
5. Define a loss function $\mathcal{L}(w_{\text{online}}) = \frac{1}{m}\sum_{b=1}^{m}\left(\tilde{Q}(x_{k_b}, u_{k_b}; w_{\text{online}}) - y_{k_b}\right)^2$.
   loss = torch.nn.MSELoss()(current_Q_values, target_values)
6. Update $w_{\text{online}} \leftarrow w_{\text{online}} - \beta \nabla_{w_{\text{online}}} \mathcal{L}(w_{\text{online}})$.
   optimizer.zero_grad() loss.backward() optimizer.step()

The target network will be updated once every n iterations.
target_net.load_state_dict(online_net.state_dict())

5.2 Clipped Double Q-Learning

Clipped double-Q learning algorithm [7] maintains two Q-networks as in DDQN. Different from double Q-learning, clipped double Q-learning uses both Q-networks to select the action and evaluate the Q-value of the selected action, but it chooses the smaller value of the two as the target value.

Clipped Double Q-Learning

Denote the weights of the two networks by w and w', respectively. Clipped double-Q learning algorithm updates the weights based on experience $(x_k, u_k, r(x_k, u_k), x_{k+1}) = (x, u, r, x')$ as follows:

$$w \leftarrow w + \beta_k(y - \tilde{Q}(x, u; w))\nabla_w \tilde{Q}(x, u; w)$$
$$w' \leftarrow w' + \beta_k(y - \tilde{Q}(x, u; w'))\nabla_{w'} \tilde{Q}(x, u; w')$$

where

$$y = r + \alpha \min\left\{\max_v \tilde{Q}(x', v; w), \max_v \tilde{Q}(x', v; w')\right\}.$$

Note that while each Q-network both selects the action and evaluates the Q-value of the action, clipped double Q-learning deals with the overestimation issue by choosing a smaller of the two Q-values. Numerical studies have confirmed its effectiveness.

Clipped Double DQN in PyTorch

The following is one iteration of the training:

1. Sample a mini-batch $\{(x_{k_b}, u_{k_b}, r_{k_b}, x_{k_b+1})\}_{b=1,\cdots,m}$ from the replay buffer.
   random.sample(replay_buffer, batch_size)
2. Generate target-values $\{y_{k_b}\}_{b=1,\cdots,m}$ using the target networks.
   next_values_1 = target_net_1(next_states).max(1)[0]
   next_values_2 = target_net_2(next_states).max(1)[0]
   target_values = rewards + alpha * torch.min(next_values_1, next_values_2).detach()
3. Calculate the current Q values $\tilde{Q}(x_{k_b}, u_{k_b}; w)$ and $\tilde{Q}(x_{k_b}, u_{k_b}; w')$ using the online networks.
   current_Q_values_1 = online_net_1(current_states).gather(1, current_actions)
   current_Q_values_2 = online_net_2(current_states).gather(1, current_actions)
4. Define a loss function
   $\mathcal{L}(w, w') = \frac{1}{m}\sum_{b=1}^m \big(\tilde{Q}(x_{k_b}, u_{k_b}; w) - y_{k_b}\big)^2 + \frac{1}{m}\sum_{b=1}^m \big(\tilde{Q}(x_{k_b}, u_{k_b}; w') - y_{k_b}\big)^2$.
   loss = torch.nn.functional.mse_loss(current_Q_values_1, target_values)
   + torch.nn.functional.mse_loss(current_Q_values_2, target_values)
5. Update $w \leftarrow w - \beta\nabla_w \mathcal{L}(w, w')$, $w' \leftarrow w' - \beta\nabla_{w'} \mathcal{L}(w, w')$.
   optimizer.zero_grad() loss.backward() optimizer.step()

The target networks will be updated once every n iterations.
target_net_1.load_state_dict(online_net_1.state_dict())
target_net_2.load_state_dict(online_net_2.state_dict())

Chapter 6

Policy Gradient Algorithms

In Chapter 2, we introduced two approaches for solving the Bellman equation: value iteration and policy iteration. Q-learning follows the value iteration approach, and learns the Q-values that solves the Bellman equation. After learning the Q-values, and the resulted policy is a greedy-policy that at each state, takes the action with the largest Q-value. Q-learning algorithms work well when the action space is finite (the state space can be infinite or continuous). For example, in a deep-Q network, each output of the Q-network is the Q-value for a specific action as shown in Figure 6.1.

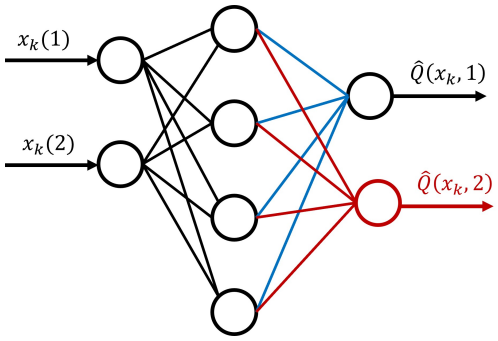


Figure 6.1: A Deep Q Network

We note that this neural network architecture does not work for applications with large even infinite action spaces, like in robotics where the action space may be continuous. In such a case, it is impossible to output all Q-values. One may change the architecture such that the Q-network takes both state and action as input and outputs the Q-value for the given state-action pair as shown in Figure 6.2. The problem of this approach is that solving the maximizing problem for selecting the action $\max_u \tilde{Q}(x, u)$ is still difficult. We next introduce algorithms that follow the idea of policy iteration, called policy gradient.

6.1 Generalized Policy Iteration

Before introducing policy gradient algorithms, we first recall the policy iteration algorithm. Under policy iteration, given policy μ_k, it includes two steps: policy evaluation and policy improvement.

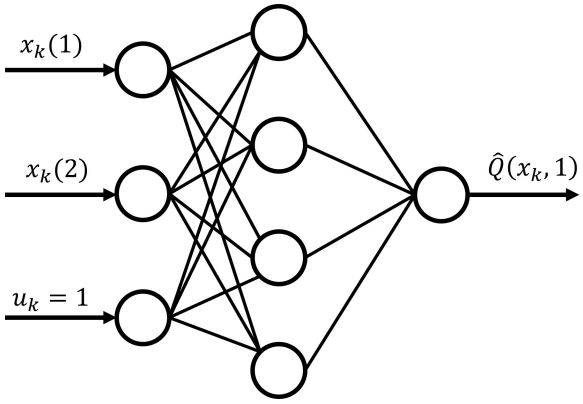


Figure 6.2: Deep Q-Network with Action as Input

Policy Iteration

- Choose an initial policy μ_0, set $k = 0$.
- **Policy Evaluation:** Let V_{μ_k} denote the value function of the policy μ_k. Compute V_{μ_k} by solving the Bellman equation:

$$V_{\mu_k}(x) = \mathbb{E}[r(x, \mu_k(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_k(x)) V_{\mu_k}(x') \quad \text{for all } x.$$

  If $V_{\mu_k} \neq V_{\mu_{k-1}}$, continue; otherwise, stop and output policy μ_k.
- **Policy Improvement:** Based on V_{μ_k}, compute a new policy μ_{k+1} as

$$\mu_{k+1}(x) \in \arg\max_a \left(\mathbb{E}[r(x, a)] + \alpha \sum_{x'} P_{x,x'}(a) V_{\mu_k}(x') \right) \quad \text{for all } x.$$

The idea behind the policy iteration algorithm is to obtain an improved policy μ_{k+1} based on the current policy μ_k. Note that in the policy iteration algorithm above, the improved policy is obtained by solving the optimization problem $\arg\max_a \left(\mathbb{E}[r(x, a)] + \alpha \sum_{x'} P_{x,x'}(a) V_{\mu_k}(x') \right)$. As we pointed out earlier, for problems with large or infinite action spaces, solving this optimization problem can be very difficult. To avoid solving an optimization problem, we consider a general class of policy iteration algorithms, called Actor-Critic algorithms.

Generalized Policy Iteration

- **Critic (Policy Evaluation):** Critic plays the role of policy evaluation, which estimates the value of the current policy (Q-function in general in most cases).
- **Actor (policy Improvement:** Actor plays the role of policy improvement, which obtains a better policy based on the value of the current policy.

When a policy is given, the policy outputs an action for a given state. Therefore, no optimization

is needed for selecting an action. During the policy evaluation, since we are evaluating a fixed policy, the Bellman equation does not have the max operator which also makes it easier to learn the value function or Q-value. The algorithms we have introduced such as Q-learning and its variations can be easily adopted for learning the Q-values of a given policy (by removing the max operator in TD).

The key challenge is to obtain an improved policy based on the old policy. To optimize over the policy space that is large or even continuous, it is often necessary to define parameterized policies where the parameters could be the weight vector of the features or the weights of a neural network. Finding the optimal policy in the parameterized policy space is equivalent to finding the corresponding weights of the best policy in the policy space. For example, consider a stochastic policy based on parameters w, defined by $\pi_w(u|x)$, which is the probability of selecting action u in state x. Therefore, the policy is parameterized by w. For such parameterized policies, the reinforcement learning problem is to find weights w^*, such that

$$w^* \in \arg\max_w \mathbb{E}\left[\sum_u Q_w(x,u)\pi_w(u|x)\right],$$

where $Q_w(x,u)$ is the Q-value of state action pair (x,u) under the policy defined by w. We next present two examples of parameterized policies:

- Gibbs policy (Softmax Policy or Boltzman exploration): Given weights w, action a is selected in state x with probability $\pi_w(a|x)$:

  $$\pi_w(a|x) = \frac{\exp\left(w^\top \phi(x,a)\right)}{\sum_{u\in A}\exp\left(w^\top\phi(x,u)\right)}$$

  where $\phi(x,u)$ is the feature vector. A Gibbs policy is defined for a finite action space.

- Gaussian policy: Given weights w, when the system is in state x, action a is sampled according to the following probability density function:

  $$\pi_w(a|x) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma_w(x))}}\exp\left\{-(a-\mu_w(x))^\top \Sigma_w^{-1}(x)(a-\mu_w(x))\right\}$$

  where it is common to choose $\mu_w(x) = \phi^\top(x)w$ and $\Sigma_w(x) = \sigma^2 I$. A Gaussian policy can be used for a continuous action space.

6.2 Policy Gradient Algorithms and Policy Gradient Theorem

The idea behind the policy gradient algorithms is to move the weight vector towards the direction such that the value of the policy improves. In particular, update the weight as follows:

$$w \leftarrow w + \beta \nabla_w V_w(x_0),$$

where β is the learning rate. Clearly, the key to policy gradient algorithms is to compute the gradient, which is a challenging problem because it is impossible to have a closed-form of function $V_w(x_0)$. In particular, note that

$$V_w(x_0) = \sum_u Q_w(x_0,u)\pi(u|x_0),$$

so

$$\nabla_w V_w(x_0) = \sum_u \left(\nabla_w Q_w(x_0, u)\right) \pi(u|x_0) + Q_w(x_0, u) \nabla_w \pi(u|x_0).$$

For parameterized policies, such as Softmax or Gaussian policy, we can calculate $\nabla_w \pi(u|x_0)$. So the key challenge is to calculate $\nabla_w Q_w(x_0, u)$.

The following policy gradient theorem states that the gradient can be computed based on $Q_w(x, u)$ instead of $\nabla_w \pi(u|x_0)$. Since $Q_w(x, u)$ is evaluated using policy evaluation, the policy gradient theorem implies that it is possible to calculate the policy gradient in a data-driven, model-free fashion, which makes model-free policy gradient algorithms possible.

Theorem 6.2.1 (Policy Gradient Theorem) *Given a policy parameterized by weight w, the gradient can be expressed as:*

$$\nabla_w V_w(x_0) = \frac{1}{1-\alpha} \mathbb{E}_{x \sim \rho_w, u \sim \pi_w(u|x)} \left[Q_w(x, u) \nabla_w \log \pi_w(u|x) \right],$$

where

$$\rho_w(x) = (1-\alpha) \sum_{k=0}^{\infty} \alpha^k \Pr\left(x_k = x\right),$$

called discounted state distribution.

Proof Recall that the gradient with respect to policy w can be written as

$$\nabla V_w(x) = \sum_u \left(\nabla_w Q_w(x, u) \pi_w(u|x) + Q_w(x, u) \nabla_w \pi_w(u|x) \right). \tag{6.1}$$

The key challenge is to calculate $\nabla_w Q_w(x, u)$ because we do not have a closed-form expression of $Q_w(x, u)$ in general. We next focus on $\nabla_w Q_w(x, u)$ and obtain

$$\begin{aligned}
\sum_u \nabla_w Q_w(x, u) \pi_w(u|x) =& \sum_u \nabla_w \left(\mathbb{E}[r(x, u)] + \alpha \sum_{x'} P_{x,x'}(u) V_w(x') \right) \pi_w(u|x) \\
=& \alpha \sum_u \left(\sum_{x'} P_{x,x'}(u) \nabla_w V_w(x') \right) \pi_w(u|x) \\
=& \alpha \mathbb{E}_{x' \sim \sum_u P_{x,x'}(u) \pi_w(u|x)} \left[\nabla_w V_w(x') \right],
\end{aligned} \tag{6.2}$$

where the second equality holds because the mean reward given state and action does not depend on the policy, and the last equality holds because $\sum_u P_{x,x'}(u) \pi_w(u|x)$ is the probability the environment is in state x' when the agent follows policy π in state x.

Now we consider the second term on the right hand side of equation (6.1) and have

$$\begin{aligned}
\sum_u Q_w(x, u) \nabla_w \pi_w(u|x) =& \sum_u Q_w(x, u) \left(\nabla_w \log \pi_w(u|x) \right) \pi_w(u|x) \\
=& \mathbb{E}_{u \sim \pi_w(u|x)} \left[Q_w(x, u) \nabla_w \log \pi_w(u|x) \right],
\end{aligned} \tag{6.3}$$

where the first equation holds because

$$\nabla_w \log \pi_w(u|x) = \frac{1}{\pi_w(u|x)} \nabla_w \pi_w(u|x)$$

and the second equality holds according to the definition of the expectation.

Combining Equations (6.2) and (6.3), we obtain

$$\begin{aligned}
&\nabla V_w(x_0) \\
=&\sum_u \left(\nabla_w Q_w(x_0,u)\pi_w(u|x_0) + Q_w(x_0,u)\nabla_w \pi_w(u|x_0)\right) \\
=&\mathbb{E}\left[Q_w(x_0,u)\nabla_w \log \pi_w(u|x_0)\right] + \alpha \mathbb{E}\left[\nabla_w V_w(x_1)\right] \\
=&\mathbb{E}\left[Q_w(x_0,u)\nabla_w \log \pi_w(u|x_0)\right] + \alpha \mathbb{E}\left[Q_w(x_1,u_1)\nabla_w \log \pi_w(u_1|x_1)\right] + \alpha^2 \mathbb{E}\left[\nabla_w V_w(x_2)\right] \\
=&\mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k \nabla_w \log \pi_w(u_k|x_k) Q_w(x_k,u_k)\right]. \qquad (6.4)
\end{aligned}$$

In policy gradient algorithms, $\nabla_w \log \pi_w(u|x)$ is called the score function. Consider the two parameterized policy classes we introduced earlier, the corresponding score functions are

- Gibbs policy:
$$\nabla_w \log \pi_w(u|x) = \phi(x,u) - \mathbb{E}_{u\sim\pi_w(x)}[\phi(x,u)].$$
- Gaussian policy:
$$\nabla_w \log \pi_w(u|x) = \frac{(u-\mu(x))\phi(x)}{\sigma^2}$$
where $\mu(x) = \phi^\top(x)w$.

The significance of the equation above is that the right hand side of the equation does not include $\nabla_w Q_w$, in other words, we can calculate the gradient $\nabla V_w(x_0)$ without $\nabla_w Q_w$. Since the gradient is written as the expectation, we can use a data-driven approach and replace the expectation with the empirical mean from data samples generated under policy π_w. The equation above leads to the first policy-gradient algorithm called REINFORCE [27], which will be introduced after the proof.

Now to complete the proof of the theorem, we write the summation in a different way:

$$\begin{aligned}
\nabla V_w(x_0) &= \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k \nabla_w \log \pi_w(u_k|x_k) Q_w(x_k,u_k)\right] \\
&= \sum_{k=0}^{\infty}\sum_{x,u} \alpha^k \nabla_w \log \pi_w(u|x) Q_w(x,u) \Pr(x_k = x, u_k = u|x_0) \\
&= \sum_{x,u} \nabla_w \log \pi_w(u|x) Q_w(x,u) \sum_{k=0}^{\infty} \alpha^k \Pr(x_k = x, u_k = u|x_0) \\
&= \sum_{x,u} \nabla_w \log \pi_w(u|x) Q_w(x,u) \sum_{k=0}^{\infty} \alpha^k \Pr(x_k = x|x_0)\pi_w(u|x) \\
&= \frac{1}{1-\alpha}\sum_{u,x} \nabla_w \log \pi_w(u|x) Q^{\pi_w}(x,u) \left((1-\alpha)\sum_{k=0}^{\infty} \alpha^k \Pr\left(x_k = x|x_0\right)\right)\pi_w(u|x) \\
&= \frac{1}{1-\alpha}\mathbb{E}_{x\sim\rho_w, u\sim\pi_w(u|x)}\left[\nabla_w \log \pi_w(u|x) Q_w(x,u)\right].
\end{aligned}$$

□

Motivated by the policy gradient theorem, we introduce the first policy gradient algorithm, called REINFORCE. The REINFORCE algorithm is based on equation (6.4) and uses the Monte-Carlo method to estimate the Q-function. In other words, given an episode of T experiences $\{(x_k, u_k, r_k, x_{k+1})\}_{0\le k\le T-1}$ generated under a policy with parameter w, we approximate the Q-value of state action pair (x_k, u_k) by the discounted total reward received starting from step k, in other words,

$$Q_w(x_k, u_k) \approx \sum_{\tau=k}^{T-1} \alpha^{\tau-k} r(x_\tau, u_\tau).$$

Therefore, the value is estimated using a single episode.

REINFORCE ([27])

Generate an episode $x_0, u_0, x_1, u_1, \cdots, x_{T-1}, u_{T-1}, x_T$ by following policy π_w. After the episode is generated, it takes T steps to update w. For each step k of the episode, update w as follows:

$$w \leftarrow w + \beta\alpha^k \nabla_w \log \pi_w(u_k|x_k) \sum_{\tau=k}^{T-1} \alpha^{\tau-k} r(x_\tau, u_\tau).$$

Continue to the next episode with the updated w.

6.3 Variance Reduction (Control Variates Method)

We note that REINFORCE estimates the policy gradient of policy w_t with experiences generated from policy w_t. The experiences generated from w_k for $k < t$ are not reused. Such a policy-gradient algorithm is called on-policy policy gradient. One drawback of on-policy policy gradient algorithms is the high variance due to the limited number of samples we have for estimating the gradient of a given policy.

In this section, we introduce the variance reduction method, also called control variates method, to reduce the variance and improves the learning. To introduce the method, we first consider a simpler problem of estimating the mean of $f(X)$ where X is a random variable. Given N independently and identically distributed samples, an unbiased estimate of the mean is the sample average, i.e.

$$\mathbb{E}[f(X)] \approx F = \frac{1}{N}\sum_{i=1}^{N} f(x_i).$$

Note that the variance of F (finite-sample average), which is $\frac{\mathrm{Var}(f(X))}{N}$ may be high, depending on distribution of X and function f.

Now image that we can replace F with $\tilde{F}$ such that

$$\mathbb{E}[F] = \mathbb{E}\left[\tilde{F}\right] \quad \text{and} \quad \mathrm{Var}\left(\tilde{F}\right) \le \mathrm{Var}(F),$$

which means we can find an unbiased estimator with a smaller variance.

The question, therefore, how to find such a $\tilde{F}$? Consider a function $\phi(x)$ such that $\mathbb{E}[\phi(X)] = 0$. Define

$$\tilde{f}(x) = f(x) - \phi(x) \quad \text{and} \quad \tilde{F} = \frac{1}{N}\sum_{i=1}^{N} \tilde{f}(x_i).$$

We note that

$$\mathbb{E}[\tilde{F}] = \mathbb{E}[F]$$

and

$$\text{Var}(\tilde{F}) = \frac{1}{N}\text{Var}(f(X) - \phi(X)) = \frac{1}{N}\left(\text{Var}(f(X)) - 2\text{Cov}(f(X), \phi(X)) + \text{Var}(\phi(X))\right).$$

Therefore, $\tilde{F}$ is also an unbiased estimate of $\mathbb{E}[f(X)]$ and the variance of $\tilde{F}$ is smaller than F when the following condition holds

$$-2\text{Cov}(f(X), \phi(X)) + \text{Var}(\phi(X)) < 0.$$

In other words, the variance can be reduced if $\phi(x)$ is positively correlated with $f(x)$.

Consider a simple example such that

$$\Pr(X = 2) = \Pr(X = -1) = 0.5 \quad \text{and} \quad f(x) = x.$$

In this case

$$\text{Var}(f(X)) = E[X^2] - E[X]^2 = 0.5(4 + 1) - 0.25 = 2.25.$$

Define $\phi(x) = 1$ if $x = 2$ and $\phi(x) = -1$ if $x = -1$. Then $\mathbb{E}[\phi(X)] = 0$ and

$$\text{Var}\left(\tilde{f}(X)\right) = 0.5 - 0.25 = 0.25,$$

which is significantly smaller than the original variance.

Now recall the policy gradient theorem where we have

$$\nabla_w V_w(x_0) = \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k \nabla_w \log \pi_w(u_k|x_k) Q_w(x_k, u_k)\right].$$

The question is what function can be added or subtracted from the function inside the expectation to reduce its variance. The following variance reduction theorem provides an answer to the question.

Theorem 6.3.1 (Variance Reduction Theorem) *Given any function $G_w(x_k)$, i.e. a function depending on the state but not the action, we have*

$$\begin{aligned}\nabla_w V_w(x_0) =& \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k \nabla_w \log \pi_w(u_k|x_k) Q_w(x_k, u_k)\right] \\ =& \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k \nabla_w \log \pi_w(u_k|x_k) \left(Q_w(x_k, u_k) - G_w(x_k)\right)\right].\end{aligned}$$

Proof To prove the theorem, we will show

$$\mathbb{E}\left[\nabla_w \log \pi_w(u_k|x_k) G_w(x_k)\right] = 0.$$

To prove this equality, we have

$$\begin{aligned}
&\mathbb{E}\left[\nabla_w \log \pi_w(u_k|x_k) G_w(x_k)\right] \\
=&\sum_{x,u} \nabla_w \log \pi_w(u|x) G_w(x) \pi_w(u|x) \Pr(x_k = x|x_0) \\
=&\sum_{x,u} \frac{\nabla_w \pi_w(u|x)}{\pi_w(u|x)} G_w(x) \pi_w(u|x) \Pr(x_k = x|x_0) \\
=&\sum_{x,u} \nabla_w \pi_w(u|x) G_w(x) \Pr(x_k = x|x_0) \\
=&\mathbb{E}_{x_k}\left[\sum_u \nabla_w \pi_w(u|x_k) G_w(x_k)\right] \\
=&\mathbb{E}_{x_k}\left[G_w(x_k) \nabla_w \left(\sum_u \pi_w(u|x_k)\right)\right] \\
=&\mathbb{E}_{x_k}\left[G_w(x_k) \nabla_w (1)\right] \\
=&0,
\end{aligned}$$

where $\sum_u \pi_w(u|x_k) = 1$ because $\pi(u|x_k)$ for given x_k is a probability distribution over the action space.

□

Now to reduce the variance in policy gradient, we need to find a function $G_w(x)$ that is positively correlated with Q-function $Q_w(x, u)$. A nature choice is to use value function $V_w(x)$ because Q-values of a state are likely to be large when the value of the state is large. $Q_w(x, u) - V_w(x)$ is called the advantage function. By using the advantage function, we have the following advantage Actor-Critic algorithm.

Advantage Actor-Critic Algorithm

- At each iteration, execute the policy with parameters w for T steps to collect T experiences.
- **Critic (Policy Evaluation):** Use TD learning, double-Q, clipped double-Q, etc to learn $Q_w(x, u)$ and $V_w(x)$.
- **Actor (policy Improvement:** Estimate $\nabla_w V(x_0)$

$$\nabla_w V(x_0) = \sum_{k=0}^{T-1} \alpha^k \nabla \log \pi_w(u_k|x_k)(Q_w(x_k, u_k) - V_w(x_k))$$

and update the policy

$$w \leftarrow w + \beta \nabla_w V(x_0).$$

The algorithm above, the advantage Actor-Critic algorithm needs to learn both the Q-function and value-function $Q_w(x,u)$ and $V_w(x)$. We note that

$$Q_w(x,u) = \mathbb{E}[r_w(x,u)] + \alpha \mathbb{E}[V_w(x')],$$

which motivated the following TD Actor-Critic Algorithm.

TD Actor-Critic Algorithm

- At each iteration, execute the policy with parameters w for T steps to collect T experiences.
- **Critic (Policy Evaluation):** Use TD learning, double-Q, clipped double-Q, etc to learn $Q_w(x,u)$ and $V_w(x)$.
- **Actor (policy Improvement:** Estimate $\nabla_w V(x_0)$

$$\nabla_w V(x_0) = \sum_{k=0}^{T-1} \alpha^k \nabla \log \pi_w(u_k|x_k)(r(x_k,u_k) + \alpha V_w(x_{k+1}) - V_w(x_k))$$

and update the policy

$$w \leftarrow w + \beta \nabla_w V(x_0).$$

Chapter 7

Variations of Policy Gradient Algorithms

In this chapter, we introduce several variations of policy gradient algorithms that improve the performance of the original policy gradient algorithm.

7.1 Natural Policy Gradient (NPG)

Recall the policy gradient algorithm

$$w \leftarrow w + \beta \nabla_w V_w(x_0).$$

In optimization, this gradient ascent algorithm can be interpreted as follows: Starting from the current point w, to move towards the minimum of $V_w(x_0)$, we solve the following optimization problem

$$\max_{\tilde{w}} V_w(x_0) + \nabla_w V_w(x_0)(\tilde{w} - w) - \frac{1}{2\beta}(\tilde{w} - w)^\top \mathbb{I}(\tilde{w} - w),$$

where $V_w(x_0) + \nabla_w V_w(x_0)(\tilde{w} - w)$ is the linear approximation of function $V_{\tilde{w}}(x_0)$ (as a function of $\tilde{w}$) and the term

$$(\tilde{w} - w)^\top \mathbb{I}(\tilde{w} - w) = \|\tilde{w} - w\|_2^2$$

is called the proximal term that prevents the algorithm moving too far away from the current point w, where $\|\cdot\|_2$ is the Euclidean norm. It can be easily verified that the optimal solution to the problem is

$$\tilde{w} = w + \beta \nabla_w V_w(x_0),$$

which is the policy gradient algorithm.

We note that the proximal term (or the penalty term) is in terms of the Euclidean distance of weight vector w. While the weights w define a policy, the change of w does not directly correlate with the change of the performance of the policy. In particular, minor (significant) changes in w may result in significant (minor) changes in policy π. In other words, $\pi_{\tilde{w}}$ and π_w can be significantly different even $\tilde{w}$ and w are close.

To overcome this problem, we can use a proximal term that directly defines on the distance of two policies, e.g., $\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x))$, where

$$\mathrm{KL}(p\|q) = \int_{-\infty}^{\infty} p(u) \log\left(\frac{p(u)}{q(u)}\right) du$$

is the Kullback-Leibler divergence of distributions p and q. With the KL divergence as the proximal term, the optimization problem becomes

$$\max_{\tilde{w}} V_w(x_0) + \nabla_w V_w(x_0)(\tilde{w} - w) - \frac{1}{\beta}\mathbb{E}_{x\sim\rho_w}\left[\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x))\right].$$

To solve the optimization problem above, we consider the second-order Taylor approximation of the KL divergence:

$$\begin{aligned}\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x)) =& \mathrm{KL}(\pi_w(\cdot|x)\|\pi_w(\cdot|x)) + \left(\nabla_{\tilde{w}}\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x))|_{\tilde{w}=w}\right)^\top (\tilde{w} - w) \\ &+ \frac{1}{2}(\tilde{w} - w)^\top \left(\nabla^2_{\tilde{w}}\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x))|_{\tilde{w}=w}\right)(\tilde{w} - w) + o(\|\tilde{w} - w\|^3).\end{aligned}$$

It is easy to verify that

$$\mathrm{KL}(\pi_w(\cdot|x)\|\pi_w(\cdot|x)) = 0$$

and

$$\nabla_{\tilde{w}}\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x))|_{\tilde{w}=w} = 0.$$

Furthermore,

$$\nabla^2_{\tilde{w}}\mathrm{KL}(\pi_{\tilde{w}}(\cdot|x)\|\pi_w(\cdot|x))|_{\tilde{w}=w} = \mathbb{E}_{\pi_w}\left[\left(\nabla_w \log \pi_w(u|x)\right)\nabla \log \pi_w(u|x)^\top\right] := \mathrm{FI}_w(x)$$

is the Fisher information matrix. Now define the average Fisher information matrix to be

$$\mathrm{FI}_w = \mathbb{E}_{\rho_w}\left[\mathrm{FI}_w(x)\right].$$

Then the optimization problem associated with the proximal gradient ascent becomes

$$\max_{\tilde{w}} V_w(x_0) + \nabla_w V_w(x_0)(\tilde{w} - w) - \frac{1}{2\beta}(\tilde{w} - w)^\top \mathrm{FI}_w(\tilde{w} - w).$$

In other words, instead of using an identity matrix in the gradient ascent algorithm, we use the average Fisher information matrix in NPG and have the following algorithm [12].

$$w \leftarrow w + \beta \mathrm{FI}_w^\dagger \nabla_w V_w(x_0),$$

where † is the Moore-Penrose pseudo-inverse.

NPG has a faster convergence rate than the vanilla policy gradient algorithm. In particular, its convergence rate that is independent of the sizes of state space and action space [2].

7.2 Trust Region Policy Optimization (TRPO)

Trust region policy gradient (TRPO) [18] is very similar to NPG but was derived using a different local approximation. Before describing TRPO, we first present the following lemma, called the performance difference lemma. The lemma itself has been widely used in other contexts of reinforcement learning and is a useful result by itself.

Lemma 7.2.1 (Performance Difference Lemma [11]) *Consider a discounted infinite-horizon RL problem with a finite state space and a finite action space. Consider two policies, parameterized by w and $\tilde{w}$, respectively. We have*

$$V_{\tilde{w}}(x_0) = V_w(x_0) + \frac{1}{1-\alpha}\mathbb{E}_{\rho_{\tilde{w}}}\left[\sum_u \pi_{\tilde{w}}(u|x)\left(Q_w(x,u) - V_w(x)\right)\right],$$

where $\rho_{\tilde{w}}$ is the discounted state visitation distribution such that

$$\rho_{\tilde{w}}(x) := \frac{1}{1-\alpha}\sum_{k=0}^{\infty}\alpha^k \mathbb{P}_{\tilde{w}}(x_k = x|x_0).$$

Proof According to definition of $V_{\tilde{w}}(x_0)$, we have

$$\begin{aligned} V_{\tilde{w}}(x_0) =& \mathbb{E}_{x_k\sim\mathbb{P}_{\tilde{w}}(x_k|x_0),u_k\sim\pi_{\tilde{w}}(x_k)}\left[\sum_{k=0}^{\infty}\alpha^k r(x_k,u_k)\right] \\ =& \mathbb{E}_{x_k\sim\mathbb{P}_{\tilde{w}}(x_k|x_0),u_k\sim\pi_{\tilde{w}}(x_k)}\left[\sum_{k=0}^{\infty}\alpha^k\left(r(x_k,u_k) + V_w(x_k) - V_w(x_k)\right)\right]. \end{aligned}$$

Since

$$\sum_{k=0}^{\infty}\alpha^k V_w(x_k) = \sum_{k=0}^{\infty}\alpha^{k+1}V_w(x_{k+1}) + V_w(x_0),$$

the equation above is equivalent to

$$\begin{aligned} V_{\tilde{w}}(x_0) =& \mathbb{E}_{x_k\sim\mathbb{P}_{\tilde{w}}(x_k|x_0),u_k\sim\pi_{\tilde{w}}(x_k),x_{k+1}\sim\mathbb{P}(x_{k+1}|x_k,u_k)}\left[\sum_{k=0}^{\infty}\alpha^k\left(r(x_k,u_k) + \alpha V_w(x_{k+1}) - V_w(x_k)\right)\right] + V_w(x_0) \\ =& \mathbb{E}_{x_k\sim\mathbb{P}_{\tilde{w}}(x_k|x_0),u_k\sim\pi_{\tilde{w}}(x_k)}\left[\sum_{k=0}^{\infty}\alpha^k\left(Q_w(x_k,u_k) - V_w(x_k)\right)\right] + V_w(x_0). \end{aligned}$$

We now rewrite the expectation in terms of the discounted state distribution as follows:

$$
\begin{aligned}
&\mathbb{E}_{x_k\sim\mathbb{P}_{\tilde{w}}(x_k|x_0),u_k\sim\pi_{\tilde{w}}(x_k)}\left[\sum_{k=0}^{\infty}\alpha^k\left(Q_w(x_k,u_k)-V_w(x_k)\right)\right]\\
=&\sum_{k=0}^{\infty}\sum_{x,u}\alpha^k\left(Q_w(x,u)-V_w(x)\right)\Pr(x_k=x,u_k=u|x_0)\\
=&\sum_{x,u}\left(Q_w(x,u)-V_w(x)\right)\sum_{k=0}^{\infty}\alpha^k\Pr(x_k=x,u_k=u|x_0)\\
=&\sum_{x,u}\left(Q_w(x,u)-V_w(x)\right)\sum_{k=0}^{\infty}\alpha^k\Pr(x_k=x|x_0)\pi_{\tilde{w}}(u|x)\\
=&\frac{1}{1-\alpha}\sum_{u,x}\left(Q_w(x,u)-V_w(x)\right)\left((1-\alpha)\sum_{k=0}^{\infty}\alpha^k\Pr\left(x_k=x|x_0\right)\right)\pi_{\tilde{w}}(u|x)\\
=&\frac{1}{1-\alpha}\mathbb{E}_{x\sim\rho_{\tilde{w}}}\left[\sum_{u}\pi_{\tilde{w}}(u|x)\left(Q_w(x,u)-V_w(x)\right)\right],
\end{aligned}
$$

which completes the proof.

□

The performance difference lemma establishes a fundamental connection between the value function under policy w and that under policy $\tilde{w}$. We also observe that if the current policy is w, then we can estimate $V_w(x)$ and $Q_w(x,u)$ from the experiences generated by policy w. Therefore, the key difficulty of estimating the value function of a new policy $\tilde{w}$, $V_{\tilde{w}}(x)$, is to find out distribution $\rho_{\tilde{w}}$. TRPO uses a local approximation of the value function and replaces $\rho_{\tilde{w}}$ with ρ_w, i.e. the discounted distribution under the current policy w:

$$
V_{\tilde{w}}(x_0)\approx L_{\tilde{w}}(x_0):=V_w(x_0)+\mathbb{E}_{\rho_w}\left[\sum_{u}\pi_{\tilde{w}}(u|x)A_w(x,u)\right],
$$

where

$$
A_w(x,u)=Q_w(x,u)-V_w(x)
$$

is the advantage function. $L_{\tilde{w}}(x_0)$ matches $V_{\tilde{w}}(x_0)$ to the first order [11] so this approximation can be justified if two policies w and $\tilde{w}$ are close to each other.

TRPO is based on this local approximation and aims at solving the following optimization problem:

$$
\begin{aligned}
&\max_{\tilde{w}}\mathbb{E}_{\rho_w}\left[\sum_{u}\pi_{\tilde{w}}(u|x)A_w(x,u)\right]\\
\text{subject to:}\quad&\mathbb{E}_{\rho_w}\left[\mathrm{KL}(\pi_{\tilde{w}}\|\pi_w)\right]\le\epsilon.
\end{aligned}
$$

Note that the constraint is imposed so that the solution to the optimization problem is close to the current policy w, which is important for the local approximation to hold. In fact, TRPO is a constrained version of NPG, which updates the parameter with

$$
w\leftarrow w+\tilde{\beta}\mathrm{FI}_w^{\dagger}\nabla_w V_w(x_0)
$$

with an adaptive learning rate such that in each step, a backtracking line search is used to make sure that the constraint $\mathbb{E}_{\rho_w}[\mathrm{KL}(\pi_{\tilde{w}}\|\pi_w)] \leq \epsilon$ is satisfied. Numerical methods for solving this optimization problem can be found in [18].

TRPO enjoys the following monotonic improvement property, which states that the policy continues to improve under TRPO. However, it does not state that the policy will converge to the optimal policy. The proof of this theorem can be found in Appendix C

Theorem 7.2.2 (Monotonic Improvement Theorem) *With carefully chosen learning rates, TRPO guarantees that under general function approximations:*

$$V_{w_{k+1}}(x_0) \geq V_{w_k}(x_0).$$

7.3 Proximal Policy Optimization (PPO)

Both NPG and TRPO are second-order methods, requiring the inverse of the Fisher information matrix. PPO [19] is similar to TRPO but is a first-order method. To introduce PPO, we first revisit the optimization formulation of TRPO:

$$\begin{aligned} &\max_{\tilde{w}} \mathbb{E}_{\rho_w}\left[\sum_u \pi_{\tilde{w}}(u|x)A_w(x,u)\right] = \max_{\tilde{w}} \mathbb{E}_{\rho_w, u\sim\pi_w}\left[\frac{\pi_{\tilde{w}}(u|x)}{\pi_w(u|x)}A_w(x,u)\right] \\ &\text{subject to:} \quad \mathbb{E}_{\rho_w}[\mathrm{KL}(\pi_{\tilde{w}}\|\pi_w)] \leq \epsilon. \end{aligned}$$

There are two versions of PPO: PPO-penalty and PPO-clip. Under PPO-penalty, we solve an unconstrained optimization problem by adding the KL divergence as a penalty.

PPO-Penalty

At the kth iteration,

- Collect multiple epochs and use minibatch SGD to optimize

$$w^* \in \arg\max_{\tilde{w}} \mathbb{E}_{\rho_w, u\sim\pi_{w_k}}\left[\frac{\pi_{\tilde{w}}(u|x)}{\pi_{w_k}(u|x)}A_{w_k}(x,u) - \beta\mathrm{KL}(\pi_{\tilde{w}}\|\pi_{w_k})\right].$$

- After obtaining w^*, compute $d = \mathbb{E}_{\rho_w}[\mathrm{KL}(\pi_{w^*}\|\pi_{w_k})]$. If $d \in (\epsilon/1.5, 1.5\epsilon)$, output $w_{k+1} = w^*$ and move to the $k+1$ iteration. Otherwise, adjust β such that if $d \leq \frac{\epsilon}{1.5}$, $\beta \leftarrow \beta/2$, and if $d \geq 1.5\epsilon$, $\beta \leftarrow 2\beta$; and resolve the optimization problem in step 1.

PPO-clip avoids the KL divergence term and considers the following modified objective.

PPO-Penalty

At the kth iteration, solves the following optimization problem:

$$\max_{\tilde{w}} \mathbb{E}_{\rho_{w_k}, u\sim\pi_{w_k}}\left[\min\left\{\frac{\pi_{\tilde{w}}(u|x)}{\pi_{w_k}(u|x)}A_{w_k}(x,u), \left(\frac{\pi_{\tilde{w}}(u|x)}{\pi_{w_k}(u|x)}\right)_{1-\epsilon}^{1+\epsilon}A_{w_k}(x,u)\right\}\right]$$

where $\{x\}_a^b$ is the clipping function such that $\{x\}_a^b$ equals to x if $x \in [a, b]$, a if $x < a$ and b if $x > b$.

In the modified objective, clip is to make sure the new policy does not deviate too much from the old policy. The min is to have a conservative estimate of the objective. Ignore the clipping if it improves the objective and include it when it makes the objective worse, which discourages the change.

7.4 Deterministic Policy Gradient (DPG)

So far, we consider only stochastic policies where $\pi_w(u|x)$ is the probability of taking action u in state x. In this section, we consider deterministic policies and derive a similar policy gradient theorem for deterministic policies. A deterministic policy $\mu_\theta(x)$, parameterized actor θ, outputs an action given state x, instead of a probability distribution over the action space.

We again consider the Actor-Critic architecture for deterministic policy gradient.

DPG

- **Critic:** A parameterized critic w to estimate the Q-function: $Q_w(x, u)$.
- **Actor:** A parameterized actor θ that outputs an action $\mu_\theta(x)$ given state x.

The training of Critic remains the same under stochastic policies and deterministic policies. Possible methods include DQN, DDQN, and TD3. Actor will be trained using policy gradient methods. The following deterministic policy gradient theorem is very similar to the one for stochastic policies, and the proof is also similar. The proof is included for the completeness of the book.

Theorem 7.4.1 (Deterministic Policy Gradient Theorem [21])

$$\nabla_\theta Q_w(x_0, \mu_\theta(x_0)) = \frac{1}{1-\alpha} \mathbb{E}_{x\sim\rho_\theta} \left[\nabla_u Q_w(x, u)|_{u=\mu_\theta(x)} \nabla_\theta \mu_\theta(x) \right],$$

where

$$\rho_\theta(x) = (1-\alpha) \sum_{k=0}^{\infty} \alpha^k \Pr(x_k = x).$$

Proof We consider the gradient of the Q-function for the initial state x_0 and the corresponding

action $\mu_\theta(x_0)$, where θ is the parameters that define the policy.

$$
\begin{aligned}
&\nabla_\theta Q_w(x_0, \mu_\theta(x_0)) \\
=&\nabla_\theta \left(r(x_0, \mu_\theta(x_0)) + \alpha \int_{\mathcal{S}} Q_w(x_1, \mu_\theta(x_1)) f(x_1|x_0; \mu_\theta(x_0)) dx_1 \right) \\
=&\nabla_u r(x_0, u)|_{u=\mu_\theta(x_0)} \nabla_\theta \mu_\theta(x_0) \\
&+ \alpha \left(\int_{\mathcal{S}} Q_w(x_1, \mu_\theta(x_1)) \nabla_u f(x_1|x_0; u)|_{u=\mu_\theta(x_0)} \nabla_\theta \mu_\theta(x_0) dx_1 \right) + \\
&+ \alpha \int_{\mathcal{S}} \nabla_\theta Q_w(x_1, \mu_\theta(x_1)) f(x_1|x_0; \mu_\theta(x_0)) dx_1 \\
=&\nabla_\theta \mu_\theta(x_0) \, \nabla_u \left(r(x_0, u) + \alpha \left(\int_{\mathcal{S}} Q_w(x_1, \mu_\theta(x_1)) f(x_1|x_0; u) dx_1 \right) \right) \Big|_{u=\mu_\theta(x_0)} \\
&+ \alpha \int_{\mathcal{S}} \nabla_\theta Q_w(x_1, \mu_\theta(x_1)) f(x_1|x_0; \mu_\theta(x_0)) dx_1
\end{aligned}
$$

Observing that

$$
Q_w(x_0, \mu_\theta(x_0)) = r(x_0, u) + \alpha \left(\int_{\mathcal{S}} Q_w(x_1, \mu_\theta(x_1)) f(x_1|x_0; u) dx_1 \right),
$$

we conclude that

$$
\nabla_\theta Q_w(x_0, \mu_\theta(x_0)) = \nabla_\theta \mu_\theta(x_0) \, \nabla_u Q_w(x_0, u)|_{u=\mu_\theta(x_0)} + \alpha \mathbb{E}_{x_1} \left[\nabla_\theta Q_w(x_1, \mu_\theta(x_1)) \right].
$$

Repeatedly using the equation above, we obtain

$$
\nabla_\theta Q_w(x_0, \mu_\theta(x_0)) = \frac{1}{1-\alpha} \mathbb{E}_{x \sim \rho_\theta} \left[\nabla_u Q_w(x, u)|_{u=\mu_\theta(x)} \nabla_\theta \mu_\theta(x) \right].
$$

□

Based on this deterministic policy gradient theorem, we have the following deterministic policy gradient (DPG).

DPG Actor

- Sample a mini-batch of experiences, say (x_b, u_b, r_b, x_{b+1}) for $b = 1, \cdots, B$.
- Input $\{x_b\}$ to the Actor network to obtain $\{\mu_\theta(x_b)\}$.
- Input $\{x_b, \mu_\theta(x_b)\}$ to the Critic network to obtain $\{Q_w(x_b, \mu_\theta(x_b))\}$.
- We then use $-\sum_b Q_w(x_b, \mu_\theta(x_b))$ as the loss function to update the Actor network. Note that the loss function is regarded as a function of θ only not a function of w and update w only.

Note that since the mini-batch is from experiences of old policies, the implementation above has a bias, which cannot be corrected using the importance sampling because it is a deterministic policy such that an action is taken either with probability one or probability zero.

Chapter 8

Exploration versus Exploitation: Multi-Armed Bandits

In this chapter, we introduce the exploration and exploitation tradeoff in RL by studying the multi-armed bandit problem. Exploration means trying actions to find the best one. Exploitation means taking the action that is "believed" to be the best. There is a tradeoff between exploration and exploitation. We may take suboptimal actions when we explore. However, without exploration, exploitation may fail to find the best action. A fundamental problem in reinforcement learning is how much exploration is necessary for an RL algorithm to identifying the optimal policy.

We consider the multi-armed bandit problem for understanding exploration and exploitation in RL. The stochastic bandits with I arms are as shown in Figure 8.1.

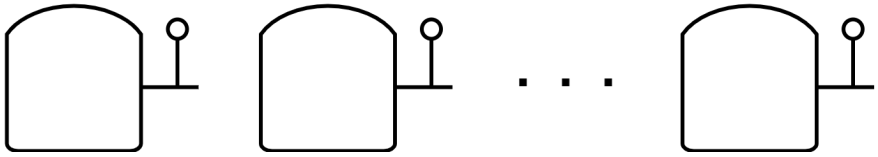


Figure 8.1: Multi-Armed Bandits

At each time step, we need to pick an arm from these I arms and play (pull) it. Let $X_i(k)$ denote the reward when arm i is played at time step k. Assume that $\{X_i(k)\}_k$ are i.i.d Bernoulli random variables with mean μ_i for each given i. The goal is to maximize the total expected reward over K time steps, i.e.,

$$\max \sum_{k=1}^{K} \mathbb{E}[X_{i_k}(k)],$$

where i_k denotes the arm played at time step k.

8.1 ϵ-Greedy Algorithm

A straightforward strategy for solving the multi-armed bandit problem is to use forced exploration such as the ϵ-greedy algorithm introduced in Chapter 3.

ϵ-Greedy

- With probability $1-\epsilon$, select arm i_k such that

$$i_k \in \arg\max_i \mu_i(k),$$

where

$$\mu_i(k) = \frac{\text{total reward received from playing arm } i}{\text{number of times arm } i \text{ has been played}}.$$

This means picking the arm with the highest empirical mean.

- With probability ϵ, randomly pick an arm.

8.2 Upper Confidence Bound (UCB) Algorithm

Another approach that is widely used in the literature is the upper confidence bound (UCB) [3] algorithm, which is shown as follows:

UCB

- Play each arm once at the beginning.
- At time $k > I$, choose arm i_k such that

$$i_k \in \arg\max_i \left(\mu_i(k) + \sqrt{\frac{4\log K}{N_i(k)}} \right), \tag{8.1}$$

where $N_i(k)$ is the number of times arm i has been played by time step k (not including k), i.e.,

$$N_i(k) = \sum_{\tau=1}^{k-1} \mathbb{I}_{i_\tau = i}$$

and $\mu_i(k)$ is the empirical mean of arm i's reward by time step k (not including k), i.e.,

$$\mu_i(k) = \frac{\sum_{\tau=1}^{k-1} \mathbb{I}_{i_\tau = i} X_{i_\tau}(\tau)}{N_i(k)}.$$

The term $\sqrt{\frac{4\log K}{N_i(k)}}$ in (8.1) is called the UCB bonus and represents the confidence interval about estimating μ_i with current observations. The term $\mu_i(k) + \sqrt{\frac{4\log K}{N_i(k)}}$ in (8.1) is the upper confidence bound of the true mean reward of arm i such that with a high probability, the true mean is no larger than $\mu_i(k) + \sqrt{\frac{4\log K}{N_i(k)}}$. Note that large $N_i(k)$ means we have a lot of samples for arm i, so we can be more confident in our estimate of the mean reward for the arm. Conversely, when $N_i(k)$ is small, we have less confidence in our estimate of the mean reward for the arm.

The intuition behind the UCB algorithm is that we should explore arms with more uncer-

tain, also called optimism in face of uncertainty. The UCB algorithm balances exploration and exploitation. For large K, as $k \to K \to \infty$, the probability of selecting the best arm goes to one because

$$\mu_i(k) \to \mu_i \quad \text{and} \quad \sqrt{\frac{4\log K}{N_i(k)}} \to 0, \text{ as } N_i(k) \to \infty.$$

Note that if $N_i(k)$ does not go to infinity, then it means $\mu_i(k) + \sqrt{\frac{4\log K}{N_i(k)}}$ is not large enough, so arm i is unlikely to be the best one.

8.3 Regret Analysis

To evaluate the performance of different algorithms, we use regret, which defines the gap between the total reward when playing the best arm over the entire learning horizon and that of the algorithm under consideration. Let us first define regret.

Definition 8.3.1 (Regret) *Assume arm 1 is the best arm, then the regret is defined by*

$$R_K = \mu_1 K - \mathbb{E}\left[\sum_{k=1}^{K} X_{i_k}(k)\right].$$

Since ϵ-greedy policy selects a random arm with probability ϵ, it is easy to see that the regret is

$$\epsilon\mu_1 K - \epsilon\frac{\sum_{i=1}^{I}\mu_i}{I}K = O(K).$$

In other words, the regret increases linearly in K.

Next, we will show that the regret bound for UCB is

$$R_K = O\left(\sqrt{K\log K}\right).$$

The proof utilizes the Azuma-Hoeffding inequality for martingales [9]. The definition of martingale and the Azuma-Hoeffding inequality are given as follows.

Definition 8.3.2 (Martingale) *A martingale is a discrete-time stochastic process $Y_1, Y_2, Y_3, \cdots$, that satisfies for any k*

$$\mathbb{E}[|Y_k|] < \infty$$
$$\mathbb{E}[Y_{k+1}|Y_k, Y_{k-1}, \cdots, Y_1] = Y_k.$$

Lemma 8.3.1 (Azuma-Hoeffding Inequality) *Consider a martingale $\{Y_k\}$ such that $Y_0 = 0$ and $|Y_k - Y_{k-1}| \le c_k$. For any $m > 0$,*

$$\Pr\left(|Y_k| \ge m\right) \le 2\exp\left(-\frac{m^2}{2\sum_{\tau=1}^{k} c_\tau^2}\right).$$

Now we are ready to present the proof of the regret bound under UCB.

Proof of the Regret Bound Under UCB Firstly, by the definition of regret (Definition 8.3.1), we have

$$\begin{aligned} R_K &= \mu_1 K - \mathbb{E}\left[\sum_{k=1}^{K} X_{i_k}(k)\right] \\ &= \mu_1 K - \sum_{k=1}^{K} \mathbb{E}\left[X_{i_k}(k)\right] \\ &= \mu_1 K - \sum_{k=1}^{K} \mathbb{E}\left[\mu_{i_k}\right], \end{aligned}$$

where the last inequality is obtained by taking expectation on the randomness of the reward (conditioned on i_k). Note that μ_{i_k} is a random variable since i_k is a random variable. Then, by subtracting and adding the term $\mu_{i_k}(k) + \sqrt{\frac{4\log K}{N_{i_k}(k)}}$, we have

$$R_K = \sum_{k=1}^{K} \mathbb{E}\left[\mu_1 - \mu_{i_k}(k) - \sqrt{\frac{4\log K}{N_{i_k}(k)}}\right] + \sum_{k=1}^{K} \mathbb{E}\left[\mu_{i_k}(k) + \sqrt{\frac{4\log K}{N_{i_k}(k)}} - \mu_{i_k}\right]. \tag{8.2}$$

Now we want to apply the Azuma-Hoeffding inequality. Define $\mu_{i,n} = \frac{\sum_{\tau=1}^{n} X_{i,\tau}}{n}$ (the estimate after getting n samples). Then

$$Y_n = n\mu_{i,n} - n\mu_i = \sum_{\tau=1}^{n} X_{i,\tau} - n\mu_i$$

is a martingale because

$$\mathbb{E}[Y_{n+1}|Y_1, \cdots, Y_n] = \mathbb{E}[Y_n + X_{i,n+1} - \mu_i | Y_n] = Y_n.$$

Note that the martingale difference is bounded by

$$|Y_{n+1} - Y_n| = |X_{i,n+1} - \mu_i| \leq 1 = c_{n+1}.$$

Define event $\mathcal{A}$ such that

$$\mathcal{A} = \left\{ |\mu_{i,n} - \mu_i| \leq \sqrt{\frac{4\log K}{n}} \quad \forall i, \forall n = 1, \cdots, K \right\}.$$

Applying the Azuma-Hoeffding inequality (Lemma 8.3.1) and the union bound, we obtain

$$\Pr(\mathcal{A}) \geq 1 - I \times K \times \frac{2}{K^2} = 1 - \frac{2I}{K}. \tag{8.3}$$

Define two functions

$$F_1(k) = \mu_1 - \mu_{i_k}(k) - \sqrt{\frac{4\log K}{N_{i_k}(k)}} \quad \text{and} \quad F_2(k) = \mu_{i_k}(k) + \sqrt{\frac{4\log K}{N_{i_k}(k)}} - \mu_{i_k}.$$

Then we have the following two facts:

Lemma 8.3.2 *At any step k,*

$$F_1(k) \le 1 \quad \textit{and} \quad F_2(k) \le 1 + 2\sqrt{\log K}.$$

When $\mathcal{A}$ occurs,

$$F_1(k) \le 0 \quad \textit{and} \quad |F_2(k)| \le 2\sqrt{\frac{4\log K}{N_{i_k}(k)}}.$$

Proof Since $\mu_{i_k}(k) \ge 0$ and $\mu_1 \le 1$, we have

$$F_1(k) \le \mu_1 \le 1.$$

Since $\mu_{i_k}(k) \le 1$ and $\mu_{i_k} \ge 0$, we have

$$F_2(k) \le 1 + \sqrt{\frac{4\log K}{N_{i_k}(k)}} \le 1 + 2\sqrt{\log K},$$

where the last inequality holds since $N_{i_k}(k) \ge 1$.

Since the algorithm picks the arm with the largest UCB, we have

$$F_1(k) = \mu_1 - \left(\mu_{i_k}(k) + \sqrt{\frac{4\log K}{N_{i_k}(k)}}\right) \le \mu_1 - \left(\mu_1(k) + \sqrt{\frac{4\log K}{N_1(k)}}\right).$$

When $\mathcal{A}$ occurs, we have

$$F_1(k) \le \mu_1 - \left(\mu_1(k) + \sqrt{\frac{4\log K}{N_1(k)}}\right) \le 0.$$

For $F_2(k)$, when $\mathcal{A}$ occurs, we have

$$F_2(k) = \mu_{i_k}(k) - \mu_{i_k} + \sqrt{\frac{4\log K}{N_{i_k}(k)}} \le 2\sqrt{\frac{4\log K}{N_{i_k}(k)}}.$$

□

Based on the above two facts and Eq. (8.2), we have

$$\begin{aligned}
R_K =& \sum_{k=1}^{K} \mathbb{E}\left[F_1(k) \middle| \mathcal{A}\right] \Pr(\mathcal{A}) + \sum_{k=1}^{K} \mathbb{E}\left[F_1(k) \middle| \mathcal{A}^c\right] \Pr(\mathcal{A}^c) \\
&+ \sum_{k=1}^{K} \mathbb{E}\left[F_2(k) \middle| \mathcal{A}\right] \Pr(\mathcal{A}) + \sum_{k=1}^{K} \mathbb{E}\left[F_2(k) \middle| \mathcal{A}^c\right] \Pr(\mathcal{A}^c) \\
\le& 0 + K \Pr(\mathcal{A}^c) + \sum_{k=1}^{K} \mathbb{E}\left[2\sqrt{\frac{4\log K}{N_{i_k}(k)}} \middle| \mathcal{A}\right] \Pr(\mathcal{A}) + K \Pr(\mathcal{A}^c)(1 + 2\sqrt{\log K}) \\
=& K \Pr(\mathcal{A}^c) + 4\sqrt{\log K}\mathbb{E}\left[\sum_{k=1}^{K} \sqrt{\frac{1}{N_{i_k}(k)}} \middle| \mathcal{A}\right] \Pr(\mathcal{A}) + K \Pr(\mathcal{A}^c)(1 + 2\sqrt{\log K}),
\end{aligned}$$

where the first equality is by the law of total expectation. From the inequality (8.3), we have $\Pr(\mathcal{A}^c) \leq \frac{2I}{K}$. Applying this bound and the bound $\Pr(\mathcal{A}) \leq 1$, we have

$$\begin{aligned} R_K \leq & 2I + 4\sqrt{\log K}\mathbb{E}\left[\sum_{k=1}^{K}\sqrt{\frac{1}{N_{i_k}(k)}}\middle|\mathcal{A}\right] + 2I(1+2\sqrt{\log K}) \\ = & 4\sqrt{\log K}\mathbb{E}\left[\sum_{k=1}^{K}\sqrt{\frac{1}{N_{i_k}(k)}}\middle|\mathcal{A}\right] + 4I + 4I\sqrt{\log K} \\ \leq & 4I\sqrt{\log K}\sum_{k=1}^{K}\sqrt{\frac{1}{k}} + 4I + 4I\sqrt{\log K}. \end{aligned}$$

The last inequality holds because for any arm i, $N_i(k) = n-1$ if it is played for the n-th time at time k and $n \leq k$. Therefore, for each arm i, with probability one,

$$\sum_{k=1}^{K}\sqrt{\frac{1}{N_{i_k}(k)}}\mathbf{1}_{i_k=i} \leq \sum_{k=1}^{K}\sqrt{\frac{1}{k}},$$

where $\mathbf{1}_{i_k=i}$ is an indicator function. Then we have

$$R_K \leq 4I\sqrt{\log K}\sum_{k=1}^{K}\sqrt{\frac{1}{k}} + 4I + 4I\sqrt{\log K} = O(\sqrt{K\log K}).$$

□

We call the above regret bound, $O(\sqrt{K\log K})$, an instance-independent regret bound because the bound does not depend on a specific instance of multi-armed bandits, i.e. $\{\mu_i\}_{i=1,\cdots,I}$. However, with a more complicated analysis, we can obtain an instance-dependent regret bound, which depends on the sub-optimality gaps of the instance, but the rate is better, i.e.,

$$R_K \leq \left(\sum_{i=2}^{I}\frac{16}{\Delta_i}\right)\log K + o(1),$$

where $\Delta_i = \mu_1 - \mu_i$ is the sub-optimality gap of arm i. The proof is omitted and can be found in [14].

Chapter 9

AlphaZero

In this chapter, we introduce AlphaZero [20], an RL algorithm that generalizes AlphaGo Zero that achieved superhuman performance in the Go game. Our introduction of the algorithm closely follows the online tutorial[1]. AlphaZero consists of two parts, a policy network and Monte Carlo Tree Search (MCTS). As shown in Fig. 9.1, one way to understand AlphaZero to view it as employing a similar approach to actor-critic algorithms. The policy network learns the improved from samples generated using self-play with MCTS, which is similar to the role of the critic in actor-critic algorithms, while MCTS improves the policy and generates new samples, which is similar to the role of the actor in actor-critic algorithms.

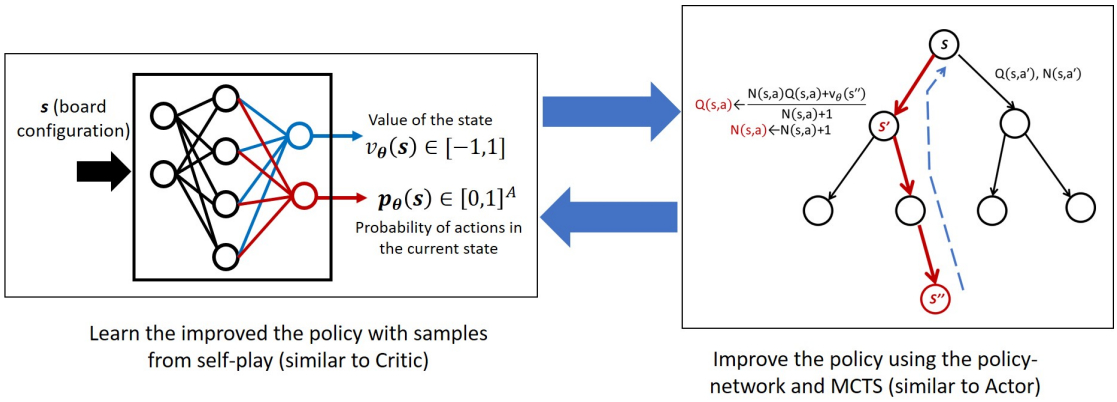


Figure 9.1: Overview of AlphaZero.

9.1 Policy Network

We first look at the policy network. The policy network tries to learn the improved policy using the samples from self-play. The input and output of the policy network are shown in Fig. 9.2. The input is the board configuration $\boldsymbol{s}$, which can usually be represented by a two-dimensional array and can be viewed as the state of the environment. The policy network has two outputs: a value

[1] `https://web.stanford.edu/~surag/posts/alphazero.html`

estimate $v_{\boldsymbol{\theta}}(\boldsymbol{s})$ and a policy vector $\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{s}) \in [0,1]^A$, where the subscript $\boldsymbol{\theta}$ denotes the weights of the policy network and A is the number of possible actions. The value estimate represents the expected outcome of the current state $\boldsymbol{s}$, with values ranging from -1 to 1. The policy vector represents the probabilities of each possible action in the current state $\boldsymbol{s}$.

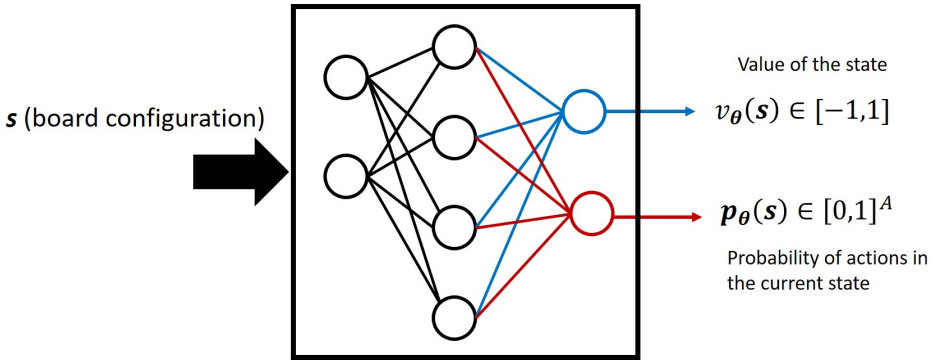


Figure 9.2: The Policy Network in AlphaZero.

At each iteration, this policy network is trained using samples generated from MCTS, which include K plays of the game. Let $(\boldsymbol{s}_{h,k}, \boldsymbol{\pi}_{h,k}, z_k)$ denote the sample from step h of the kth play, where

- $\boldsymbol{s}_{h,k}$: The state, i.e. the board configuration;
- $\boldsymbol{\pi}_{h,k}$: An A-dimension vector such that $\boldsymbol{\pi}_{h,k}(a)$ represents the probability of taking action a in state $\boldsymbol{s}_{h,k}$;
- $z_k \in \{-1, 0, +1\}$: The final outcome of the kth play, where -1 means a loss, 0 means a draw, and 1 means a win.

The policy network is trained using mini-batches of these samples and the following loss function:

$$\mathcal{L}(\boldsymbol{\theta}) = \sum_b (v_{\boldsymbol{\theta}}(\boldsymbol{s}_{h,k}) - z_k)^2 - \boldsymbol{\pi}_{h,k} \cdot \log(\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{s}_{h,k})) + c\|\boldsymbol{\theta}\|^2,$$

where $\cdot$ is the dot product of two vectors and the last term is for regularization.

We can see the training of the policy network can be viewed as supervised learning so that the policy network learns a new policy $\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{s}_{h,k}) \approx \boldsymbol{\pi}_{h,k}$ and the value of the new policy $v_{\boldsymbol{\theta}}(\boldsymbol{s}_{h,k}) \approx z_k$.

9.2 Monte Carlo Tree Search

We next discuss the way Monte Carlo tree search (MCTS) improves the policy and generates new samples through self-play. A self-play game is a game in which the agent is playing against itself. After a certain number of self-play games, new samples are obtained and they will be fed to the policy network for training. For each self-play game, the agent plays the game from the beginning to the end. For each step in a self-play game, given the current board configuration $\boldsymbol{s}_{h,k}$ and policy network $\boldsymbol{\theta}$, we want to generate a new distribution $\boldsymbol{\pi}_{h,k}$, a randomized policy, based on MCTS, i.e. a sample $(\boldsymbol{s}_{h,k}, \boldsymbol{\pi}_{h,k}, ?)$, where "?" is the result of the game and is determined after the game ends.

Before illustrating how to generate a new sample, we first show what a tree in MCTS looks like. Each node represents the state at that step. Each state $\boldsymbol{s}$ maintains the following quantities:

- $Q(\boldsymbol{s}, a)$: The Q-value for state $\boldsymbol{s}$ and action a.
- $N(\boldsymbol{s}, a)$: The number of visits to state $\boldsymbol{s}$ and action a in the self-play.

Each edge represents an action a and connects the current state $\boldsymbol{s}$ and the next state $\boldsymbol{s}$ under action a, as shown in Fig. 9.3.

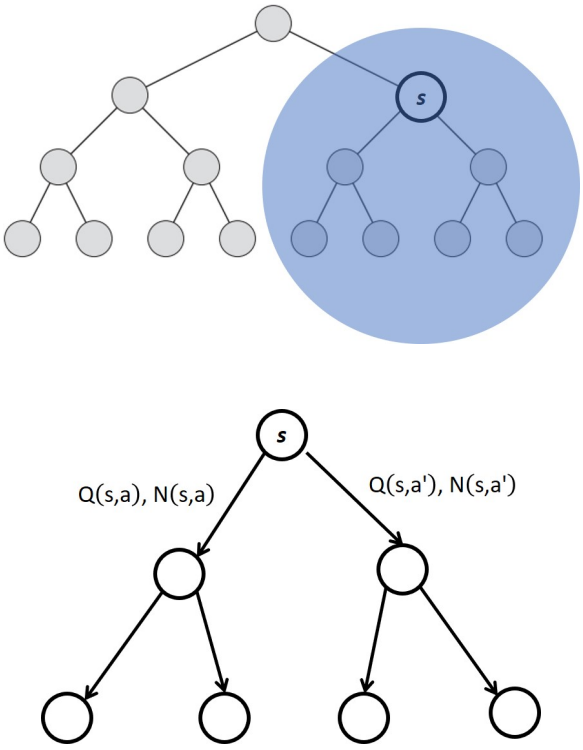


Figure 9.3: MCTS Tree Structure.

To generate a new sample, MCTS performs a number of simulations. Each simulation includes the following steps.

A Self-Play Simulation

- Step 1: Select action a such that

$$a \in \arg\max_{u} Q(\boldsymbol{s}, u) + hp_{\boldsymbol{\theta}}(\boldsymbol{s}, u) \frac{\sqrt{\sum_v N(\boldsymbol{s}, v)}}{1 + N(\boldsymbol{s}, u)},$$

  where h is a constant and $p_{\boldsymbol{\theta}}(\boldsymbol{s}, u)$ denotes the uth element of the vector $\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{s}_{h,k})$, which corresponds to the probability of taking action u based on the policy network. Then go to the next node following the edge associated with action a.

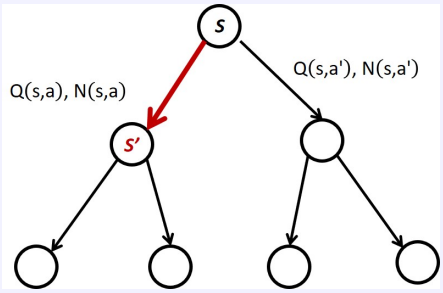


Figure 9.4: One Simulation.

- Step 2: Repeat Step 1 as shown in Fig. 9.4. Terminate when meeting state $\tilde{\boldsymbol{s}}$, which is a new state or a terminal state. A terminal state is a state where the result of the game can be declared.

  – If the terminal state is met, update $Q(\boldsymbol{s}', a)$ and $N(\boldsymbol{s}', a)$ along the path with the game result $z \in \{-1, 0, +1\}$ as follows:

$$Q(\boldsymbol{s}', a) \longleftarrow \frac{Q(\boldsymbol{s}', a)N(\boldsymbol{s}', a) \ +/- \ z}{N(\boldsymbol{s}', a) + 1}$$
$$N(\boldsymbol{s}', a) \longleftarrow N(\boldsymbol{s}', a) + 1,$$

  where the symbol $+/-$ represents a positive or negative sign, depending on whether the player who takes the action in $\boldsymbol{s}'$ is the same as the player who takes the action in state $\tilde{\boldsymbol{s}}$.

  – If a new state is met, add it to the tree and set $N(\tilde{\boldsymbol{s}}, a) = Q(\tilde{\boldsymbol{s}}, a) = 0$ for all a. Then update $Q(\boldsymbol{s}', a)$ and $N(\boldsymbol{s}', a)$ along the path with $v_{\boldsymbol{\theta}}(\tilde{\boldsymbol{s}})$ as follows:

$$Q(\boldsymbol{s}', a) \longleftarrow \frac{Q(\boldsymbol{s}', a)N(\boldsymbol{s}', a) + / - v_{\boldsymbol{\theta}}(\tilde{\boldsymbol{s}})}{N(\boldsymbol{s}', a) + 1}$$
$$N(\boldsymbol{s}', a) \longleftarrow N(\boldsymbol{s}', a) + 1,$$

  where $+/-$ depends on whether the player who takes the action in state $\boldsymbol{s}'$ is the same as the player who takes the action in state $\tilde{\boldsymbol{s}}$. See Fig. 9.5.

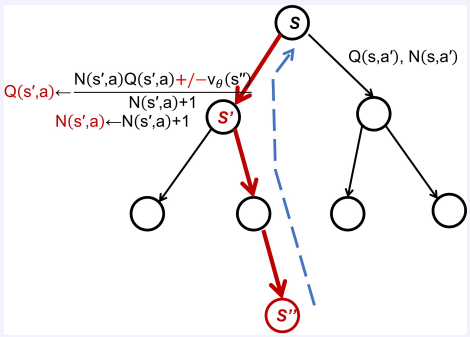


Figure 9.5: One Simulation: Update Q and N.

We note that $\frac{\sqrt{\sum_v N(\boldsymbol{s},v)}}{1+N(\boldsymbol{s},u)}$ plays a similar role as the UCB bonus in multi-armed bandits to encourage exploration.

After many simulations, we sample an action according to $\pi(a|\boldsymbol{s}) = \frac{N(\boldsymbol{s},a)}{\sum_{a'} N(\boldsymbol{s},a')}$, and move the state to the next state by following the sampled action, which concludes one step of the self-play. Both black and white are played by the same agent (self-play) until a game is finished.

9.3 AlphaZero

After one iteration (including many self-play games and the training of the new policy network based on the new samples), the new and old policies are pit against each other. We update the policy if the new policy wins and otherwise, we repeat the iteration to obtain more samples to retrain the policy.

9.4 Monte Carlo Tree Search: Example

In this section, we use an example to illustrate the process of MCTS. Suppose we have a 5×5 board. The initial board position is shown in Figure 9.6

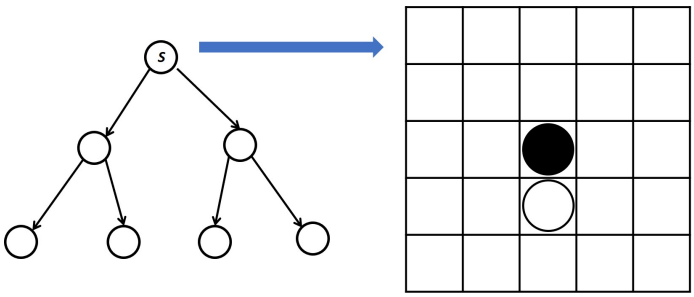


Figure 9.6: Initial Board and the Tree.

In one simulation, we first select action a according to Step 1 and then go to next state $\boldsymbol{s}'$ on the MCTS tree, as shown in Fig. 9.7. We continue this simulation as shown in Fig. 9.8 and Fig. 9.9. Suppose the simulation terminates after encountering a new state $\boldsymbol{s}'''$. Then we calculate $v_{\boldsymbol{\theta}}(\boldsymbol{s}''')$ using the policy network, as shown in Fig. 9.10.

Assume the original (Q, N) values are

$$\begin{aligned} N(\boldsymbol{s},a) &= 2, \quad Q(\boldsymbol{s},a) = 0.7 \\ N(\boldsymbol{s}',a') &= 1, \quad Q(\boldsymbol{s}',a') = 0.6 \\ N(\boldsymbol{s}'',a'') &= 0, \quad Q(\boldsymbol{s}'',a'') = 0, \end{aligned}$$

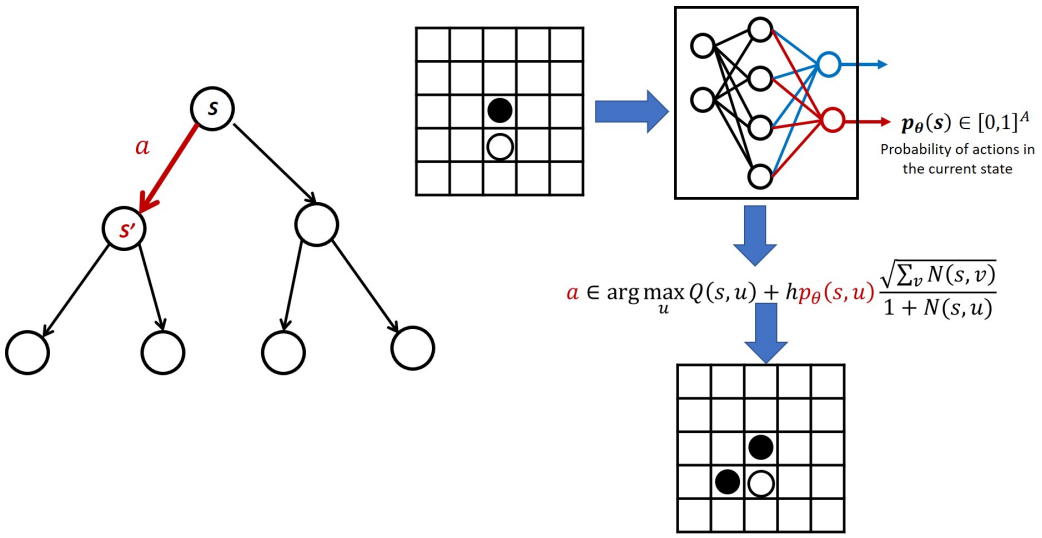


Figure 9.7: Search the Tree by Taking a Virtual Action.

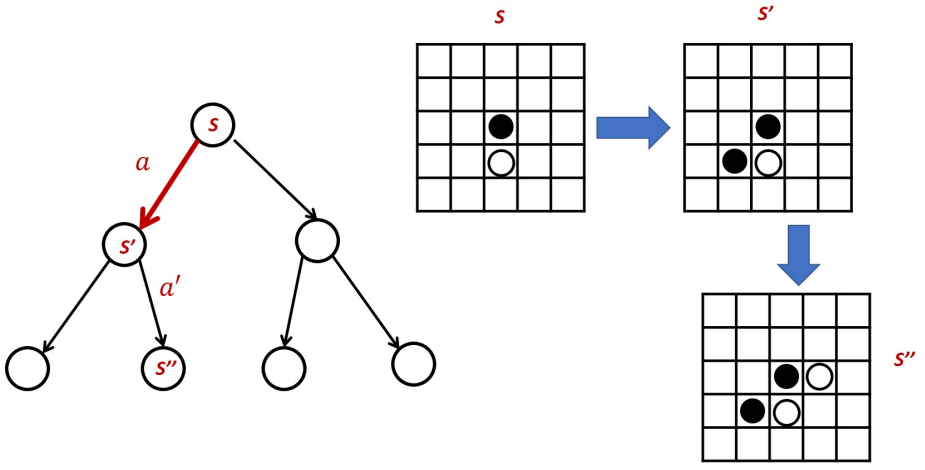


Figure 9.8: Continue Searching the Tree.

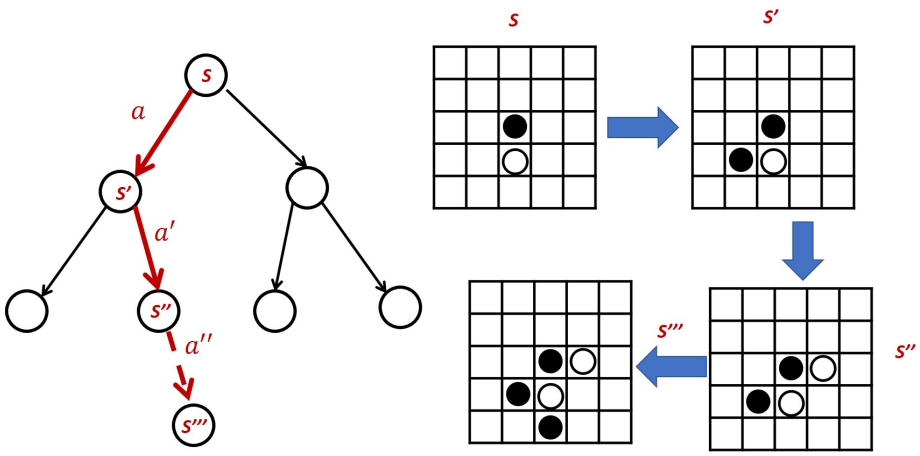


Figure 9.9: Continue Searching the Tree (ctd).

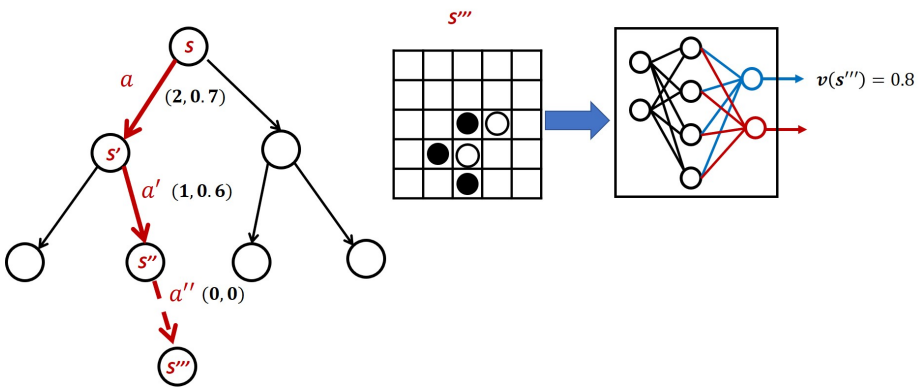


Figure 9.10: Encounter a New State

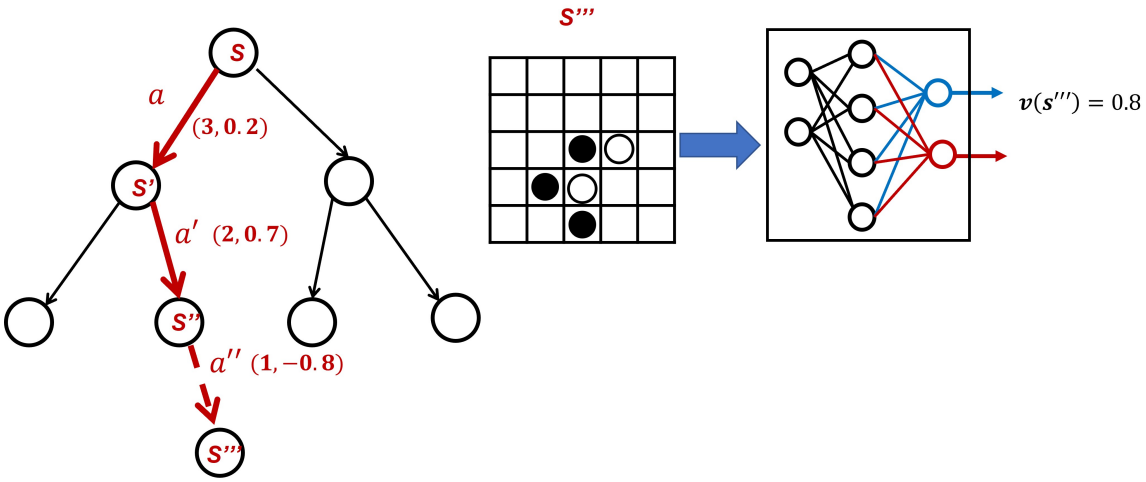


Figure 9.11: Update Q and N.

which are also shown in Fig. 9.10. Then we can update the (Q, N) values according to step 2, i.e.,

$$N(\boldsymbol{s}, a) = 2 + 1 = 3, \quad Q(\boldsymbol{s}, a) = \frac{2 \times 0.7 - 0.8}{2 + 1} = 0.2$$
$$N(\boldsymbol{s}', a') = 1 + 1 = 2, \quad Q(\boldsymbol{s}', a') = \frac{1 \times 0.6 + 0.8}{1 + 1} = 0.7$$
$$N(\boldsymbol{s}'', a'') = 0 + 1 = 1, \quad Q(\boldsymbol{s}'', a'') = \frac{0 \times 0 - 0.8}{0 + 1} = -0.8.$$

The results are also shown in Fig. 9.11.

Suppose after many simulations, we obtain the tree in Fig. 9.12, where the numbers next to the edges are $N(\boldsymbol{s}, a)$. Then we can calculate new action probabilities for state $\boldsymbol{s}$ as follows:

$$\boldsymbol{\pi}(\boldsymbol{s}) = \left(\frac{8}{16}, \frac{4}{16}, \frac{4}{16}\right) = (0.5, 0.25, 0.25),$$

which finishes one step of the self-play and generates a data sample

$$\text{data sample} = (s, (0.5, 0.25, 0.25), ?),$$

where "?" will be determined by the final result of the self-play game.

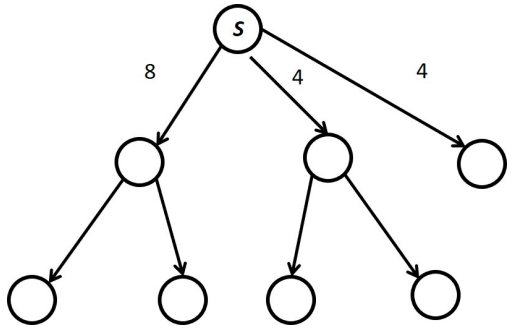


Figure 9.12: The Tree After Many Simulations.

Chapter 10

Optimistic Q-Learning (UCB-SARSA)

The online Q-learning algorithm, SARSA, introduced in Chapter 3 uses ϵ-greedy to explore. From the multi-armed bandit setting, we have observed that UCB has a much smaller regret than ϵ-greedy. In this chapter, we will introduce a variation of SARSA based on UCB [10] and discuss its regret bound.

10.1 UCB-SARSA (Optimistic Q-Learning) Algorithm

Consider an episodic MDP with episode length of H. Assume the reward is deterministic $r_h(x,a) \in [0,1]$. Then the UCB-SARSA, also called the optimistic Q-Learning algorithm, is as follows:

UCB-SARSA (Optimistic Q-Learning) [10]

Initialize $\tilde{Q}_{1,h}(x,a) = H$ and $N_{1,h}(x,a) = 0$. For episode k,

- At step h, choose action u_h such that

$$u_h \in \arg\max_v \tilde{Q}_{k,h}(x_h, v).$$

- With data $(x_{h-1} = x, u_{h-1} = u, x_h, u_h)$ and $r_h(x_{h-1}, u_{h-1}) = r$, update

$$\tilde{Q}_{k+1,h-1}(x,u) = \tilde{Q}_{k,h-1}(x,u) + \beta_{k,h}(r + \tilde{Q}_{k,h}(x_k, u_k) + b_{k,h-1} - \tilde{Q}_{k,h-1}(x,u))$$

We note that the key difference between UCB-SARSA and SARSA is the UCB bonus term $b_{k,h-1}$ added to the Q-value update. The UCB-SARSA uses a state-dependent learning rate and the UCB bonus is carefully designed. Let $N = N_{k,h}(x,a)$ where $N_{k,h}(x,a)$ is the number of visits to state action pair (x,a) at step h from episode 1 to $k-1$. The following learning rate and UCB bonus are used in the algorithm:

- Learning rate: $\beta_{k,h} = \frac{H+1}{H+N}$.
- UCB bonus: $b_{k,h} = c_K\sqrt{\frac{H^3}{N}}$, where c_K is logarithmic function in K.

10.2 Regret Analysis of UCB-SARSA

Let π^* denote the optimal policy, $V_h^\pi(x)$ be the value function associated with step h and state x under policy π, and x_1 is the initial state of each episode (assuming it is deterministic for simplicity). The regret of UCB-SARSA is defined to be

$$R_K = KV_1^{\pi^*}(x_1) - \sum_{k=1}^{K} \mathbb{E}\left[V_1^{\pi_k}(x_1)\right] = \tilde{\mathcal{O}}\left(\sqrt{K}\right),$$

where $\tilde{\mathcal{O}}(\cdot)$ differs from $O(\cdot)$ by a poly-logarithmic factor and the expectation is taken over policy π_k. Policy π_k is random because it is determined by the Q-values $\tilde{Q}_{k,h}$, which are random variables.

The first step of proving the regret bound of UCB-SARSA is regret decomposition by adding and subtracting $\sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right]$:

$$\begin{aligned}
R_K =& KV_1^{\pi^*}(x_1) - \sum_{k=1}^{K} \mathbb{E}\left[V_1^{\pi_k}(x_1)\right] \\
=& KV_1^{\pi^*}(x_1) - \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] + \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] - \sum_{k=1}^{K} \mathbb{E}\left[Q_1^{\pi_k}(x_1, \pi_k(x_1))\right] \\
=& \sum_{k=1}^{K} \mathbb{E}_{\pi^*}\left[Q_1^{\pi^*}(x_1, \pi^*(x_1))\right] - \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] \qquad (10.1) \\
&+ \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] - \sum_{k=1}^{K} \mathbb{E}\left[Q_1^{\pi_k}(x_1, \pi_k(x_1))\right]. \qquad (10.2)
\end{aligned}$$

Note that the regret now is decomposed into two terms, where (10.1) is the difference between the estimated Q-value of the current policy versus the Q-value under the optimal policy, and (10.2) is the difference between estimated Q-value of policy π_k and the actual Q-value of policy π_k. The first important result is an optimistic estimation, as stated in the following lemma, which bounds (10.1). The proof of this lemma can be found in [10] and in Appendix D.

Lemma 10.2.1 *With a high probability, for any h, (x, a) and π,*

$$Q_h^\pi(x, a) \leq \tilde{Q}_{k,h}(x, a),$$

where k is the episode index.

With this result, we have

$$Q_1^{\pi^*}(x_1, \pi^*(x_1)) \leq \tilde{Q}_{k,h}(x_1, \pi^*(x_1)) \leq \max_a \tilde{Q}_{k,h}(x_1, a) = \tilde{Q}_{k,h}(x_1, \pi_k(x_1)).$$

Hence, with a high probability,

$$\sum_{k=1}^{K} \mathbb{E}_{\pi^*}\left[Q_1^{\pi^*}(x_1, \pi^*(x_1))\right] - \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] \leq 0,$$

which implies that with a high probability,

$$R_K \leq \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] - \sum_{k=1}^{K} \mathbb{E}\left[Q_1^{\pi_k}(x_1, \pi_k(x_1))\right].$$

Note that $\tilde{Q}_{k,1}(x_1, \pi_k(x_1))$ is an "estimate" of $Q_1^{\pi_k}(x_1, \pi_k(x_1))$ based on π_m for $m < k$. We need to understand the bias of this estimation. The result is summarized in the following lemma. The proof of this lemma can be found in [10] and in Appendix D.

Lemma 10.2.2

$$\sum_{k=1}^{K} E_{\pi_k}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1))\right] - \sum_{k=1}^{K} E_{\pi_k}\left[Q_1^{\pi_k}(x_1, \pi_k(x_1))\right] = \tilde{\mathcal{O}}(\sqrt{K})$$

Therefore, we conclude that under UCB-SARSA, with a high probability,

$$R_K = \tilde{\mathcal{O}}(\sqrt{K}).$$

The regret comes from (10.2).

Appendix A

Convergence and Optimality of Model-Based Solutions

In this chapter, we study the convergence and the optimality of model-based algorithms. We consider infinite-horizon discounted MDPs in this chapter.[1]

A key mathematical tool we will use to prove the convergence and optimality is the following contraction mapping theorem. We first present the definition of contraction mapping. Let $\mathcal{T}$ be a mapping from $\mathbb{R}^n$ to $\mathbb{R}^n$. $\mathcal{T}$ is called a contraction mapping if

$$\|\mathcal{T}(x) - \mathcal{T}(y)\| \leq \alpha \|x - y\|, \quad \forall x, y \in \mathbb{R}^n,$$

where $\alpha \in [0, 1)$ and $\|\cdot\|$ is some norm. The contraction mapping theorem is presented below.

Theorem A.0.1 (Contraction Mapping Theorem) *Under a contraction mapping $\mathcal{T}$, the following results hold.*

1. *There exists a unique $x^* \in \mathbb{R}^n$ such that*

$$x^* = \mathcal{T}(x^*).$$

   x^* is called a fixed point of the mapping.

2. *The iteration $x_{k+1} = \mathcal{T}(x_k)$ converges to x^*, starting from any initial condition $x_0 \in \mathbb{R}^n$.*

Proof Given a vector $x_0 \in \mathbb{R}^n$, we define $x_1 = \mathcal{T}(x_0)$, $x_2 = \mathcal{T}(x_1) = \mathcal{T}^2(x_0)$, and so on. Then by the triangle inequality, for any integers $k \geq 0$, $l > 0$, we have

$$\|x_k - x_{k+l}\| \leq \|x_k - x_{k+1}\| + \cdots + \|x_{k+l-1} - x_{k+l}\|. \tag{A.1}$$

[1]This chapter is based on Prof. R. Srikant's lecture notes on *MDPs with discounted cost* available at `https://sites.google.com/illinois.edu/mdps-and-rl/lectures?authuser=1` and extensive discussions on convergence and optimality can be found in [6]

According to the definition of contraction mapping, for any $k \geq 0$, we have

$$\begin{aligned}\|x_{k+1} - x_k\| &= \|\mathcal{T}(x_k) - \mathcal{T}(x_{k-1})\| \\ &\leq \alpha\|x_k - x_{k-1}\| \\ &= \alpha\|\mathcal{T}(x_{k-1}) - \mathcal{T}(x_{k-2})\| \\ &\leq \alpha^2\|x_{k-1} - x_{k-2}\| \\ &\leq \cdots \leq \alpha^k\|x_1 - x_0\|. \end{aligned} \tag{A.2}$$

Combining (A.1) and (A.2), we have

$$\begin{aligned}\|x_k - x_{k+l}\| &\leq (\alpha^k + \alpha^{k+1} + \cdots + \alpha^{k+l-1})\|x_1 - x_0\| \\ &= \alpha^k(1 + \alpha + \cdots + \alpha^{l-1})\|x_1 - x_0\| \\ &\leq \frac{\alpha^k}{1-\alpha}\|x_1 - x_0\|,\end{aligned}$$

where the last inequality holds due to the fact that $\sum_{i=0}^{l-1}\alpha^i \leq \sum_{i=0}^{\infty}\alpha^i = \frac{1}{1-\alpha}$. We note that this upper bound on $\|x_k - x_{k+l}\|$ is independent of l. Since $\alpha \in [0, 1)$, given $\epsilon > 0$, there exists $N_\epsilon > 0$ such that

$$\|x_k - x_{k+l}\| \leq \epsilon \quad \forall k \geq N_\epsilon \text{ and } l \geq 1,$$

which implies that

$$\|x_k - x_m\| \leq \epsilon \quad \forall k, m \geq N_\epsilon.$$

Therefore, $(x_k)_{k=0}^{\infty}$ is a Cauchy sequence, so it converges. Let x^* be the point to which the sequence converges to. Since $\mathcal{T}$ is a contraction mapping, it is continuous and

$$x^* = \lim_{k\to\infty} x_k = \lim_{k\to\infty} \mathcal{T}(x_{k-1}) = \mathcal{T}(\lim_{k\to\infty} x_{k-1}) = \mathcal{T}(x^*).$$

Now it remains to prove the uniqueness of x^*. Suppose there exists another fixed point $y^* \neq x^*$ such that $\mathcal{T}(y^*) = y^*$. Then by the definition of contraction mapping, we have

$$\|y^* - x^*\| = \|\mathcal{T}(y^*) - \mathcal{T}(x^*)\| \leq \alpha\|y^* - x^*\|.$$

Since $\alpha < 1$, the inequality above implies that

$$\|y^* - x^*\| = 0,$$

which contradicts the assumption that $y^* \neq x^*$.

□

A.1 Value Iteration

Recall the definition of the value function under the optimal policy:

$$V^*(x) = \sup_{\mu_0,\mu_1,\cdots} \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, \mu_k(x_k)) \middle| x_0 = x\right],$$

and the value function under a given policy $\mu = \{\mu_0, \mu_1, \cdots\}$:

$$V_\mu(x) = \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k r(x_k, \mu_k(x_k)) \middle| x_0 = x\right],$$

Theorem A.1.1 *The value function V^* satisfies the following Bellman equation:*

$$\begin{aligned} V^*(x) &= \max_u \mathbb{E}[r(x,u) + \alpha V^*(x_1) | x_0 = x, u_0 = u] \\ &= \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{xx'}(u) V^*(x'), \quad \forall x. \end{aligned}$$

Proof Given policy $\mu = \{\mu_0, \mu_1, \dots\}$, we define $\mu^k = \{\mu_k, \mu_{k+1}, \dots\}$ which is a subpolicy of μ starting from time k. From the definitions of μ, μ^k and the value function for a given policy, we have

$$V_\mu(x) = \mathbb{E}\left[r(x, \mu_0(i)) + \alpha \sum_{x'} P_{x,x'}(\mu_0(x)) V_{\mu^1}(x') \middle| x_0 = x\right].$$

Recall that $V^*(x') \geq V_\mu(x')$ for all μ and x' by its definition. Hence, for any policy μ, we have

$$\begin{aligned} V_\mu(x) &\leq \mathbb{E}[r(x, \mu_0(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_0(x)) V^*(x') \\ &= \max_u \left\{ \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V^*(x') \right\}. \end{aligned} \tag{A.3}$$

Taking supreme on both sides yields

$$\begin{aligned} V^*(x) = \sup_\mu V_\mu(x) &\leq \sup_\mu \max_u \left\{ \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V^*(x') \right\} \\ &= \max_u \left\{ \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V^*(x') \right\}, \end{aligned}$$

where the last equality holds because it is independent of the policy.

Next given $\epsilon > 0$, let $\mu^{(x)}$ be a policy such that

$$V_{\mu^{(x)}}(x) \geq V^*(x) - \epsilon.$$

In other words, starting from initial state x, $\mu^{(x)}$ is no worse than μ^* by an amount of ϵ. We note that $\mu^{(x)}$ exists by the definition of $V^*(x) = \sup_\mu V_\mu(x)$. Now consider the policy such that we

choose action u_0 at step 0 and then follow policy $\mu^{(x')}$ if the environment moves to state x'. Then we have for any u_0,

$$\begin{aligned} V^*(x) &\geq \mathbb{E}[r(x,u_0)] + \alpha \sum_{x'} P_{x,x'}(u_0) V_{\mu^{(x')}}(x') \\ &= \mathbb{E}[r(x,u_0) + \alpha V_{\mu^{(x)}}(x') | x_0 = x, u_0] \\ &\geq \mathbb{E}[r(x,u_0) + \alpha V^*(x') | x_0 = x, u_0] - \alpha\epsilon. \end{aligned}$$

Since the inequality holds for any u_0, we further have

$$V^*(x) \geq \max_{u_0} \mathbb{E}[r(x,u_0) + \alpha V^*(x') | x_0 = x, u_0] - \alpha\epsilon.$$

Letting $\epsilon \to 0$, we obtain

$$V^*(x) \geq \max_{u_0} \mathbb{E}[r(x,u_0) + \alpha V^*(x') | x_0 = x, u_0]. \tag{A.4}$$

Combining the inequalities (A.3) and (A.4), the theorem is proved.

□

Theorem A.1.1 shows that the value function of the optimal policy is a fixed point of the Bellman equation. The next questions to be answered are: (i) does the fixed point unique? (ii) if the fixed point is unique, does the value iteration algorithm converge?

We will answer these two questions by showing that the Bellman equation defines a contraction mapping. We define the Bellman operator $\mathcal{T}$ to be a mapping from $\mathbb{R}^n$ to $\mathbb{R}^n$ such that

$$\mathcal{T}(V)(x) = \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V(x')$$

where $V(x)$ and $\mathcal{T}(V)(x)$ denotes the x-th element of V and the x-th element of $\mathcal{T}(V)$, respectively. Under this definition, the value iteration algorithm can be written as

$$V_{k+1} = \mathcal{T}(V_k)$$

and the Bellman Equation can be written as

$$V^* = \mathcal{T}(V^*).$$

Theorem A.1.2 *$\mathcal{T}$ is a contraction mapping given $\alpha \in [0,1)$.*

Proof We first prove the following two results:

- If $V_1(x) \leq V_2(x)$ for any x, then

  $$\mathcal{T}(V_1)(x) \leq \mathcal{T}(V_2)(x)$$

  for any x.

- Let $e = \begin{pmatrix} 1 \\ 1 \\ \vdots \end{pmatrix}$, then

$$\mathcal{T}(V + \gamma e) = \mathcal{T}(V) + \alpha\gamma e$$

for any scalar γ.

To prove the first result, we note that

$$\begin{aligned}\mathcal{T}(V_1)(x) =& \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_1(x') \\ \leq& \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_2(x') \\ =& \mathcal{T}(V_2)(x).\end{aligned}$$

To prove the second result, we note

$$\begin{aligned}\mathcal{T}(V + \gamma e)(x) =& \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) \left(V(x') + \gamma\right) \\ \leq& \max_u \mathbb{E}[r(x,u)] + \left(\alpha \sum_{x'} P_{x,x'}(u) V(x')\right) + \alpha\gamma \\ =& \mathcal{T}(V)(x) + \alpha\gamma.\end{aligned}$$

Now, we will use these two facts to prove Theorem A.1.2. Consider any two value functions (vectors) V_1 and V_2. Define

$$b = \max_x |V_1(x) - V_2(x)| = \|V_1 - V_2\|_\infty.$$

Then for any x,

$$V_2(x) - b \leq V_1 \leq V_2(x) + b.$$

By Result 1, we have

$$\mathcal{T}(V_2 - be) \leq \mathcal{T}(V_1) \leq \mathcal{T}(V_2 + be).$$

Further from Result 2, we have

$$\mathcal{T}(V_2) - \alpha be \leq \mathcal{T}(V_1) \leq \mathcal{T}(V_2) + \alpha be.$$

Hence,

$$\|\mathcal{T}(V_1) - \mathcal{T}(V_2)\|_\infty \leq \alpha b = \alpha \|V_1 - V_2\|_\infty.$$

Since $\alpha < 1$, by the definition of contraction mapping, $\mathcal{T}$ is a contraction mapping.

□

Since the Bellman operator is a contraction mapping, from Theorem A.0.1(Contraction Mapping Theorem), the Bellman equation $V^* = \mathcal{T}(V^*)$ has a unique solution, and the value iteration algorithm $V_{k+1} = \mathcal{T}(V_k)$ converges to V^*.

Next we will show that an optimal policy can be obtained by taking greedy actions based on the value function. To show this, we first present a lemma for the uniqueness of the value function for a given policy, whose proof is based on the contraction mapping theorem.

Lemma A.1.3 (Uniqueness) *Given a policy μ, there exists a unique vector V_μ which satisfies the following Bellman equation*

$$V_\mu(x) = \bar{r}(x, \mu(x)) + \alpha \sum_{x'} P_{x,x'}(\mu(x)) V_\mu(x'), \quad \forall x \tag{A.5}$$

and $V_\mu(x)$ is the value function under policy μ.

Proof For convenience, let $P_{x,x'} = P_{x,x'}(\mu(x))$. Define a mapping $\mathcal{T}_\mu$ from $\mathbb{R}^n$ to $\mathbb{R}^n$ such that

$$\mathcal{T}_\mu(x) = \bar{r}_\mu + \alpha P x$$

for any vector $x \in \mathbb{R}^n$, where

$$\bar{r}_\mu = \begin{pmatrix} \bar{r}(1, \mu(1)) \\ \bar{r}(2, \mu(2)) \\ \vdots \end{pmatrix} \quad \text{and} \quad P = \begin{pmatrix} P_{11}, & P_{12}, & \cdots \\ P_{21}, & P_{2,2}, & \cdots \\ \vdots, & \vdots, & \cdots \end{pmatrix}.$$

Then for any vectors $V_1, V_2 \in \mathbb{R}^n$, we have

$$\begin{aligned} \|\mathcal{T}_\mu(V_1) - \mathcal{T}_\mu(V_2)\|_\infty &= \|\alpha P(V_1 - V_2)\|_\infty \\ &= \alpha \max_x |P(V_1 - V_2)(x)| \\ &= \alpha \max_x \left| \sum_{x'} P_{x,x'}(V_1(x') - V_2(x') \right| \\ &\leq \alpha \max_x \sum_{x'} P_{x,x'} |V_1(x') - V_2(x')| \\ &\leq \alpha \max_x \sum_{x'} P_{x,x'} \max_{x'} |V_1(x') - V_2(x')| \\ &= \alpha \max_x \sum_{x'} P_{x,x'} \|V_1 - V_2\|_\infty. \end{aligned}$$

where the first inequality is by the triangle inequality. Note that $\sum_{x'} P_{x,x'} = 1$ for any x since P is a transition probability matrix. Hence, we have

$$\|\mathcal{T}_\mu(V_1) - \mathcal{T}_\mu(V_2)\|_\infty \leq \alpha \|V_1 - V_2\|_\infty.$$

Thus $\mathcal{T}_\mu$ is a contraction mapping, which implies that $V_\mu = \mathcal{T}_\mu(V_\mu)$ has a unique solution by Theorem A.0.1.

Now given V_μ which is a fixed point the Bellman equation for a given policy, we have

$$\begin{aligned} V_\mu(x) =& \mathbb{E}\left[r(x, \mu(x)) + \alpha V_\mu(x_1) | x_0 = x\right] \\ =& \mathbb{E}\left[r(x, \mu(x)) + \alpha r(x_1, \mu(x_1)) + \alpha^2 V_\mu(x_2) | x_0 = x\right] \\ =& \mathbb{E}\left[\sum_{t=0}^{\infty} \alpha^t r(x_t, \mu(x_t)) \middle| x_0 = x\right]. \end{aligned}$$

This shows that V_μ is the value function under policy μ.

□

Theorem A.1.4 *Let μ^* be a policy such that*

$$\mu^*(x) \in \arg\max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V^*(x').$$

Then μ^* is an optimal policy.

Proof Given policy μ^* defined above, by Lemma A.1.3, there exists a unique V_{μ^*} such that

$$V_{\mu^*} = \mathcal{T}_{\mu^*}(V_{\mu^*}).$$

Furthermore, for any x,

$$\begin{aligned}
\mathcal{T}_{\mu^*}(V^*)(x) &= \mathbb{E}[r(x,\mu^*(x))] + \alpha \sum_{x'} P_{x,x'}(\mu^*(x)) V^*(x') \\
&= \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V^*(x') \\
&= \mathcal{T}(V^*)(x) \\
&= V^*(x),
\end{aligned}$$

where the second equality is by the definition of μ^*. The result above implies that $\mathcal{T}_{\mu^*}(V^*) = V^*$. Therefore, V^* is also a fixed point of $\mathcal{T}_{\mu^*}$. Since T_{μ^*} has a unique fixed point,

$$V_{\mu^*} = V^*.$$

Since V_{μ^*} is the value function under the policy μ^* and V^* is the optimal value function, μ^* is an optimal policy.

□

A.2 Policy Iteration

For policy iteration, we will first show that the policy continues to improve and then prove that the obtained policy is optimal if it cannot be improved under policy iteration.

Theorem A.2.1 *Let μ_k denote the policy at the beginning of iteration k. For any x, we have*

$$V_{\mu_k}(x) \leq V_{\mu_{k+1}}(x).$$

Proof Define $Q_{\mu_k}(x,u)$ to be the Q-value of state-action pair (x,u) under policy μ_k, i.e.,

$$Q_{\mu_k}(x,u) = \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_{\mu_k}(x').$$

Considering action $\mu_{k_+}(x)$, we have

$$Q_{\mu_k}(x,\mu_{k+1}(x)) = \mathbb{E}[r(x,\mu_{k+1}(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_{k+1}(x)) V_{\mu_k}(x').$$

Under the policy iteration algorithm, V_{μ_k} is the solution to the following set of equations

$$V_{\mu_k}(x) = \mathbb{E}[r(x, \mu_k(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_k(x)) V_{\mu_k}(x'),$$

and

$$\mu_{k+1}(x) \in \arg\max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_{\mu_k}(x').$$

Therefore, we have for any state x,

$$V_{\mu_k}(x) \leq Q_{\mu_k}(x, \mu_{k+1}(x)). \tag{A.6}$$

Expanding Q_{μ_k} iteratively, we have for any state x,

$$\begin{aligned}
V_{\mu_k}(x) \leq & Q_{\mu_k}(x, \mu_{k+1}(x)) \\
= & \mathbb{E}[r(x, \mu_{k+1}(x))] + \alpha \sum_j P_{x,x'}(\mu_{k+1}(x)) V_{\mu_k}(x') \\
= & \mathbb{E}_{x' \sim P_{x,x'}(\mu_{k+1}(x))} \left[r(x, \mu_{k+1}(x)) + \alpha V_{\mu_k}(x') \right] \\
\leq & \mathbb{E}_{x' \sim P_{x,x'}(\mu_{k+1}(x))} \left[r(x, \mu_{k+1}(x)) + \alpha Q_{\mu_k}(x', \mu_{k+1}(x')) \right] \\
= & \mathbb{E}_{x' \sim P_{x,x'}(\mu_{k+1}(x))} \left[r(x, \mu_{k+1}(x)) + \alpha r(x', \mu_{k+1}(x')) + \alpha^2 \sum_{x''} P_{x',x''}(\mu_{k+1}(jx') V_{\mu_k}(x'') \right] \\
= & \mathbb{E}_{x' \sim P_{x,x'}(\mu_{k+1}(x)), x'' \sim P_{x',x''}(\mu_{k+1}(x'))} \left[r(i, \mu_{k+1}(x)) + \alpha r(x', \mu_{k+1}(x')) + \alpha^2 V_{\mu_k}(x'') \right].
\end{aligned}$$

We note that x' and x'' are the states starting from state x and under policy μ_{k+1}. Continuing the expansion above, we obtain

$$V_{\mu_k}(x) \leq \mathbb{E}_{\mu_{k+1}} \left[\sum_{t=0}^{\infty} \alpha^t r(x_t, \mu_{k+1}(x_t)) \middle| x_0 = x \right] = V_{\mu_{k+1}}(x)$$

where x_t is the state at time t.

□

The theorem above shows that the value of the policy continues to improve under policy iteration. Finally, the following theorem shows that the optimal policy is obtained when policy iteration terminates, i.e. when $\mu_{k+1} = \mu_k$.

Theorem A.2.2 *If $V_{\mu_k} = V_{\mu_{k+1}}$, i.e., the policy does not improve, then μ_k and μ_{k+1} are both optimal policies.*

Proof Since $V_{\mu_{k+1}}(x) = V_{\mu_k}(x)$ for all x, we have

$$\begin{aligned}
V_{\mu_{k+1}}(x) = & \mathbb{E}[r(x, \mu_{k+1}(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_{k+1}(x)) V_{\mu_{k+1}}(x') \\
= & \mathbb{E}[r(x, \mu_{k+1}(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_{k+1}(x)) V_{\mu_k}(x').
\end{aligned}$$

Since under policy iteration, for any x,

$$\mu_{k+1}(x) \in \arg\max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_{\mu_k}(x'),$$

we have

$$\begin{aligned}
&\max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_{\mu_k}(x') \\
=&\mathbb{E}[r(x,\mu_{k+1}(x))] + \alpha \sum_{x'} P_{x,x'}(\mu_{k+1}(x)) V_{\mu_k}(x') \\
=&V_{\mu_{k+1}}(x).
\end{aligned}$$

Therefore, we conclude

$$V_{\mu_k}(x) = \max_u \mathbb{E}[r(x,u)] + \alpha \sum_{x'} P_{x,x'}(u) V_{\mu_k}(x'), \tag{A.7}$$

i.e., V_{μ_k} satisfies the Bellman optimality equation. Since the Bellman operator has a unique fixed point, we have $V_{\mu_{k+1}} = V_{\mu_k} = V^*$. Therefore, μ_k and μ_{k+1} are both optimal policies.

□

Appendix B

Convergence of Q-Learning

In this chapter, we present a simplified proof of the convergence of Q-learning. Recall that Q-learning is an off-policy algorithm that learns the optimal policy from data generated from a behavior policy.

To prove the convergence of the learned Q-values to the Q-values of the optimal policy. Let $\tilde{Q}$ be the estimate of Q-values. Given an experience (current state $x_k = x$, current action $a_t = u$, received reward $r(x, u)$, and next state $x_{k+1} = x'$), Q-learning updates $\tilde{Q}$ as follows:

$$\begin{aligned}\tilde{Q}(x,u) \longleftarrow & (1-\beta_k)\tilde{Q}(x,u) + \beta_k(r(x,u) + \alpha \max_v \tilde{Q}(x',v)) \\ = & \tilde{Q}(x,u) + \beta_k \left(r(x,u) + \alpha \max_v \tilde{Q}(x',v) - \tilde{Q}(x,u)\right).\end{aligned}$$

Note that given an experience, Q-learning updates the Q-value for one state and action pair, which makes the analysis more challenging than the value-iteration and policy-iteration algorithms.

B.1 Batch Q-Learning

To simplify our proof, we will consider a modified Q-learning algorithm under which the algorithm uses N experiences at each iteration and updates all Q-values. The modified algorithm is still data-driven and updates the Q-function instead of the value-function. Therefore, the proof of the modified algorithm keeps the key ideas of the proof of the original Q-learning but avoid some technical details.

We name the modified algorithm batch-Q-learning, under which we update the Q-function with a batch of N experiences at each iteration. Before introducing the algorithm, we define the following notations.

- $\gamma_{x,u}$ is the fraction of experiences such that the current state is x and current action is u. We assume $\gamma_{x,u} > 0$ for any $(x, u) \in (\mathcal{S}, \mathcal{A})$.
- $\hat{p}_{x,x'}(u)$ is the fraction of experiences that the next state is x' given the current state and action are (x, u).

We further assume $r(x, u) \leq r_{\max}$ is deterministic and the learning rate is fixed and $Q_0(x, u) \leq \frac{r_{\max}}{1-\alpha}$ for all (x, u). The modified algorithm with learning rate β approximates the original Q-learning

with learning rate $\frac{\beta}{N}$. For the analysis purpose, we use $Q_k(x,u)$ to denote the *estimated* Q-value of state action pair (x,u) at iteration k. The Q table is updated as follows.

Batch Q-Learning

With a batch N experiences, first compute $\gamma_{i,u}$ and $\hat{p}_{ij}(u)$ for each i,j,u, and then update Q-values as follows:

$$Q_{k+1}(x,u) = (1-\beta)Q_k(x,u) + \beta\left(r(x,u) + \alpha\sum_{x'}\hat{p}_{x,x'}(u)\max_v Q_k(x',v)\right).$$

When β is small, the Q-values does not change much over N updates under the original Q-learning. So the original Q-learning behaves similarly as the batch Q-learning.

B.2 Convergence of Batch Q-Learning

Theorem B.2.1 *After k iterations,*

$$\mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right] \le \left(1-\beta+\beta^2\right)^k \mathbb{E}\left[\|Q_0 - Q^*\|_\infty^2\right] + \frac{c\beta}{1-\beta}\frac{1}{N},$$

where c is a constant independent of k and N.

Proof To prove the convergence, we will compare $\|Q_k - Q^*\|_\infty$ and $\|Q_{k+1} - Q^*\|_\infty$ and show that the value becomes smaller and decreases geometrically. We first note that according to the definition of batch Q-learning and Q-values, we have

$$\begin{aligned}
Q_{k+1}(x,u) =& (1-\beta)Q_k(x,u) + \beta\left(r(x,u) + \alpha\sum_{x'}\hat{p}_{x,x'}(u)\max_v Q_k(x',v)\right)\\
Q^*(x,u) =& (1-\beta)Q^*(x,u) + \beta\left(r(x,u) + \alpha\sum_{x'}p_{x,x'}(u)\max_v Q^*(x',v)\right).
\end{aligned}$$

From the two equations above, we obtain

$$\begin{aligned}
&Q_{k+1}(x,u) - Q^*(x,u)\\
=&(1-\beta)\left(Q_k(x,u) - Q^*(x,u)\right) + \alpha\beta\left(\sum_{x'}\hat{p}_{x,x'}(u)\max_v Q_k(x',v) - \sum_{x'}p_{x,x'}(u)\max_v Q^*(x',v)\right).
\end{aligned}$$

We next add and subtract the following term $\sum_j p_{ij}(u) \max_v Q_k(j,v)$ in the equation above:

$$\begin{aligned}
&Q_{k+1}(x,u) - Q^*(x,u) \\
=&(1-\beta)\left(Q_k(x,u) - Q^*(x,u)\right) \\
&+ \alpha\beta\left(\sum_{x'} p_{x,x'}(u) \max_v Q_k(x',v) - \sum_{x'} p_{x,x'}(u) \max_v Q^*(x',v)\right) \\
&+ \alpha\beta\left(\sum_{x'} \hat{p}_{x,x'}(u) \max_v Q_k(x',v) - \sum_{x'} p_{x,x'}(u) \max_v Q_k(x',v)\right) \\
=&(1-\beta)\left(Q_k(x,u) - Q^*(x,u)\right) + \alpha\beta \sum_{x'} \left(\hat{p}_{x,x'}(u) - p_{x,x'}(u)\right) \max_v Q_k(x',v) \\
&+ \alpha\beta \sum_{x'} p_{x,x'}(u) \left(\max_v Q_k(x',v) - \max_v Q^*(x',v)\right).
\end{aligned}$$

Define $W_{k,x,u} = \alpha\beta \sum_{x'} \left(\hat{p}_{x,x'}(u) - p_{x,x'}(u)\right) \max_v Q_k(x',v)$. We note that

$$\mathbb{E}\left[W_{k,x,u} | Q_k\right] = 0$$

because $\hat{p}_{x,x'}(u)$ is an unbiased estimator of $p_{x,x'}(u)$ for all (x,x',u), which further implies that

$$\begin{aligned}
&\mathbb{E}\left[W_{k,x,u}\right] = \mathbb{E}\left[\mathbb{E}\left[W_{k,x,u}|Q_k\right]\right] = 0 \\
&\mathbb{E}\left[W_{k,x,u}\alpha\beta \sum_{x'} p_{x,x'}(u) \left(\max_v Q_k(x',v) - \max_v Q^*(x',v)\right)\right] \\
=&\mathbb{E}\left[\mathbb{E}\left[W_{k,x,u}\alpha\beta \sum_{x'} p_{x,x'}(u) \left(\max_v Q_k(x',v) - \max_v Q^*(x',v)\right) | Q_k\right]\right] = 0.
\end{aligned}$$

Based on the results above, we have

$$\begin{aligned}
&\mathbb{E}\left[\left(Q_{k+1}(i,u) - Q^*(i,u)\right)^2\right] \\
=&(1-\beta)^2 \mathbb{E}\left[\left(Q_k(i,u) - Q^*(i,u)\right)^2\right] + \\
&\alpha^2\beta^2 \mathbb{E}\left[\left(\sum_j p_{ij}(u) \left(\max_v Q_k(j,v) - \max_v Q^*(j,v)\right)\right)^2\right] + \mathbb{E}[W_{k,x,u}^2] + \\
&(1-\beta)\alpha\beta \mathbb{E}\left[\left(Q_k(i,u) - Q^*(i,u)\right) \sum_j p_{ij}(u) \left(\max_v Q_k(j,v) - \max_v Q^*(j,v)\right)\right],
\end{aligned}$$

We note that

$$\left|\max_v Q_k(j,v) - \max_v Q^*(j,v)\right| \leq \max_v \left|Q_k(j,v) - Q^*(j,v)\right|.$$

The proof can be the later of this chapter (Lemma B.2.2), so for any (i,u), we have

$$\begin{aligned}
&\mathbb{E}\left[\left(Q_{k+1}(i,u) - Q^*(i,u)\right)^2\right] \\
\leq&(1-\beta)^2 \mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right] + \alpha^2\beta^2 \mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right] + \mathbb{E}[W_{k,x,u}^2] + (1-\beta)\alpha\beta \mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right].
\end{aligned}$$

The inequality above holds for any (i, u), so we have

$$\begin{aligned}
&\mathbb{E}\left[\|Q_{k+1} - Q^*\|_\infty^2\right] \\
\leq &\left((1-\beta)^2 + \alpha^2\beta^2 + (1-\beta)\alpha\beta\right)\mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right] + \max_{x,u}\mathbb{E}[W_{k,x,u}^2] \\
\leq &\left(1 - \beta + \beta^2\right)\mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right] + \mathbb{E}[W_{k,x,u}^2],
\end{aligned}$$

where the last inequality holds because $0 \leq \alpha < 1$. It is easy to verify that $\|Q_k\|_\infty \leq \frac{r_{\max}}{1-\alpha}$ for all k if $\|Q_0\|_\infty \leq \frac{r_{\max}}{1-\alpha}$

$$\begin{aligned}
\mathbb{E}[W_{k,x,u}^2] =&\mathbb{E}\left[\left(\alpha\beta\sum_{x'}\left(\hat{p}_{x,x'}(u) - p_{x,x'}(u)\right)\max_v Q_k(x', v)\right)^2\right] \\
\leq&\frac{\alpha^2\beta^2 r_{\max}^2}{(1-\alpha)^2}\mathbb{E}\left[\left(\sum_{x'}\left|\hat{p}_{x,x'}(u) - p_{x,x'}(u)\right|\right)^2\right] \\
\leq&\frac{\alpha^2\beta^2 r_{\max}^2}{(1-\alpha)^2}\left(\left(1 - \frac{|\mathcal{S}|}{\gamma_{\min}^2 N^2}\right)\frac{|\mathcal{S}|^2\log^2 N}{N} + \frac{|\mathcal{S}|}{\gamma_{\min}^2 N^2}|\mathcal{S}|^2\right),
\end{aligned}$$

where $\gamma_{\min} = \min_{x,u}\gamma_{x,u}$ and the last inequality holds due to Lemma B.2.3 and the fact that $\frac{\log N}{N}$ is a decreasing function in N for $N \geq 1$. For a sufficiently large N such that $N \geq \frac{|\mathcal{S}|}{\gamma_{\min}\log^2 N}$, we have

$$\mathbb{E}[W_{k,x,u}^2] \leq \frac{\alpha^2\beta^2 r_{\max}^2}{(1-\alpha)^2}\frac{2|\mathcal{S}|^2\log^2 N}{N} = \frac{c\beta^2\log^2 N}{N},$$

where c is a constant independent of N, β and k.

Define $\rho = 1 - \beta + \beta^2$. We have

$$\begin{aligned}
\mathbb{E}\left[\|Q_{k+1} - Q^*\|_\infty^2\right] \leq&\rho\mathbb{E}\left[\|Q_k - Q^*\|_\infty^2\right] + \frac{c\beta^2\log^2 N}{N} \\
\leq&\rho^2\mathbb{E}\left[\|Q_{k-1} - Q^*\|_\infty\right] + (1+\rho)\frac{c\beta^2\log^2 N}{N} \\
\leq&\rho^{k+1}\mathbb{E}\left[\|Q_0 - Q^*\|_\infty\right] + \left(\sum_{m=0}^{k}\rho^k\right)\frac{c\beta^2\log^2 N}{N} \\
\leq&\rho^{k+1}\mathbb{E}\left[\|Q_0 - Q^*\|_\infty\right] + \frac{c\beta^2\log^2 N}{(1-\rho)N} \\
=&(1-\beta+\beta^2))^{k+1}\mathbb{E}\left[\|Q_0 - Q^*\|_\infty\right] + \frac{c\beta\log^2 N}{(1-\beta)N}.
\end{aligned}$$

□

From the result above, as $k \to \infty$,

$$\lim_{k\to\infty}\mathbb{E}\left[\|Q_{k+1} - Q^*\|_\infty^2\right] \leq \frac{c\beta\log^2 N}{(1-\beta)N}.$$

If the learning rate β is small, then the learned Q values are very close to the optimal Q values. We also note that Q_k does not converge to Q^* when the learning rate β is fixed. Therefore, in practice, an adaptive learning rate $\beta_k \to 0$ is used so that Q_k converges to Q^* as $k \to \infty$.

Lemma B.2.2 *Given two functions $f(x)$ and $g(x)$,*

$$\left|\max_x f(x) - \max_x g(x)\right| \leq \max_x |f(x) - g(x)|.$$

We assume max *are well-defined in the inequality above.*

Proof Let $x_1^* \in \arg\max_x f(x)$ and $x_2^* \in \arg\max_x g(x)$. According to the definition, we have

$$\left|\max_x f(x) - \max_x g(x)\right| = |f(x_1^*) - g(x_2^*)|. \tag{B.1}$$

We next consider two cases.

Case 1: Suppose $f(x_1^*) \geq g(x_2^*)$. In this case,

$$\begin{aligned} |f(x_1^*) - g(x_2^*)| =& f(x_1^*) - g(x_2^*) \\ \leq& f(x_1^*) - g(x_1^*) \\ \leq& |f(x_1^*) - g(x_1^*)| \\ \leq& \max_x |f(x) - g(x)| \end{aligned} \tag{B.2}$$

where the first inequality follows from $x_2^* \in \arg\max_x g(x)$.

Case 2: Suppose $f(x_1^*) \leq g(x_2^*)$. In this case,

$$\begin{aligned} |f(x_1^*) - g(x_2^*)| =& g(x_2^*) - f(x_1^*) \\ \leq& g(x_2^*) - f(x_2^*) \\ \leq& |g(x_2^*) - f(x_2^*)| \\ \leq& \max_x |g(x) - f(x)|, \end{aligned} \tag{B.3}$$

where the first inequality follows from $x_1^* \in \arg\max_x f(x)$.

Hence, by (B.1), Case 1, and Case 2, we conclude $|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|$.

□

Lemma B.2.3 *Given $\tilde{N} = N\gamma_{x,u}$ experiences with current state x and current action u, with probability at least $1 - \frac{1}{\tilde{N}^2}$, we have*

$$\left|\hat{p}_{x,x'}(u) - p_{x,x'}(u)\right| \leq \frac{\log \tilde{N}}{\sqrt{\tilde{N}}}.$$

Proof Under the assumption of the lemma, we have $N\gamma_{x,u}$ experiences with the current state being x and the current action being u. Considering these $\tilde{N}$ experiences $(x, u, x'_m)_{m=1,\cdots,\tilde{N}}$, where x'_m is the next state in the mth experience, we then have

$$\hat{p}_{x,x'}(u) = \frac{\sum_{m=1}^{\tilde{N}} \mathbf{1}(x'_m = x')}{\tilde{N}},$$

where $\mathbf{1}(x'_m = x')$ is the indicator function such that $\mathbb{E}[\mathbf{1}(x'_m = x')] = p_{x,x'}(u)$ and $\mathbf{1}(x'_m = x')$ are i.i.d. Bernoulli random variables.

According to the Chernoff bound (Theorem B.2.4) with $\delta = \frac{\log \tilde{N}}{p_{x,x'}(u)\sqrt{\tilde{N}}}$, we conclude that

$$\mathbb{P}\left(\left|\hat{p}_{x,x'}(u) - p_{x,x'}(u)\right| \leq \frac{\log \tilde{N}}{\sqrt{\tilde{N}}}\right) \leq e^{-\frac{\log^2 \tilde{N}}{3p_{x,x'}(u)}}.$$

When $\tilde{N}$ is sufficiently large such that $\frac{\log \tilde{N}}{3p_{x,x'}(u)} \geq 2$, the lemma holds.

□

Theorem B.2.4 (Chernoff Bound) *Consider a collection of i.i.d. random variables $X_1, X_2, \ldots, X_N$ with mean μ. For any $x > \mu$, we have*

$$\mathbb{P}\left(\sum_{i=1}^{N} X_i \geq Nx\right) \leq e^{-N \sup_{\theta>0}(\theta x - \Lambda(\theta))},$$

where $\Lambda(\theta) = \log \mathbb{E}\left[e^{\theta X_i}\right]$ is the log moment generating function. If $X_1, \ldots, X_N$ are i.i.d. Bernoulli random variables, we further have that for any $0 < \delta < 1$,

$$\mathbb{P}\left(\left|\frac{\sum_{i=1}^{N} X_i}{N} - \mu\right| \leq \delta\mu\right) \leq e^{-\frac{\delta^2 N\mu}{3}}.$$

Appendix C

Monotonic Improvement Theorem

To prove this theorem, we consider the difference between $V_{\tilde{w}}(x_0)$ and $L_{\tilde{w}}(x_0)$. According to their definitions, we have

$$\begin{aligned} V_{\tilde{w}}(x_0) - L_{\tilde{w}}(x_0) =& \mathbb{E}_{x_k \sim \mathbb{P}_{\tilde{w}}(x_k|x_0), u_k \sim \pi_{\tilde{w}}(x_k)} \left[\sum_{k=0}^{\infty} \alpha^k \left(Q_w(x_k, u_k) - V_w(x_k) \right) \right] \\ & - \mathbb{E}_{x_k \sim \mathbb{P}_w(x_k|x_0), u_k \sim \pi_{\tilde{w}}(x_k)} \left[\sum_{k=0}^{\infty} \alpha^k \left(Q_w(x_k, u_k) - V_w(x_k) \right) \right] \\ =& \mathbb{E}_{x_k \sim \mathbb{P}_{\tilde{w}}(x_k|x_0)} \left[\sum_{k=0}^{\infty} \alpha^k \bar{A}(x_k) \right] - \mathbb{E}_{x_k \sim \mathbb{P}_w(x_k|x_0)} \left[\sum_{k=0}^{\infty} \alpha^k \bar{A}(x_k) \right] \end{aligned} \tag{C.1}$$

where

$$\bar{A}(x_k) = \sum_u \pi_{\tilde{w}}(u|x_k) \left(Q_w(x_k, u) - V_w(x_k) \right).$$

Now define $\epsilon = \max_x \mathrm{TV}(\pi_w(\cdot|x)) \| \pi_{\tilde{w}}(\cdot|x))$, where $\mathrm{TV}(\pi_w(\cdot|x)) \| \pi_{\tilde{w}}(\cdot|x)) = \frac{1}{2} \sum_u |(\pi_w(u|x)) - \pi_{\tilde{w}}(u|x)|$ is the total variation distance of two distributions. We note that

$$\begin{aligned} & \frac{1}{2} \sum_u |(\pi_w(u|x)) - \pi_{\tilde{w}}(u|x)| \\ =& \frac{1}{2} \sum_{u:(\pi_w(u|x)) \geq \pi_{\tilde{w}}(u|x)} (\pi_w(u|x) - \pi_{\tilde{w}}(u|x)) + \frac{1}{2} \sum_{u:(\pi_w(u|x)) < \pi_{\tilde{w}}(u|x)} (\pi_{\tilde{w}}(u|x) - \pi_w(u|x)) \\ =& \frac{1}{2} \sum_u (\pi_w(u|x)) - \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\} + \frac{1}{2} \sum_u (\tilde{\pi}(u|x)) - \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\} \\ =& 1 - \sum_u \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\}. \end{aligned}$$

Now given policies $\pi_w(u|x)$ and $\pi_{\tilde{w}}(u|x)$, we couple the episodes $\{x_k, u_k, r_k\}_k$ and $\{\tilde{x}_k, \tilde{u}_k, \tilde{r}_k\}_k$ generated under the two policies such that if $x_k = \tilde{x}_k = x$, then with probability $\sum_u \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\}$, we sample action u with distribution

$$\frac{\min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\}}{\sum_v \min\{\pi_w(v|x), \pi_{\tilde{w}}(v|x)\}}$$

and the action is taken by both policies; and with probability $1-\sum_u \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\}$, policy π_w samples action u with distribution

$$\frac{\pi_w(u|x)) - \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\}}{\sum_v(\pi_w(v|x)) - \min\{\pi_w(v|x), \pi_{\tilde{w}}(v|x)\}}$$

and policy $\pi_{\tilde{w}}$ samples action u with distribution

$$\frac{\pi_{\tilde{w}}(u|x)) - \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\}}{\sum_v(\pi_{\tilde{w}}(v|x)) - \min\{\pi_w(v|x), \pi_{\tilde{w}}(v|x)\}}.$$

With this coupling, we have that

$$\Pr\left(u_k \neq \tilde{u}_k | x_k = \tilde{x}_k\right) = 1 - \sum_u \min\{\pi_w(u|x), \pi_{\tilde{w}}(u|x)\} = \mathrm{TV}(\pi_w(\cdot|x)) \| \pi_{\tilde{w}}(\cdot|x)) \leq \epsilon.$$

Furthermore, if both policies take the same action in the same state, the next state and the reward received are the same, i.e.,

$$\Pr\left(x_{k+1} = \tilde{x}_{k+1}, r_k = \tilde{r}_k | x_k = \tilde{x}_k, u_k = \tilde{u}_k\right) = 1.$$

Define event $\phi_k = \left\{\{x_t, u_t, r_t\}_{t\leq k} = \{\tilde{x}_t, \tilde{u}_t, \tilde{r}_t\}_{t\leq k}\right\}$, i.e., the two episodes are identical. From the coupling constructed above, we have

$$\Pr(\phi_k) \geq (1-\epsilon)^k$$

which implies

$$\Pr(\phi_k^c) \leq 1 - (1-\epsilon)^k.$$

We know that

$$\mathbb{E}\left[\bar{A}(x_k)\right] - \mathbb{E}\left[\bar{A}(\tilde{x}_k)\right] = \Pr(\phi_k^c)\left(\mathbb{E}\left[\bar{A}(x_k)|\phi_k^c\right] - \mathbb{E}\left[\bar{A}(\tilde{x}_k)|\phi_k^c\right]\right). \tag{C.2}$$

Note that

$$\bar{A}(x_k) = \sum_u \pi_{\tilde{w}}(u|x_k)\left(Q_w(x_k,u) - V_w(x_k)\right) = \sum_u \left(\pi_{\tilde{w}}(u|x_k) - \pi_w(u|x_k)\right)\left(Q_w(x_k,u) - V_w(x_k)\right)$$

because $\sum_u \pi_w(u|x_k)\left(Q_w(x_k,u) - V_w(x_k)\right) = 0$ by definition. Therefore,

$$|\bar{A}(x_k)| \leq 2\epsilon A_{\max}, \tag{C.3}$$

where $A_{\max} = \max_{x,u} |Q_w(x,u) - V_w(x)|$.

Substituting (C.3) into (C.2), we have

$$\left|\mathbb{E}\left[\bar{A}(x_k)\right] - \mathbb{E}\left[\bar{A}(\tilde{x}_k)\right]\right| \leq 4\left(1-(1-\epsilon)^k\right)\epsilon A_{\max}. \tag{C.4}$$

Substituting the inequality above into (C.1), we obtain

$$\begin{aligned}
&|V_{\tilde{w}}(x_0) - L_{\tilde{w}}(x_0)| \leq \sum_{k=0}^{\infty} \alpha^k 4 \left(1 - (1-\epsilon)^k\right) \epsilon A_{\max} \\
=&4A_{\max}\epsilon \left(\frac{1}{1-\alpha} - \frac{1}{1-\alpha\,(1-\epsilon)}\right) \\
=&\frac{4\epsilon^2 \alpha A_{\max}}{(1-\alpha)(1-\alpha\,(1-\epsilon))} \\
\leq&\frac{4\epsilon^2 \alpha A_{\max}}{(1-\alpha)^2}.
\end{aligned}$$

We now note that the following relationship between the total variation distance and KL divergence

$$\mathrm{TV}(p\|q)^2 \leq \mathrm{KL}(p\|q),$$

so

$$|V_{\tilde{w}}(x_0) - L_{\tilde{w}}(x_0)| \leq \frac{4\alpha A_{\max} \mathrm{KL}_{\max}}{(1-\alpha)^2},$$

where $\mathrm{KL}_{\max} = \max_x \mathrm{KL}(\pi_w(\cdot|x))\|\pi_{\tilde{w}}(\cdot|x))$ or

$$V_{\tilde{w}}(x_0) \geq L_{\tilde{w}}(x_0) - \frac{4\alpha A_{\max} \mathrm{KL}_{\max}}{(1-\alpha)^2}.$$

Since there are two policies π_w (old policy) and $\pi_{\tilde{w}}$ (new policy) in the analysis above, we explicitly add w into the notation

$$V_{\tilde{w}}(x_0) \geq L_{\tilde{w},w}(x_0) - \frac{4\alpha A_{\max} \mathrm{KL}_{\max}}{(1-\alpha)^2}.$$

Define

$$M_{\tilde{w},w}(x_0) = L_{\tilde{w},w}(x_0) - \frac{4\alpha A_{\max} \mathrm{KL}_{\max}(\tilde{w}, w)}{(1-\alpha)^2}.$$

Note that $M_{w,w}(x_0) = L_{w,w}(x_0) = V_w(x_0)$ because $\mathrm{KL}_{\max}(w, w) = 0$. Since $V_{w_{k+1}}(x_0) \geq M_{w_{k+1},w_k}(x_0)$, we have

$$V_{w_{k+1}}(x_0) - V_{w_k}(x_0) \geq M_{w_{k+1},w_k}(x_0) - M_{w_k,w_k}(x_0).$$

Therefore, by maximizing $M_{\tilde{w},w_k}(x_0)$, we guarantee the performance is nondecreasing.

Appendix D

Key Lemmas for UCB-SARSA

Lemma 10.2.1: With a high probability, for any h, (x, a) and π,

$$Q_h^\pi(x,a) \leq \tilde{Q}_{k,h}(x,a),$$

where k is the episode index.

Proof For simplicity, we consider the case $H = 2$ to illustrate the proof idea. Note that $\tilde{Q}_{1,H} = H \geq r_H(x,u) \in [0,1]$, and $Q_H^\pi(x,u) = r_H(x,u)$. Therefore, if (x,u) is visited at step H of episode k, we have

$$\tilde{Q}_{k+1,H}(x,u) = (1-\beta_H)\tilde{Q}_{k,H}(x,u) + \beta_H(r_H(x,u) + b_{H-1}) \geq Q_H^\pi(x,u).$$

Consider $h = 1$, state action pair (x,u) and a general k. Assume that (x,u) was visited m times at episodes $t_1, t_2, \cdots, t_m$ before episode k. Considering the update of Q-table at time t_m, we have

$$\begin{aligned}
&Q_1^\pi(x,u) - \tilde{Q}_{k,1}(x,u) \\
=&(1-\beta_{t_m,1})Q_1^\pi(x,u)) + \beta_{t_m,1}(r(x,u) + \mathbb{E}\left[Q_2^\pi(x_2,\pi(x_2))\right]) && \text{(D.1)} \\
&-(1-\beta_{t_m,1})\tilde{Q}_{t_m,1}(x,u) - \beta_{t_m,1}(r(x,u) + \tilde{Q}_{t_m,2}(x_2,\pi_{t_m}(x_2)) + b_{k,1}) && \text{(D.2)} \\
=&(1-\beta_{t_m,1})\left(Q_1^\pi(x,u) - \tilde{Q}_{t_m,1}(x,u)\right) + \beta_{t_m,1}\left(\mathbb{E}\left[Q_2^\pi(x_2,\pi(x_2))\right] - \tilde{Q}_{t_m,2}(x_2,\pi_{t_m}(x_2)) - b_{t_m,1}\right)
\end{aligned}$$

where (D.1) is the Bellman equation under policy π and (D.2) follows from the optimistic update of UCB-SARSA algorithm for the Q-table. By iteratively expanding the difference based on the m updates, we obtain

$$\begin{aligned}
&Q_1^\pi(x,u) - \tilde{Q}_{k,1}(x,u) \\
=&\left(\prod_{l=1}^m(1-\beta_{t_l,1})\right)\left(Q_1^\pi(x,u) - \tilde{Q}_{1,1}(x,u)\right) + \sum_{l=1}^m \tilde{\beta}_{t_l,1}\left(\mathbb{E}\left[Q_2^\pi(x_2,\pi(x_2))\right] - \tilde{Q}_{t_l,2}(x_2,\pi_{t_l}(x_2)) - b_{t_l,1}\right),
\end{aligned}$$

where

$$\tilde{\beta}_{t_l,1} = \beta_{t_l,1}\left(\prod_{z=l+1}^m(1-\beta_{t_z,1})\right).$$

We note that (i) $\tilde{Q}_{1,1}(x,u) = H \geq Q_1^{\pi}(x,u)$ and (ii)

$$\tilde{Q}_{t_l,2}(x_2, \pi_{t_l}(x_2)) \geq_{(1)} \tilde{Q}_{t_l,2}(x_2, \pi(x_2)) \geq_{(2)} Q^{\pi}(x_2, \pi(x_2)),$$

where (1) holds because π_{t_l} is a greedy policy based on $\tilde{Q}_{t_l,2}$ and (2) holds because $\tilde{Q}_{k,2}$ is an optimistic estimate of Q-values.

Based on the results above, we obtain

$$Q_1^{\pi}(x,u) - \tilde{Q}_{k,1}(x,u) \leq \sum_{l=1}^{m} \tilde{\beta}_{t_l,1} \left(\mathbb{E}\left[Q_2^{\pi}(x_2, \pi(x_2)) \right] - Q_2^{\pi}(x_2, \pi(x_2)) - b_{t_l,1} \right)$$

It can be easily verified that

$$Y_m = \sum_{l=1}^{m} \tilde{\beta}_{t_l,1} \left(E\left[Q_2^{\pi}(x_2, \pi(x_2)) \right] - Q_2^{\pi}(x_2, \pi(x_2)) \right)$$

is a Martingale. We can then establish the lemma based on the Azuma-Hoeffding inequality to show that Y_m is bounded by $\sum_l b_{t_l,1}$ with a high probability based on the choice of the UCB bonus. More details can be found in [10].

□

Lemma 10.2.2: Under UCB-SARSA, we have

$$\sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1, \pi_k(x_1)) \right] - \sum_{k=1}^{K} \mathbb{E}\left[Q_1^{\pi_k}(x_1, \pi_k(x_1)) \right] = \tilde{\mathcal{O}}(\sqrt{K})$$

Proof Similar to the analysis in the proof of Lemma 10.2.1, we have

$$\begin{aligned}
&\tilde{Q}_{k,1}(x,u) - Q_1^{\pi^*}(x,u) \\
=&\left(\prod_{l=1}^{N_{k,1}(x,u)} (1 - \beta_{t_l,1}) \right) \left(\tilde{Q}_{1,1}(x,u) - Q_1^{\pi^*}(x,u) \right) \\
&+ \sum_{l=1}^{N_{k,1}(x,u)} \tilde{\beta}_{t_l,1} \left(\tilde{Q}_{t_l,2}(x_{t_l,2}, \pi_{t_l}(x_{t_l,2})) + b_{t_l,1} - E_{x_2}\left[Q_2^{\pi^*}(x_2, \pi^*(x_2)) \right] \right).
\end{aligned}$$

Summing up over k yields

$$\begin{aligned}
&\sum_{k=1}^{K} \tilde{Q}_{k,1}(x,u) - Q_1^{\pi^*}(x,u) \\
\leq&\left(\prod_{l=1}^{N_{k,1}(x,u)} (1 - \beta_{t_l,1}) \right) \left(\tilde{Q}_{1,1}(x,u) - Q_1^{\pi^*}(x,u) \right) + \sum_{k=1}^{K} \sum_{l=1}^{N_{k,1}(x,u)} \tilde{\beta}_{t_l,1} b_{t_l,1} \\
&+ (1 + \frac{1}{H}) \sum_{k=1}^{K} \left(\tilde{Q}_{k,2}(x_{k,2}, \pi_k(x_{k,2})) - E_{x_2}\left[Q_2^{\pi^*}(x_2, \pi^*(x_2)) \right] \right) \\
\leq& HSA + \tilde{\mathcal{O}}(\sqrt{H^2 SAK}) \\
&+ (1 + \frac{1}{H}) \sum_{k=1}^{K} \left(\tilde{Q}_{k,2}(x_{k,2}, \pi_k(x_{k,2})) - E_{x_2}\left[Q_2^{\pi^*}(x_2, \pi^*(x_2)) \right] \right)
\end{aligned}$$

Taking expectation yields

$$\begin{aligned}
&\sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x,u) - Q_1^{\pi^*}(x,u)\right] \\
\leq & HSA + \tilde{\mathcal{O}}(\sqrt{H^2SAK}) + (1+\frac{1}{H})\sum_{k=1}^{K}\left(\mathbb{E}[\tilde{Q}_{k,2}(x_2,\pi_k(x_2))] - \mathbb{E}\left[Q_2^{\pi^*}(x_2,\pi^*(x_2))\right]\right),
\end{aligned}$$

where the last inequality holds due to the design of UCB bonus. Details can be found in [10]. We note that

$$0 \leq Q_1^{\pi^*}(x,u) - Q_1^{\pi_k}(x,u) \leq \mathbb{E}[Q_2^{\pi^*}(x_2,\pi^*(x_2)) - Q_2^{\pi_k}(x_2,\pi_k(x_2))].$$

Therefore,

$$\begin{aligned}
R_K \leq & \sum_{k=1}^{K} \mathbb{E}\left[\tilde{Q}_{k,1}(x_1,\pi_k(x_1))\right] - \sum_{k=1}^{K}\mathbb{E}\left[Q_1^{\pi_k}(x_1,\pi_k(x_1))\right] \\
\leq & HSA + \tilde{\mathcal{O}}(\sqrt{H^2SAK}) + \left(1+\frac{1}{H}\right)\sum_{k=1}^{K}\left(\mathbb{E}[\tilde{Q}_{k,2}(x_2,\pi_k(x_2))] - \mathbb{E}\left[Q_2^{\pi_k}(x_2,\pi_k(x_2))\right]\right) \\
\leq & HSA + \tilde{\mathcal{O}}(\sqrt{H^2SAK}) + SA.
\end{aligned}$$

where the last equality holds because $\tilde{Q}_{k,2}(x,a) = Q_2^{\pi_k}(x,a) = r_2(x,a)$ if the state action pair (x,a) has been visited at least once.

□

Bibliography

[1] Introduction to reinforcement learning with examples and codes. `https://github.com/Multi-wavelength/Introduction-to-Reinforcement-Learning-with-Examples-and-Codes`, 2022.

[2] Alekh Agarwal, Sham Kakade, Akshay Krishnamurthy, and Wen Sun. Flambe: Structural complexity and representation learning of low rank mdps. In *nips*, page 20095–20107. Curran Associates Inc., 2020.

[3] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, May 2002.

[4] MJ Beckmann. Dynamic programming and the secretary problem. *Computers & Mathematics with Applications*, 19(11):25–28, 1990.

[5] Richard E. Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[6] Dimitri Bertsekas and John N Tsitsiklis. *Neuro Dynamic Programming*. Athena Scientific, 1996.

[7] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *ICML*, pages 1582–1591, 2018.

[8] Hado Hasselt. Double q-learning. *Advances Neural Information Processing Systems (NeurIPS)*, 23, 2010.

[9] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[10] Chi Jin, Zeyuan Allen-Zhu, Sebastien Bubeck, and Michael I Jordan. Is q-learning provably efficient? In *Advances Neural Information Processing Systems (NeurIPS)*, volume 31, pages 4863–4873, 2018.

[11] Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Int. Conf. Machine Learning (ICML)*, pages 267–274, 2002.

[12] Sham M. Kakade. A natural policy gradient. In *Advances Neural Information Processing Systems (NeurIPS)*, 2001.

[13] Hans Kellerer, Ulrich Pferschy, and David Pisinger. Introduction. In *Knapsack problems*, pages 1–14. Springer, 2004.

[14] T.L Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 1985.

[15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[16] Martin L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, Inc., 1994.

[17] Gavin A Rummery and Mahesan Niranjan. *On-line Q-learning using connectionist systems*, volume 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.

[18] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Int. Conf. Machine Learning (ICML)*, pages 1889–1897. PMLR, 2015.

[19] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[20] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[21] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning*, 2014.

[22] Nima Taghipour, Ahmad Kardan, and Saeed Shiry Ghidary. Usage-based web recommendations: a reinforcement learning approach. In *Proceedings of the 2007 ACM conference on Recommender systems*, pages 113–120, 2007.

[23] Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 Connectionist Models Summer School Hillsdale, NJ. Lawrence Erlbaum*, volume 6, pages 1–9, 1993.

[24] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.

[25] Christopher John Cornish Hellaby Watkins. *Learning from Delayed Rewards*. PhD thesis, King's College, Cambridge, UK, May 1989.

[26] Richard Weber. Optimization and control. Course notes, `http://www.statslab.cam.ac.uk/~rrw1/oc/oc2016.pdf`, January 2016. [Online; accessed 16-Nov-2022].

[27] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.